MOST IMPORTANT FIRST | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB

Google study path (clean full code) - EASY only

38 Easy questions in 22 groups (the same code-template groups as the full book, only the Easy questions). Each group shares ONE code template, shown on the page before its questions.

38	0	0
EASY	MEDIUM	HARD

Sections (tap one to jump to it)

1. Frontier search (BFS or DFS), Tier 1: spread, count and measure distance — 1 question | page 5
2. Tree recursion, Tier 1: one function that visits a node, asks its children, a... — 3 questions | page 13
3. Runs and groups: extend while it fits, restart or close when it does not — 2 questions | page 34
4. A dict of per-key state that you update, expire, or rank — 1 question | page 45
5. Binary search on a yes/no condition — 1 question | page 53
6. Greedy: scan once, carry the best choice so far, act only when forced — 3 questions | page 60
7. Sets and counters: remember the first collection, check the second again... — 4 questions | page 77
8. bisect on a sorted list: find a position, count a range, keep it sorted — 1 question | page 99
9. Prefix sums, Tier 1: precompute once, answer ranges and balance point... — 3 questions | page 106
10. Sorted intervals: merge while they touch, then read the gaps or search — 1 question | page 122
11. A dict for lookup plus an ordered container for who is next (lazy deletion) — 1 question | page 130
12. Rolling state: the next state comes only from the previous one — 2 questions | page 141
13. Two pointers: move i and j toward each other (or along two lists) — 4 questions | page 156
14. Walk two strings at once with two indices — 3 questions | page 180
15. Simulate a machine: a class with state and one branch per kind of event — 1 question | page 198
16. Digits and characters: peel one at a time and carry — 1 question | page 206
17. Group by a canonical key — 1 question | page 213
18. K-way merge: a heap of the next item from each source — 1 question | page 220

Sections, continued (tap one to jump to it)

19. Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up	1 question | page 228
20. Parent pointers: store who is above each node and climb	1 question | page 236
21. Build or transform a grid with index arithmetic	1 question | page 245
22. Split into pieces, fix each piece, join (and pack words into lines)	1 question | page 252

- The master contents lists every main topic and question; each row is a link.
- Inside a topic, the bar at the top of each page moves Prev / Contents / Next; the link at the bottom returns here.
- Questions run Easy, then Medium, then Hard inside each main topic, newest date first.

Go to Contents >

Contents

38 questions | tap a topic, a main topic or a question to jump to it

START HERE: topics in the order of importance, the most important question first inside each

1. Frontier search (BFS or DFS), Tier 1: spread, count and measure distance (1) Go >

E 1. Flood Fill — **ColorStack 2026-01-28**

2. Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upw... Go >

E 2. Find right-side view of binary tree — **PracHub 2025-11-01**

E 3. Given two trees, determine if the pre-order traversals of two trees are same or not. — **same or not. Exponent no date**

E 4. Calculate the height of a binary tree. — **Exponent no date**

3. Runs and groups: extend while it fits, restart or close when it does not (2) Go >

E 5. First Out-of-Order Character — **ColorStack 2026-01-30**

E 6. Group Timestamps Into Close Intervals — **ColorStack 2025-10-03**

4. A dict of per-key state that you update, expire, or rank (1) Go >

E 7. Robot Status Message Deduplication Window — **1Point3Acres 2026-09-10**

5. Binary search on a yes/no condition (1) Go >

E 8. First Index Reaching a Threshold (Binary Search) — **ColorStack 2025-10-11**

6. Greedy: scan once, carry the best choice so far, act only when forced (3) Go >

E 9. Alarm for the Next Upcoming Birthday — **ColorStack 2025-10-07**

E 10. Coins on the Table — **ColorStack 2025-09-20**

E 11. Maximum Profit — **Exponent no date**

7. Sets and counters: remember the first collection, check the second against it (4) Go >

E 12. Exclude First-K-Unique From Another List — **ColorStack 2025-10-17**

E 13. Positional Matches Between Two Lists — **ColorStack 2025-10-15**

E 14. Contains Duplicate — **ColorStack 2025-09-16**

E 15. Find the Duplicates — **Exponent no date**

8. bisect on a sorted list: find a position, count a range, keep it sorted (1) Go >

E 16. Count People in a Postal Code Range — **ColorStack 2025-10-19**

9. Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) (3) Go >

E 17. Cake Balance Line — **ColorStack 2025-09-26**

E 18. Find the Pivot Index (Balanced Split) in O(n) — **ColorStack 2025-09-21**

E 19. Parity Alternating Subarray Query — **HackTheRounds no date**

10. Sorted intervals: merge while they touch, then read the gaps or search (1) Go >

E 20. Check an Interval Against Existing Intervals for Conflicts — **PracHub 2026-08-08**

Contents

38 questions | tap a topic, a main topic or a question to jump to it

11. A dict for lookup plus an ordered container for who is next (lazy deletion) (1) Go >

E 21. Implement Batched Undo/Redo Layer — PracHub 2026-02-04

12. Rolling state: the next state comes only from the previous one (2) Go >

E 22. Climbing Stairs — Exponent no date

E 23. Fibonacci Numbers — Exponent no date

13. Two pointers: move i and j toward each other (or along two lists) (4) Go >

E 24. Reverse a linked list. — Exponent no date

E 25. Sort a binary array in linear time. — Exponent no date

E 26. Is this a valid palindrome? — Exponent no date

E 27. Reverse a Sentence — Exponent no date

14. Walk two strings at once with two indices (3) Go >

E 28. Find the First Occurrence of a Substring — ColorStack 2025-09-16

E 29. Determine if two words are sorted in lexicographic order. — Exponent no date

E 30. Return all unique substrings of a given string. — Exponent no date

15. Simulate a machine: a class with state and one branch per kind of event (1) Go >

E 31. Mario-Style Score Tracker (OOP) — ColorStack 2025-10-11

16. Digits and characters: peel one at a time and carry (1) Go >

E 32. Caesar Cipher: Recover Key, Encrypt, and Decrypt — ColorStack 2026-02-08

17. Group by a canonical key (1) Go >

E 33. Given an array of strings, return the words that can be typed using letter... — letters of the alphabet on only o... Exponent no date

18. K-way merge: a heap of the next item from each source (1) Go >

E 34. Find k pairs with smallest sums — PracHub 2025-11-01

19. Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up (1) Go >

E 35. Next Birthday (BST In-Order Successor with Wraparound) — ColorStack 2025-10-17

20. Parent pointers: store who is above each node and climb (1) Go >

E 36. Tree Node. — Exponent no date

21. Build or transform a grid with index arithmetic (1) Go >

E 37. Filled ASCII Diamond — ColorStack 2025-09-26

22. Split into pieces, fix each piece, join (and pack words into lines) (1) Go >

E 38. Implement string formatting and request tracking tasks — PracHub 2026-02-11

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 1 OF 22 |

Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 1 questions | tap a title to jump to it

READ FIRST: the code template for these 1 questions (tap here) >

E 1. Flood Fill ColorStack 2026-01-28

CODE TEMPLATE | GROUP 1 OF 22 | 1 QUESTIONS, ONE PATTERN

Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

The one idea these questions share

Every one of these keeps a frontier of cells or nodes still to visit, takes one out, looks at its neighbours, and adds the new ones. A dict of distances doubles as the seen set. Change what you start from, which neighbours are allowed, and what you do when a node comes out.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import deque


def frontier_search(starts, neighbors, usable=lambda v: True):
    dist = {s: 0 for s in starts}          # doubles as the seen set
    frontier = deque(starts)               # popleft = BFS, pop = DFS
    while frontier:
        u = frontier.popleft()
        # CHANGE 1: act on u here (repaint it, count it, stop at the goal)
        for v in neighbors(u):             # CHANGE 2: grid steps or adj[u]
            if v not in dist and usable(v):    # CHANGE 3: colour, wall, blocked
                dist[v] = dist[u] + 1
                frontier.append(v)
    return dist
```

2. Helper functions that usually get written next to it

```
def grid_steps(rows, cols):
    def step(cell):
        r, c = cell
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols:
                yield (nr, nc)
    return step


def rebuild_path(parent, target):         # walk parent[] back from the target
    path = [target]
    while parent[path[-1]] is not None:
        path.append(parent[path[-1]])
    return path[::-1]
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
1	Flood Fill	starts = [(sr, sc)]; usable = same colour as the start; repaint every cell you reach; return early if old colour == new colour

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 1 of 38 | topic 1 of 22: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 1 of 1

Template fit: starts = [(sr, sc)]; usable = same colour as the start; repaint every cell you reach; return early if old colour == new colour | asked 265 days ago | frequency Single report

EASY | IN APP LC 733 | ColorStack | technical interview | 1 of 38

Flood Fill

ASKED 2026-01-28 | 265 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 242/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

Optimal answer from: your app's interview approach (LeetCode #733)

LeetCode match: LC 733 Flood Fill

Frequency: Single report | Last asked: 2026-01-28 | Roles: SWE Intern

QUESTION

You are given an m x n grid image where image[r][c] is the integer color of the pixel at row r , column c . You are also given a starting pixel (sr, sc) and a new_color . Perform a flood fill: starting from (sr, sc) , repaint that pixel and every pixel connected to it 4-directionally (up, down, left, right) that shares the SAME original color as the starting pixel, changing them all to new_color . Pixels connected only diagonally are not filled. Return the modified grid.

Function signature

```
from typing import List
def flood_fill(image: List[List[int]], sr: int, sc: int, new_color: int) ->
List[List[int]]:
    ...
```

Follow-up (as reported): Implement with DFS (recursive or explicit stack) or BFS; discuss the base case that prevents re-visiting when new_color equals the original color.

EXAMPLES

Example 1	
Input	`image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, new_color = 2`
Output	`[[2,2,2],[2,2,0],[2,0,1]]`
Why	`Starting color is 1. All 1-pixels connected 4-directionally to (1,1) become 2. The bottom-right 1 is only diagonally reachable, so it stays.`

Example 2	
Input	`image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, new_color = 5`
Output	`[[5,5,5],[5,5,5]]`
Why	`The whole grid is one connected region of color 0.`

Example 3	
Input	`image = [[0,0,0],[0,1,0]], sr = 1, sc = 1, new_color = 1`
Output	`[[0,0,0],[0,1,0]]`
Why	`The start pixel already equals new_color, so nothing changes.`

CONSTRAINTS

- 1 <= m, n <= 50

QUESTION

- 0 <= image[r][c], new_color < 2^16
- 0 <= sr < m , 0 <= sc < n
- If the starting pixel already has new_color , the grid is returned unchanged (guard against infinite recursion).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_flood(image, sr, sc, new_color):
    old = image[sr][sc]
    marked = {(sr, sc)}
    changed = True
    while changed:
        changed = False
        for r in range(len(image)):
            for c in range(len(image[0])):
                if (
                    image[r][c] == old
                    and (r, c) not in marked
                ):
                    near = (
                        (r + 1, c),
                        (r - 1, c),
                        (r, c + 1),
                        (r, c - 1),
                    )
                    if any(
                        p in marked
                        for p in near
                    ):
                        marked.add((r, c))
                        changed = True
    return [
        [
            (
                new_color
                if (r, c) in marked
                else:
                    v
            )
            for c, v in enumerate(row)
        ]
        for r, row in enumerate(image)
    ]
```

TIME	SPACE
O((mn)^2)	O(1)

WHY IT IS SLOW

Each sweep touches every pixel even if only one more pixel changes.

OPTIMAL SOLUTION · Iterative DFS with a guard

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it with BFS. Why is the early return needed?

A deque gives BFS with the same cost. If the new colour equals the old one, repainting never changes the test 'pixel has the old colour', so pixels would be pushed again forever; the early return (or a visited set) stops that.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Flood Fill
#
# You are given an m x n grid image where image[r][c] is
# the integer color of the pixel at row r , column c . You
# are also given a starting pixel (sr, sc) and a new_color
# . Perform a flood fill: starting from (sr, sc) , repaint
# that pixel and every pixel connected to it
# 4-directionally (up, down, left, right) that shares the
# SAME original color as the starting pixel, changing them
# all to new_color . Pixels connected only diagonally are
# not filled. Return the modified grid. Function
# signature: from typing import List def flood_fill(image:
# List[List[int]], sr: int, sc: int, new_color: int) ->
# List[List[int]]: ... Follow-up (as reported): Implement
# with DFS (recursive or explicit stack) or BFS; discuss
# the base case that prevents re-visiting when new_color
# equals the original color.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it with BFS. Why is the early return needed?
#
# INTERVIEW NOTES
# Remember the starting colour, then spread from the start
# pixel to its four neighbours that have that colour,
# repainting as you go. If the new colour equals the old
# colour there is nothing to do, and returning early also
# prevents an infinite loop. An explicit stack avoids
# Python's recursion limit.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import copy


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
```

```
    # the cells above, below, left and right of (r, c) that exist
    result = []
    for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result
```

```python
# ---- main question: main function ----
# Idea: spread over neighbours that still have the old colour and
# repaint them. If the new colour equals the old one there is nothing
# to do (and the loop below would never finish), so return at once.
# Complexity: Time O(m * n), Space O(m * n)
def flood_fill(image, sr, sc, new_color):
    old = image[sr][sc]
    if old == new_color:
        return image
    rows, cols = len(image), len(image[0])
    image[sr][sc] = new_color
    stack = [(sr, sc)]
    while stack:
        r, c = stack.pop()
        for nr, nc in neighbors(r, c, rows, cols):
            if image[nr][nc] == old:
                image[nr][nc] = new_color
                stack.append((nr, nc))
    return image


# ---- main question: tests ----
IMAGE = [[1, 1, 1], [1, 1, 0], [1, 0, 1]]
FILLED = [[2, 2, 2], [2, 2, 0], [2, 0, 1]]


def test_main_question():
    assert flood_fill(copy.deepcopy(IMAGE), 1, 1, 2) == FILLED
    assert flood_fill(copy.deepcopy(IMAGE), 1, 1, 1) == IMAGE      # same colour: nothing changes


# ========================================================
# FOLLOW-UP 1: Do it with BFS. Why is the early return needed?
# ========================================================
# ---- follow-up 1: function ----
# Answer: a queue (BFS) costs the same. If the new colour equals the
# old one, repainting never changes the test "has the old colour", so
# cells would be pushed again forever; the early return (or a visited
# set) stops that.
# Complexity: Time O(m * n), Space O(m * n)
def flood_fill_bfs(image, sr, sc, new_color):
    old = image[sr][sc]
    if old == new_color:
        return image
    rows, cols = len(image), len(image[0])
    image[sr][sc] = new_color
    queue = deque([(sr, sc)])
    while queue:
        r, c = queue.popleft()
        for nr, nc in neighbors(r, c, rows, cols):
            if image[nr][nc] == old:
                image[nr][nc] = new_color
                queue.append((nr, nc))
    return image


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert flood_fill_bfs(copy.deepcopy(IMAGE), 1, 1, 2) == FILLED
    assert flood_fill_bfs(copy.deepcopy(IMAGE), 1, 1, 1) == IMAGE


# ========================================================
```

```
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 2 OF 22 |

Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 3 questions | tap a title to jump to it

READ FIRST: the code template for these 3 questions (tap here) >

E 2. Find right-side view of binary tree — **PracHub 2025-11-01**

E 3. Given two trees, determine if the pre-order traversals of two trees are same or not. — **Exponent no date**

E 4. Calculate the height of a binary tree. — **Exponent no date**

CODE TEMPLATE | GROUP 2 OF 22 | 3 QUESTIONS, ONE PATTERN

Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

The one idea these questions share

Write dfs(node, depth). An empty node returns a base value, a real node calls dfs on both children, updates the best answer so far, and returns one number or flag to its parent. Only three lines change: the base value, the update, and what you return.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def solve(root):
    best = 0

    def dfs(node, depth=0):
        nonlocal best
        if node is None:
            return 0                          # CHANGE 1: what an empty tree reports
        left = dfs(node.left, depth + 1)
        right = dfs(node.right, depth + 1)
        best = max(best, left + right)        # CHANGE 2: update the answer at this node
        return 1 + max(left, right)           # CHANGE 3: what this node reports upward

    dfs(root)
    return best
```

2. Helper functions that usually get written next to it

```
from collections import deque


class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def levels(root):                             # BFS by level: height, right-side view, zigzag
    out, queue = [], deque([root] if root else [])
    while queue:
        out.append([n.val for n in queue])
        for _ in range(len(queue)):
            n = queue.popleft()
            queue.extend(c for c in (n.left, n.right) if c)
    return out


def preorder(root):                           # iterative: push right first, then left
    stack = [root] if root else []
    while stack:
        n = stack.pop()
        yield n.val
        stack.extend(c for c in (n.right, n.left) if c)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
2	Find right-side view of binary tree	pass depth down; the first node you reach at a new depth (visiting right before left) is the one you can see

#	Question	What you change in the template
3	Given two trees, determine if the pre-order traversals of two trees are same or not.	walk both trees in preorder with an explicit stack (push right, then left) and compare value by value; stop at the first difference
4	Calculate the height of a binary tree.	no update needed; return 1 + max(left, right) and read the value at the root

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 2 of 38 | topic 2 of 22: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 1 of 3

Template fit: pass depth down; the first node you reach at a new depth (visiting right before left) is the one you can see | asked 353 days ago

EASY | IN APP LC 199 | PracHub | Technical Screen | 2 of 38

Find right-side view of binary tree

POSTED 2025-11-01 | 353 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 34.9 | 30 DAYS 0 | PRIORITY 128/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #199 Binary Tree Right Side View

LeetCode match: LC 199 Binary Tree Right Side View

Posted on PracHub: 2025-11-01 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/find-right-side-view-of-binary-tree

QUESTION

You are given the root of a binary tree. From the **right side** of the tree, at each depth you can see exactly one node: the rightmost node at that depth. Write a function that returns a list of the values of the nodes visible when looking at the tree from the right side, ordered from **top to bottom**. ---

Function Signature

(one possible form)

```
rightSideView(root: TreeNode) -> List[int]
```

Where **`TreeNode`** is defined as:

```
class TreeNode { int val; TreeNode left; TreeNode right; }
```

Example

Consider this binary tree:

```
1 / \ 2 3 \ \ 5 4
```

The right-side view is:

```
[1, 3, 4]
```

Explain the algorithm you would use, including its time and space complexity. You may use either breadth-first search (BFS) or depth-first search (DFS).

EXAMPLES

Example 1	
Input	`[1, 2, 3, null, 5, null, 4]`
Output	`[1, 3, 4]`
Why	`Looking from the right you see 1, then 3 (hiding 2), then 4 (hiding 5).`

QUESTION

Example 2	
Input	`[1, 2]`
Output	`[1, 2]`
Why	`A left child is visible when there is no right child at that depth.`

Example 3	
Input	`[]`
Output	`[]`
Why	`An empty tree shows nothing.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def right_side_view(root):
    levels = level_order(root)  # list of lists
    return [level[-1] for level in levels]
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Stores every node's value when only one per level is needed, O(n) space. A depth-first walk visiting the right child first and recording the first value at each depth needs O(h).

OPTIMAL SOLUTION · DFS visiting right before left, first node at each depth

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Left side view, or both.

Swap the order of the two recursive calls. For both views record the first and last node per depth in one pass.

FOLLOW-UP 2 Avoid recursion.

Use the BFS version and keep only the last node per level (no per-level list), or an explicit stack of (node, depth) pushing left before right so that right is popped first.

FOLLOW-UP 3 Top view or bottom view.

Track a horizontal distance per node (left minus one, right plus one) and keep the first (top) or last (bottom) node seen per distance in BFS order.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find right-side view of binary tree
#
# You are given the root of a binary tree. From the
# **right side** of the tree, at each depth you can see
# exactly one node: the rightmost node at that depth.
# Write a function that returns a list of the values of
# the nodes visible when looking at the tree from the
# right side, ordered from **top to bottom**. --- ###
# Function Signature (one possible form) text
# rightSideView(root: TreeNode) -> List[int] Where
# TreeNode is defined as: text class TreeNode { int val;
# TreeNode left; TreeNode right; } --- ### Example
# Consider this binary tree: text 1 / \ 2 3 \ \ 5 4 The
# right-side view is: text [1, 3, 4] Explain the algorithm
# you would use, including its time and space complexity.
# You may use either breadth-first search (BFS) or depth-
# first search (DFS).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Left side view, or both.
# 2. Avoid recursion.
# 3. Top view or bottom view.
#
# INTERVIEW NOTES
# The right side view is the last node of every level, or
# equivalently the first node you reach at each depth if
# you visit right children before left. Either way you
# need the depth of each node, so carry it in the
# traversal.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
```

```
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ---- main question: main function ----
# Idea: visit the RIGHT child before the left one and pass the depth
# down. The first node met at a depth that is not recorded yet is the
# rightmost node of that depth. Memory is O(height).
# Complexity: Time O(n), Space O(h)
def right_side_view(root):
    view = []
```

```
    def visit(node, depth):
        if node is None:
            return
        if depth == len(view):
            view.append(node.val)                # first node seen at this depth
        visit(node.right, depth + 1)
        visit(node.left, depth + 1)

    visit(root, 0)
    return view


# ---- main question: tests ----
def example_tree():
    #     1
    #    / \
    #   2   3
    #    \    \
    #     5    4
    return TreeNode(1, TreeNode(2, None, TreeNode(5)), TreeNode(3, None, TreeNode(4)))


def test_main_question():
    assert right_side_view(example_tree()) == [1, 3, 4]
    assert right_side_view(None) == []
    chain = TreeNode(1, TreeNode(2, TreeNode(3)))            # a left-only tree is still seen from the rig
ht
    assert right_side_view(chain) == [1, 2, 3]


# =====================================================
# FOLLOW-UP 1: Left side view, or both
```

```
# =====================================================
# ---- follow-up 1: function ----
# Answer: for the left view swap the order of the two recursive calls.
# For both views at once, go level by level and take the first and the
# last node of each level.
# Complexity: Time O(n), Space O(w)
def left_and_right_views(root):
    left_view = []
    right_view = []
    queue = deque()
    if root:
        queue.append(root)
    while queue:
        left_view.append(queue[0].val)
        right_view.append(queue[-1].val)
        for i in range(len(queue)):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return left_view, right_view


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert left_and_right_views(example_tree()) == ([1, 2, 5], [1, 3, 4])
    assert left_and_right_views(None) == ([], [])


# =====================================================
```

```
# FOLLOW-UP 2: Avoid recursion
# ========================================================
# ---- follow-up 2: function ----
# Answer: go level by level with a queue and keep only the LAST node of
# every level (or use your own stack of (node, depth) and push the left
# child first so the right one is taken first).
# Complexity: Time O(n), Space O(w)
def right_side_view_bfs(root):
    view = []
    queue = deque()
    if root:
        queue.append(root)
    while queue:
        view.append(queue[-1].val)
        for i in range(len(queue)):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return view


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert right_side_view_bfs(example_tree()) == [1, 3, 4]
    assert right_side_view_bfs(None) == []


# ========================================================
# FOLLOW-UP 3: Top view or bottom view
# ========================================================
# ---- follow-up 3: function ----
# Answer: give every node a horizontal distance (left child: -1, right
# child: +1 compared with its parent) and go level by level. The FIRST
# node seen at a distance is the top view there, the LAST one is the
# bottom view.
# Complexity: Time O(n + c log c) for c columns, Space O(n)
def top_and_bottom_view(root):
    top = {}
    bottom = {}
    queue = deque()
    if root:
        queue.append((root, 0))
    while queue:
        node, distance = queue.popleft()
        if distance not in top:
            top[distance] = node.val
        bottom[distance] = node.val
        if node.left:
            queue.append((node.left, distance - 1))
        if node.right:
            queue.append((node.right, distance + 1))
    top_view = []
    bottom_view = []
    for distance in sorted(top):
        top_view.append(top[distance])
        bottom_view.append(bottom[distance])
    return top_view, bottom_view
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
```

```
    top, bottom = top_and_bottom_view(example_tree())
    assert top == [2, 1, 3, 4] and bottom == [2, 5, 3, 4]


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def rightSideView(level_order: list[int | None]) -> list[int]:
```

Sample call

```
rightSideView([1, 2, 3, None, 5, None, 4])
```

Test cases

Input	Expected
`([1, 2, 3, None, 5, None, 4],)`	`[1, 3, 4]`
`([1, 2, 3, 4],)`	`[1, 3, 4]`

STUDY PATH: question 3 of 38 | topic 2 of 22: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 2 of 3

Template fit: walk both trees in preorder with an explicit stack (push right, then left) and compare value by value; stop at the first difference | no date

EASY | Exponent | 3 of 38

Given two trees, determine if the pre-order traversals of two trees are same or not.

ASKED (ANY CO.) ~A YEAR AGO | OLDER

STUDY: TYPE | ALL TIME 45.8 | 30 DAYS 0 | PRIORITY 91/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #100 Same Tree

LeetCode match: LC 100 Same Tree (related: traversal comparison) (inferred from short statement)

Last asked (any company): a year ago, about 2025-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/7182/compare-preorder-traversals-two-trees

QUESTION

Given two binary trees, determine whether their pre-order traversals (node, then left subtree, then right subtree) produce the same sequence of values. The trees themselves may have different shapes.

Function signature

def same_preorder(a: TreeNode, b: TreeNode) -> bool

EXAMPLES

Example 1	
Input	`[1, 2, 3], [1, 2, 3]`
Output	`true`
Why	`Identical trees.`

Example 2	
Input	`[1, 2, null, 3], [1, null, 2, null, 3]`
Output	`true`
Why	`Different shapes (a chain going left versus a chain going right) but both read 1, 2, 3.`

Example 3	
Input	`[1, 2, 3], [1, 3, 2]`
Output	`false`
Why	`1, 2, 3 versus 1, 3, 2.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def preorder(node):
    if node is None:
        return []
    left = preorder(node.left)
    right = preorder(node.right)
    return [node.val] + left + right
same = preorder(a) == preorder(b)
```

TIME	SPACE
O(n + m)	O(n + m)

WHY IT IS SLOW

It always visits and stores both trees completely, O(n + m) time and space. Walking both pre-orders lazily in lockstep stops at the first difference with O(h) space.

OPTIMAL SOLUTION · Lazy pre-order generators compared in lockstep

TIME	SPACE
O(min(n, m)) with early exit, O(n + m) worst case	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if the interviewer meant the trees must be identical?

Then compare structure as well as values: same_tree above walks both trees together and treats two missing nodes as equal. Equivalently, serialize pre-order with null markers and compare the strings, since pre-order with nulls is unique. Ask which definition they mean before coding.

FOLLOW-UP 2 Do it with O(1) extra space.

Morris traversal visits pre-order using temporary threads on right pointers, giving O(1) space, but it mutates the tree while running (restoring it after), so it is not safe for concurrent readers. Mention that trade-off.

FOLLOW-UP 3 They want identical trees, not just sequences.

Compare structure as well: walk both trees together and treat a missing child on one side only as a mismatch, or serialise pre-order with null markers and compare the strings.

FOLLOW-UP 4 They want identical trees (structure and values), not just the same pre-order values.

Compare the two trees together: a node must match in value and both children must match; two missing children match. Equivalently compare the pre-order serialisation with null markers (which is unique). O(min(n, m)) with early exit.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given two trees, determine if the pre-
# order traversals of two trees are same or not.
#
# Given two binary trees, determine whether their pre-
# order traversals (node, then left subtree, then right
# subtree) produce the same sequence of values. The trees
# themselves may have different shapes.
#
# Signature:
#   def same_preorder(a: TreeNode, b: TreeNode) -> bool
#
# Examples:
# [1, 2, 3], [1, 2, 3]  ->  true
# [1, 2, null, 3], [1, null, 2, null, 3]  ->  true
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if the interviewer meant the trees must be
# identical?
# 2. Do it with O(1) extra space.
# 3. They want identical trees, not just sequences.
# 4. They want identical trees (structure and values), not
# just the same pre-order values.
#
# INTERVIEW NOTES
# Ask whether they want identical pre-order value
# sequences or identical trees; different trees can share
# a pre-order. For sequences, compare lazily so the first
# difference ends the work, instead of building two full
# lists.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def push_children(stack, node):
    # right goes in first so that the LEFT child comes out first (pre-order)
    if node.right:
        stack.append(node.right)
    if node.left:
        stack.append(node.left)


# ---- main question: main function ----
# Idea: walk both trees in pre-order at the same time with two stacks
# and compare the values one by one. The first mismatch stops the work
# and no list of values is built. If one walk ends before the other,
# the sequences differ.
# Complexity: Time O(n), Space O(h)
def same_preorder(a, b):
    stack_a = []
    stack_b = []
```

```
    if a:
        stack_a.append(a)
    if b:
        stack_b.append(b)
    while stack_a and stack_b:
        x = stack_a.pop()
        y = stack_b.pop()
        if x.val != y.val:
            return False
        push_children(stack_a, x)
        push_children(stack_b, y)
    return not stack_a and not stack_b


# ---- main question: tests ----
def two_example_trees():
    t1 = TreeNode(1, TreeNode(2), TreeNode(3))
    t2 = TreeNode(1, TreeNode(2, None, TreeNode(3)))       # a different shape, same pre-order 1, 2, 3
    return t1, t2


def test_main_question():
    t1, t2 = two_example_trees()
    assert same_preorder(t1, t2)
    assert same_preorder(None, None)
```

```
    assert not same_preorder(t1, None)
    assert not same_preorder(t1, TreeNode(1, TreeNode(2)))


# ========================================================
# FOLLOW-UP 1: What if the interviewer meant the trees must be identical?
# ========================================================


# ========================================================
# FOLLOW-UP 3: They want identical trees, not just sequences
# ========================================================


# ========================================================
# FOLLOW-UP 4: They want identical trees (structure and values), not just the same pre-order values
# ========================================================
# ---- follow-up 1: functions ----
# Answer: compare structure as well as values: walk both trees together;
# a node must match in value, and both children must match; two missing
# nodes match, one missing node does not. Equivalently, write each tree
# out in pre-order with a marker ("#") for every missing child; that text
# is unique, so equal text means identical trees.
# Complexity: Time O(n), Space O(h)
def same_tree(a, b):
    stack = [(a, b)]
    while stack:
        x, y = stack.pop()
        if x is None and y is None:
            continue
        if x is None or y is None or x.val != y.val:
            return False
        stack.append((x.left, y.left))
        stack.append((x.right, y.right))
    return True


# Complexity: Time O(n), Space O(n)
def serialise(root):
```

```
    parts = []
    stack = [root]
    while stack:
        node = stack.pop()
        if node is None:
            parts.append("#")
```

```
            continue
        parts.append(str(node.val))
        stack.append(node.right)
        stack.append(node.left)
    return ",".join(parts)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    t1, t2 = two_example_trees()
    assert not same_tree(t1, t2)
    assert serialise(t1) != serialise(t2)
    assert same_tree(t1, TreeNode(1, TreeNode(2), TreeNode(3)))
    assert same_tree(None, None)
    assert serialise(t1) == "1,2,#,#,3,#,#"


# =====================================================
# FOLLOW-UP 2: Do it with O(1) extra space
# =====================================================
# ---- follow-up 2: functions ----
MISSING = object()          # "this walk has ended"


# Answer: Morris traversal visits the nodes in pre-order using a
# temporary link from the rightmost node of the left subtree back to the
# node, so no stack is needed. It changes the tree while it runs (and puts
# it back at the end), so it is not safe for other readers at the same
# time: mention that trade-off. The generator must be run to the end,
# otherwise a temporary link is left in the tree.
# Complexity: Time O(n), Space O(1)
def morris_preorder(root):
    node = root
    while node:
        if node.left is None:
            yield node.val
            node = node.right
            continue
        pred = node.left                          # the last node visited inside the left subtree
        while pred.right and pred.right is not node:
            pred = pred.right
        if pred.right is None:
            yield node.val
            pred.right = node                     # leave a link to come back
            node = node.left
```

```
        else:
            pred.right = None                     # second visit: remove the link
            node = node.right


# Complexity: Time O(n), Space O(1)
def same_preorder_constant_space(a, b):
    one = morris_preorder(a)
    two = morris_preorder(b)
    same = True
    while True:
        x = next(one, MISSING)
```

```
        y = next(two, MISSING)
        if x is MISSING and y is MISSING:
            break
        if x != y:
            same = False                        # never stop early: that would leave links in the trees
    return same


# ---- follow-up 2: tests ----
def test_follow_up_2():
    t1, t2 = two_example_trees()
    assert list(morris_preorder(t2)) == [1, 2, 3]
    assert same_preorder_constant_space(t1, t2)
    assert not same_preorder_constant_space(t1, TreeNode(1, TreeNode(9), TreeNode(3)))
    assert t1.left.right is None and t1.right.left is None        # the trees are left exactly as they were


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 4 of 38 | topic 2 of 22: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 3 of 3

Template fit: no update needed; return 1 + max(left, right) and read the value at the root | no date

EASY | IN APP LC 104 | Exponent | 4 of 38

Calculate the height of a binary tree.

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TYPE | ALL TIME 40 | 30 DAYS 33 | PRIORITY 63/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

Optimal answer from: your app's interview approach (LeetCode #104)

LeetCode match: LC 104 Maximum Depth of Binary Tree (inferred from short statement)

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/1153/calculate-height-binary-tree

QUESTION

Given the root of a binary tree, return its height: the number of nodes on the longest path from the root down to a leaf (an empty tree has height 0).

Function signature

def height(root: TreeNode) -> int

EXAMPLES

Example 1	
Input	`[3, 9, 20, null, null, 15, 7]`
Output	`3`
Why	`Levels: [3], [9, 20], [15, 7]: height 3.`

Example 2	
Input	`[1, 2]`
Output	`2`
Why	`A root with one child.`

Example 3	
Input	`[]`
Output	`0`
Why	`An empty tree.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def height(node):
    if node is None:
        return 0
    left = height(node.left)
    right = height(node.right)
    return 1 + max(left, right)
```

BRUTE FORCE

TIME	SPACE
O(n)	O(h) call stack

WHY IT IS SLOW

Linear time, which is optimal; the call stack is as deep as the tree, so a chain of 100,000 nodes can overflow. Counting levels with a queue avoids recursion.

OPTIMAL SOLUTION · Level-order traversal counting levels

TIME	SPACE
O(n)	O(width)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Nodes or edges?

Height in nodes is what the code returns; in edges subtract one (an empty tree is -1 and a single node is 0). Say which you use before you code.

FOLLOW-UP 2 Diameter (the longest path between any two nodes).

Return the height from the recursion and update a global best with left + right at each node: one pass, O(n).

FOLLOW-UP 3 Is the tree balanced?

Return the height, or -1 as soon as a subtree is unbalanced or the heights differ by more than one, so the whole check is one O(n) pass.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Calculate the height of a binary tree.
#
# Given the root of a binary tree, return its height: the
# number of nodes on the longest path from the root down
# to a leaf (an empty tree has height 0).
#
# Signature:
#   def height(root: TreeNode) -> int
#
# Examples:
# [3, 9, 20, null, null, 15, 7]  ->  3
# [1, 2]  ->  2
#
# FOLLOW-UPS (answered below, in this order):
# 1. Nodes or edges?
# 2. Diameter (the longest path between any two nodes).
# 3. Is the tree balanced?
#
# INTERVIEW NOTES
# Height is defined recursively: 1 plus the larger child
# height, and 0 for an empty tree. Say whether you count
```

```
# nodes or edges. A level-order traversal gives the same
# number without recursion, which is the safe choice for a
# very deep tree.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

```
# ---- main question: main function ----
# Idea: process the tree one level at a time with a queue and count the
# levels. No recursion, so a deep tree cannot overflow the stack; the
# memory used is the widest level. Height is counted in NODES: an empty
# tree is 0 and a single node is 1 (follow-up 1 explains edges).
# Complexity: Time O(n), Space O(w) for the widest level w
def height(root):
    levels = 0
    queue = deque()
    if root:
        queue.append(root)
    while queue:
        levels += 1
        for i in range(len(queue)):               # exactly the nodes of this level
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return levels


# ---- main question: tests ----
def example_tree():
    return TreeNode(3, TreeNode(9), TreeNode(20, TreeNode(15), TreeNode(7)))


def test_main_question():
    assert height(example_tree()) == 3
    assert height(None) == 0
    assert height(TreeNode(1)) == 1
    chain = TreeNode(1, TreeNode(2, TreeNode(3)))
    assert height(chain) == 3


# ============================================================
# FOLLOW-UP 1: Nodes or edges?
# ============================================================
# ---- follow-up 1: function ----
# Answer: height in nodes is what height() returns; in edges subtract one
```

```
# (an empty tree is -1, a single node is 0). Say which one you use
# before you code.
# Complexity: Time O(n), Space O(w)
def height_edges(root):
    return height(root) - 1


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert height_edges(example_tree()) == 2
    assert height_edges(None) == -1
    assert height_edges(TreeNode(1)) == 0


# ========================================================
# FOLLOW-UP 2: Diameter (the longest path between any two nodes)
# ========================================================
# ---- follow-up 2: function ----
# Answer: the recursion returns the height of a subtree, and at every
# node the best path THROUGH it is left height + right height. Keep the
# biggest such sum: one pass, O(n). The diameter here is in edges.
# Complexity: Time O(n), Space O(h)
def diameter(root):
    best = 0

    def depth(node):
        nonlocal best
        if node is None:
            return 0
        left = depth(node.left)
        right = depth(node.right)
        if left + right > best:
            best = left + right
        if left > right:
            return 1 + left
        return 1 + right

    depth(root)
    return best


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert diameter(example_tree()) == 3
    assert diameter(None) == 0
    assert diameter(TreeNode(1)) == 0


# ========================================================
```

```
# FOLLOW-UP 3: Is the tree balanced?
# ========================================================
# ---- follow-up 3: function ----
# Answer: at EVERY node the heights of the two sides differ by at most
# one. Return the height, or -1 as soon as a subtree is unbalanced, so
# the whole check is one O(n) pass.
# Complexity: Time O(n), Space O(h)
def is_balanced(root):
    def depth(node):
        if node is None:
            return 0
        left = depth(node.left)
        if left == -1:
            return -1
```

```
        right = depth(node.right)
        if right == -1:
            return -1
        difference = left - right
        if difference < 0:
            difference = -difference
        if difference > 1:
            return -1
        if left > right:
            return 1 + left
        return 1 + right

    return depth(root) != -1


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert is_balanced(example_tree())
    assert is_balanced(None)
    chain = TreeNode(1, TreeNode(2, TreeNode(3)))
    assert not is_balanced(chain)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 3 OF 22 |

Runs and groups: extend while it fits, restart or close when it does not

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

2
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Runs and groups: extend while it fits, restart or close when it does not | 2 questions | tap a title to jump to it

READ FIRST: the code template for these 2 questions (tap here) >

E 5. First Out-of-Order Character **ColorStack 2026-01-30**

E 6. Group Timestamps Into Close Intervals **ColorStack 2025-10-03**

CODE TEMPLATE | GROUP 3 OF 22 | 2 QUESTIONS, ONE PATTERN

Runs and groups: extend while it fits, restart or close when it does not

The one idea these questions share

Walk the sequence once with a counter. If the next item fits the rule, the run grows; if not, the run restarts (or the group closes). Longest run, first break, number of groups, and sums over runs are all small changes to what you record.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def longest_run(a, fits):
    run = best = 1 if a else 0
    for i in range(1, len(a)):
        # CHANGE 1: the rule for continuing a run
        run = run + 1 if fits(a[i - 1], a[i]) else 1
        best = max(best, run)            # CHANGE 2: record longest, first break, count, sum
    return best
```

2. Helper functions that usually get written next to it

```
def groups_of(items, fits_group):          # close a group when the next item does not fit
    groups, current = [], []
    for x in items:
        if current and not fits_group(current, x):
            groups.append(current)
            current = []
        current.append(x)
    if current:
        groups.append(current)
    return groups


def breaks(a, fits):                       # every index where the rule fails
    return [i for i in range(1, len(a)) if not fits(a[i - 1], a[i])]
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
5	First Out-of-Order Character	fits = s[i] >= s[i-1]; return i at the first item that does not fit
6	Group Timestamps Into Close Intervals	close the group when the gap to the next timestamp is at least k; otherwise extend it

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 5 of 38 | topic 3 of 22: Runs and groups: extend while it fits, restart or close when it does not | 1 of 2

Template fit: fits = s[i] >= s[i-1]; return i at the first item that does not fit | asked 263 days ago | frequency Single report

EASY | ColorStack | technical interview | 5 of 38

First Out-of-Order Character

ASKED 2026-01-30 | 263 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 241/288

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1752 Check if Array Is Sorted and Rotated (related: first violation)

Frequency: Single report | Last asked: 2026-01-30 | Roles: SWE Intern

QUESTION

You are given a string of lowercase English letters. Scanning left to right, the string is considered "in order" as long as each character is greater than or equal to the character before it (a non-decreasing sequence; equal adjacent characters are allowed). Return the index of the FIRST character that breaks this rule — that is, the first index i >= 1 where s[i] < s[i-1] . If the entire string is non-decreasing, return -1 .

Function signature

```
def first_out_of_order(s: str) -> int:
    ...
```

Follow-up (as reported): Solve in a single O(N) pass with O(1) extra space; state the time/space complexity as the interviewer expected.

EXAMPLES

Example 1	
Input	`"abcda"`
Output	`4`
Why	`a<=b<=c<=d holds, but at index 4 'a' < 'd' (s[3]), so 4 is returned.`

Example 2	
Input	`"aabbz"`
Output	`-1`
Why	`Every character is >= the previous one; fully non-decreasing.`

Example 3	
Input	`"cba"`
Output	`1`
Why	`At index 1, 'b' < 'c', the first violation.`

Example 4	
Input	`""`
Output	`-1`
Why	`An empty string is trivially in order.`

CONSTRAINTS

QUESTION

- 0 <= len(s) <= 10^5 s consists only of lowercase English letters.
- An empty string or a single-character string is trivially in order -> return -1 .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_first_break(s):
    for i in range(1,_len(s)):
        if list(s[:
            i + 1]) != sorted(
            s[: i + 1]
        ):
            return i
    return -1
```

TIME	SPACE
O(n^2 log n)	O(n)

WHY IT IS SLOW

It sorts every prefix just to learn one comparison.

OPTIMAL SOLUTION · One pass, compare with the previous letter

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Report every break and the longest in-order run.

Collect each index where s[i] < s[i - 1]; the runs are the gaps between breaks, so the longest run is the largest difference between consecutive break positions (with the ends included).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: First Out-of-Order Character
#
# You are given a string of lowercase English letters.
# Scanning left to right, the string is considered "in
# order" as long as each character is greater than or
# equal to the character before it (a non-decreasing
# sequence; equal adjacent characters are allowed). Return
# the index of the FIRST character that breaks this rule —
# that is, the first index i >= 1 where s[i] < s[i-1] . If
# the entire string is non-decreasing, return -1 .
# Function signature: def first_out_of_order(s: str) ->
# int: ... Follow-up (as reported): Solve in a single O(N)
# pass with O(1) extra space; state the time/space
# complexity as the interviewer expected.
#
# FOLLOW-UPS (answered below, in this order):
```

```python
# 1. Report every break and the longest in-order run.
#
# INTERVIEW NOTES
# Compare each character with the one before it as you
# scan. The first index where s[i] < s[i - 1] is the
# answer, and no other state is needed: one pass, constant
# extra space.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: one pass, comparing each letter with the one before it. The
# order is broken exactly where a letter is SMALLER than the previous
# letter; return that index, or -1 if the string is in order.
# Complexity: Time O(n), Space O(1)
def first_out_of_order(s):
    for i in range(1, len(s)):
        if s[i] < s[i - 1]:
            return i
    return -1


# ---- main question: tests ----
def test_main_question():
    assert first_out_of_order("abcda") == 4
    assert first_out_of_order("aabbz") == -1
    assert first_out_of_order("cba") == 1
    assert first_out_of_order("") == -1


# ========================================================
# FOLLOW-UP 1: Report every break and the longest in-order run
# ========================================================
# ---- follow-up 1: function ----
# Answer: collect every index where s[i] < s[i - 1]. The in-order runs are
# the stretches between breaks, so the longest run is the biggest gap
# between neighbouring break positions (counting the two ends of the string).
# Complexity: Time O(n), Space O(b) for b breaks
def order_breaks(s):
    breaks = []
    for i in range(1, len(s)):
        if s[i] < s[i - 1]:
            breaks.append(i)
    return breaks


# Complexity: Time O(n), Space O(b)
def longest_in_order_run(s):
    edges = [0] + order_breaks(s) + [len(s)]
    longest = 0
    for i in range(len(edges) - 1):
        if edges[i + 1] - edges[i] > longest:
            longest = edges[i + 1] - edges[i]
```

```
    return longest


# ---- follow-up 1: tests ----
def longest_run_slowly(s):
    # check by walking the string once more, counting the current run
    longest = 0
    run = 0
    for i in range(len(s)):
        if i > 0 and s[i] >= s[i - 1]:
            run += 1
        else:
            run = 1
        if run > longest:
            longest = run
    return longest


def test_follow_up_1():
    assert order_breaks("abcda") == [4]
    random.seed(84)
    for round_number in range(400):
        s = ""
        for i in range(random.randint(0, 9)):
            s += random.choice("abcd")
        assert longest_in_order_run(s) == longest_run_slowly(s)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 6 of 38 | topic 3 of 22: Runs and groups: extend while it fits, restart or close when it does not | 2 of 2

Template fit: close the group when the gap to the next timestamp is at least k; otherwise extend it | asked 382 days ago | frequency Single report

EASY | ColorStack | technical interview | 6 of 38

Group Timestamps Into Close Intervals

ASKED 2025-10-03 | 382 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.9 | 30 DAYS 33 | PRIORITY 69/288

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #228 Summary Ranges

LeetCode match: Array scan / grouping (related: LC 228 Summary Ranges)

Frequency: Single report | Last asked: 2025-10-03 | Roles: SWE Intern

QUESTION

You are given an array of positive integer timestamps in ascending order and an integer k . Partition the timestamps into maximal runs such that within each run, the difference between every pair of adjacent values is strictly less than k . A run breaks wherever an adjacent difference is >= k . Return each run as an interval [start, end] (its first and last value), in order.

Function signature

```
def group_into_intervals(timestamps: list[int], k: int) -> list[list[int]]:
    ...
```

Follow-up (as reported): One pass, O(n) time and O(1) extra space (besides the output).

EXAMPLES

Example 1	
Input	`timestamps = [1,2,3,4,11,14,15,17,21,22,23], k = 2`
Output	`[[1,4],[11,11],[14,15],[17,17],[21,23]]`
Why	`Adjacent diffs of 1 stay in a run; a diff of >=2 (e.g. 4->11, 11->14, 15- >17, 17->21) starts a new run.`

Example 2	
Input	`timestamps = [5], k = 100`
Output	`[[5,5]]`
Why	`A single timestamp forms a run by itself.`

Example 3	
Input	`timestamps = [1,2,3], k = 1`
Output	`[[1,1],[2,2],[3,3]]`
Why	`A diff of 1 is not strictly less than k=1, so every element stands alone.`

CONSTRAINTS

- 1 <= len(timestamps) <= 10^5 , values strictly increasing.
- k >= 1 .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_group(ts, k):
    if not ts:
        return []
    cuts = (
        [0]
        + [
            i
            for i in range(1, len(ts))
            if ts[i] - ts[i - 1] >= k
        ]
        + [len(ts)]
    )
    return [
        [ts[a], ts[b - 1]]
        for a, b in zip(cuts, cuts[1:])
    ]
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

It builds a cut list although a single running start suffices.

OPTIMAL SOLUTION · One pass with a run start

TIME	SPACE
O(n)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A run must also never last k seconds or more from its first value (a session limit).

Compare against the run's start: close the run when cur - start >= k (the neighbour gap rule is then implied). One pass again.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Group Timestamps Into Close Intervals
#
# You are given an array of positive integer timestamps in
# ascending order and an integer k . Partition the
# timestamps into maximal runs such that within each run,
# the difference between every pair of adjacent values is
# strictly less than k . A run breaks wherever an adjacent
# difference is >= k . Return each run as an interval
# [start, end] (its first and last value), in order.
# Function signature: def group_into_intervals(timestamps:
# list[int], k: int) -> list[list[int]]: ... Follow-up (as
# reported): One pass, O(n) time and O(1) extra space
# (besides the output).
#
```

```
# FOLLOW-UPS (answered below, in this order):
# 1. A run must also never last k seconds or more from its
# first value (a session limit).
#
# INTERVIEW NOTES
# An adjacent difference of at least k cuts the sorted
# list into runs. One left-to-right pass remembers where
# the current run started and closes it whenever the next
# gap is k or more; nothing else is needed, so extra space
# is O(1) besides the output.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: one pass. Remember where the current run started; whenever the
# gap to the previous timestamp reaches k, close the run [start, previous]
# and start a new one at this timestamp.
# Complexity: Time O(n), Space O(n) for the output
def group_into_intervals(timestamps, k):
    if not timestamps:
        return []
    runs = []
    start = timestamps[0]
    for i in range(1, len(timestamps)):
        previous = timestamps[i - 1]
        current = timestamps[i]
        if current - previous >= k:
            runs.append([start, previous])
            start = current
    runs.append([start, timestamps[len(timestamps) - 1]])
    return runs


# ---- main question: tests ----
def test_main_question():
    result = group_into_intervals([1, 2, 3, 4, 11, 14, 15, 17, 21, 22, 23], 2)
    assert result == [[1, 4], [11, 11], [14, 15], [17, 17], [21, 23]]
    assert group_into_intervals([5], 100) == [[5, 5]]
    assert group_into_intervals([1, 2, 3], 1) == [[1, 1], [2, 2], [3, 3]]
    assert group_into_intervals([], 3) == []


# ========================================================
# FOLLOW-UP 1: A run must also never last k seconds or more from its first value (a session limit)
# ========================================================
# ---- follow-up 1: function ----
# Answer: compare with the run's START instead of the previous value:
# close the run when current - start >= k (that also covers the
# neighbour-gap rule). Still one pass.
# Complexity: Time O(n), Space O(n) for the output
def group_by_span(timestamps, k):
    if not timestamps:
        return []
```

```
    runs = []
    start = timestamps[0]
    previous = timestamps[0]
    for i in range(1, len(timestamps)):
        current = timestamps[i]
        if current - start >= k:
            runs.append([start, previous])
            start = current
        previous = current
    runs.append([start, previous])
    return runs
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert group_by_span([1, 2, 3, 4, 5, 6], 3) == [[1, 3], [4, 6]]
    random.seed(45)
    for round_number in range(400):
        timestamps = sorted(random.sample(range(1, 40), random.randint(0, 9)))
        k = random.randint(1, 6)
        for first, last in group_by_span(timestamps, k):
            assert last - first < k                        # no run lasts k seconds or more


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 4 OF 22 |

A dict of per-key state that you update, expire, or rank

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

A dict of per-key state that you update, expire, or rank | 1 questions | tap a title to jump to it

READ FIRST: the code template for these 1 questions (tap here) >

E 7. Robot Status Message Deduplication Window **1Point3Acres 2026-09-10**

CODE TEMPLATE | GROUP 4 OF 22 | 1 QUESTIONS, ONE PATTERN

A dict of per-key state that you update, expire, or rank

The one idea these questions share

Keep a dict from a key (message, user, word, activity) to the little bit of state you need: the last time seen, a Counter, a set, a list of timestamps. Every event updates one key's state; the rule that decides what to accept, expire or report is the only part that changes.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
class KeyedState:
    def __init__(self, window):
        self.window = window
        self.last = {}                          # CHANGE 1: what you keep per key

    def accept(self, t, key):
        if key in self.last and t - self.last[key] < self.window:   # CHANGE 2: the rule
            return False
        self.last[key] = t                      # CHANGE 3: update this key's state
        return True
```

2. Helper functions that usually get written next to it

```
from collections import defaultdict, Counter, deque


def group_by(pairs):                        # key -> list of values, in arrival order
    out = defaultdict(list)
    for k, v in pairs:
        out[k].append(v)
    return out


def hits_in_window(times, now, span=300):
    q = deque(times)
    while q and q[0] <= now - span:
        q.popleft()
    return len(q)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
7	Robot Status Message Deduplication Window	the template as written: show a message only if it was not shown in the last 10 seconds

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 7 of 38 | topic 4 of 22: A dict of per-key state that you update, expire, or rank | 1 of 1

Template fit: the template as written: show a message only if it was not shown in the last 10 seconds | asked 40 days ago | frequency Low

EASY | 1Point3Acres | onsite-coding | 7 of 38

Robot Status Message Deduplication Window

ASKED 2026-09-10 | 40 DAYS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 51.2 30 DAYS 33 PRIORITY 41/288

ABOUT THIS QUESTION

Topic: A dict of per-key state that you update, expire, or rank

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter

LeetCode match: LC 359 Logger Rate Limiter (related)

Frequency: Low | Last asked: 2026-09-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/robot-status-message-deduplication

QUESTION

Status events arrive from a robot as (time, message). Show a message only if the same message was not shown during the previous 10 seconds; a hidden message does not restart the timer, and exactly 10 seconds later it is shown again. show(t, msg) returns True when the message should be displayed.

Function signature

class Dedup:

    def show(self, t: int, msg: str) -> bool

EXAMPLES

Example 1

Input	`d = Dedup()`
	`d.show(0, 'low battery')`
	`d.show(4, 'low battery')`
	`d.show(6, 'door open')`
	`d.show(9, 'low battery')`
	`d.show(10, 'low battery')`

Output	`d.show(0, 'low battery') -> true`
	`d.show(4, 'low battery') -> false`
	`d.show(6, 'door open') -> true`
	`d.show(9, 'low battery') -> false`
	`d.show(10, 'low battery') -> true`

Why	The repeats at 4 and 9 are hidden and do not move the timer; at time 10 it is exactly 10 seconds after the last display, so it shows again.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
shown = []  # (time, message) displayed
def on_event(t, msg):
    for t0, m0 in shown:
        if m0 == msg and t - t0 < 10:
            return False  # hidden
    shown.append((t, msg))
    return True  # displayed
```

BRUTE FORCE

TIME	SPACE
O(displayed) per event	**O(displayed)**

WHY IT IS SLOW

Each event scans the whole history. A map from message to its last displayed time answers each event in O(1).

OPTIMAL SOLUTION · Hash map of last displayed time

TIME	SPACE
O(1) per event	**O(distinct messages)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Events can arrive out of order.

Decide the contract first: either a late event is judged against the stored time with abs(t - last), or you keep a sorted history per message and compare with the nearest displayed neighbours on both sides. The simple map is only correct for a chronological stream.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Robot Status Message Deduplication Window
#
# Status events arrive from a robot as (time, message).
# Show a message only if the same message was not shown
# during the previous 10 seconds; a hidden message does
# not restart the timer, and exactly 10 seconds later it
# is shown again. show(t, msg) returns True when the
# message should be displayed.
#
# Signature:
#   class Dedup:
#       def show(self, t: int, msg: str) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. Events can arrive out of order.
#
# INTERVIEW NOTES
# The state that matters is the last time each message was
# displayed, not the last time it arrived. A message that
# is hidden must not move that time, otherwise a message
# repeated every few seconds would be hidden forever. A
# hash map from message to last displayed time answers
# each event in O(1).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================
```

```
# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
# Complexity: Time O(log n), Space O(1)
def first_at_least(items, x):
    # binary search in a sorted list: the first index whose value is >= x
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle] < x:
            low = middle + 1
        else:
```

```
            high = middle
    return low


# Complexity: Time O(log n), Space O(1)
def first_greater(items, x):
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle] <= x:
            low = middle + 1
        else:
            high = middle
    return low


# ---- main question: main class ----
# show(t, msg) says whether the message may be displayed now: not if it was
# displayed less than `window` seconds ago. Events arrive in time order.
# Idea: a dictionary with the time each message was last DISPLAYED. If the
# gap is still inside the window, hide the message WITHOUT changing the stored
# time (only a displayed event starts a new window); otherwise display it and
# store the new time. boundary_allowed says whether exactly `window` seconds later
# may show.
# Complexity: Time show O(1), Space O(distinct messages)
class Dedup:
    def __init__(self, window=10, boundary_allowed=True):
        self.window = window
        self.last = {}
        self.boundary_allowed = boundary_allowed

    def too_soon(self, gap):
        if self.boundary_allowed:
            return gap < self.window
        return gap <= self.window

    def show(self, t, msg):
        last = self.last.get(msg)
        if last is not None and self.too_soon(t - last):
            return False
        self.last[msg] = t
        return True
```

```
# ---- main question: tests ----
def test_main_question():
    d = Dedup()
    answers = []
```

```
    for t in (10, 13, 21):
        answers.append(d.show(t, "solar"))
    assert answers == [True, False, True]                # 13 is hidden and does not move the window
    d = Dedup()
    d.show(0, "a")
    assert d.show(10, "a")                               # exactly 10 seconds later shows by default
    d = Dedup(boundary_allowed=False)
    d.show(0, "a")
    assert not d.show(10, "a")                           # ... but is hidden if the boundary is not allowe
d


# =====================================================
# FOLLOW-UP 1: Events can arrive out of order
# =====================================================
# ---- follow-up 1: function ----
# Answer: decide the contract first. One option: judge a late event against
# the stored time with the absolute difference. Better: keep a SORTED list of
# the displayed times per message and check whether ANY displayed time is
# within `window` seconds on either side (two binary searches).
# Complexity: Time show is O(log n) to search and O(n) for the sorted insert, Space O(shown messages)
class DedupOutOfOrder:
    def __init__(self, window=10):
        self.window = window
        self.shown = {}                                  # message -> sorted list of displayed times

    def show(self, t, msg):
        if msg not in self.shown:
            self.shown[msg] = []
        times = self.shown[msg]
        low = first_greater(times, t - self.window)  # the first displayed time later than t - window
        high = first_at_least(times, t + self.window)
        if low < high:
            return False                                 # a display within the window on one of the two sides
        times.insert(first_at_least(times, t), t)
        return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    dedup = DedupOutOfOrder()
```

```
    assert dedup.show(20, "x")
    assert not dedup.show(15, "x")                       # a late event close to a displayed one is hidden
    assert dedup.show(5, "x")                            # a late event far from every display shows


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand. Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 5 OF 22 |

Binary search on a yes/no condition

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Binary search on a yes/no condition | 1 questions | tap a title to jump to it

READ FIRST: the code template for these 1 questions (tap here) >

E 8. First Index Reaching a Threshold (Binary Search) **ColorStack 2025-10-11**

CODE TEMPLATE | GROUP 5 OF 22 | 1 QUESTIONS, ONE PATTERN

Binary search on a yes/no condition

The one idea these questions share

If answers go no, no, no, yes, yes, yes as the value grows, binary search finds the first yes. Write ok(x), set lo and hi, and halve. The questions only differ in what x is (a length, a speed, a time, an index, a version) and how ok(x) is checked.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def first_true(lo, hi, ok):                    # smallest x in [lo, hi) with ok(x); hi if none
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid):                            # CHANGE: the yes/no test
            hi = mid
        else:
            lo = mid + 1
    return lo
```

2. Helper functions that usually get written next to it

```
def last_true(lo, hi, ok):                     # largest x in [lo, hi] with ok(x); lo - 1 if none
    return first_true(lo, hi + 1, lambda x: not ok(x)) - 1


def ceil_div(a, b):
    return -(-a // b)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
8	First Index Reaching a Threshold (Binary Search)	x = an index; ok(x) = arr[x] >= k on a sorted array

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 8 of 38 | topic 5 of 22: Binary search on a yes/no condition | 1 of 1

Template fit: x = an index; ok(x) = arr[x] >= k on a sorted array | asked 374 days ago | frequency Single report

EASY | IN APP LC 35 | ColorStack | technical interview | 8 of 38

First Index Reaching a Threshold (Binary Search)

ASKED 2025-10-11 | 374 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 237/288

ABOUT THIS QUESTION

Topic: Binary search on a yes/no condition

Optimal answer from: your app's interview approach (LeetCode #35)

LeetCode match: LC 35 Search Insert Position / lower bound

Frequency: Single report | Last asked: 2025-10-11 | Roles: SWE Intern

QUESTION

Given a non-decreasing (sorted ascending, duplicates allowed) array arr and a target value k , return the smallest index i such that arr[i] >= k — the first position where the array "reaches" the value k . If no element is >= k , return -1 . Solve in O(log n) time and O(1) extra space.

Function signature

```
def first_index_at_least(arr: list[int], k: int) -> int:
    ...
```

Follow-up (as reported): The interviewer probed space complexity; note the solution is already O(1) space. If arr is a huge sorted stream/file, describe reading via random access rather than loading it all.

EXAMPLES

Example 1	
Input	`arr = [1, 2, 2, 3, 5, 7], k = 2`
Output	`1`
Why	`arr[1] = 2 is the first element >= 2.`

Example 2	
Input	`arr = [1, 2, 2, 3, 5, 7], k = 4`
Output	`4`
Why	`The first element >= 4 is 5 at index 4.`

Example 3	
Input	`arr = [1, 2, 2, 3, 5, 7], k = 8`
Output	`-1`
Why	`No element is >= 8.`

CONSTRAINTS

- 0 <= len(arr) <= 10**7
- Values and k are integers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_first_at_least(arr, k):
    for i, x in enumerate(arr):
        if x >= k:
            return i
    return -1
```

TIME	SPACE
O(n)	O(1)

WHY IT IS SLOW

It ignores that the array is sorted.

OPTIMAL SOLUTION · Binary search for the lower bound

TIME	SPACE
O(log n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The array is a huge sorted file with random access and an unknown length.

Probe positions 1, 2, 4, 8, ... until the value there is at least k or the position is past the end, then binary search inside the last doubled range. It needs O(log position) reads and no loading of the data.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: First Index Reaching a Threshold (Binary
# Search)
#
# Given a non-decreasing (sorted ascending, duplicates
# allowed) array arr and a target value k , return the
# smallest index i such that arr[i] >= k — the first
# position where the array "reaches" the value k . If no
# element is >= k , return -1 . Solve in O(log n) time and
# O(1) extra space. Function signature: def
# first_index_at_least(arr: list[int], k: int) -> int: ...
# Follow-up (as reported): The interviewer probed space
# complexity; note the solution is already O(1) space. If
# arr is a huge sorted stream/file, describe reading via
# random access rather than loading it all.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The array is a huge sorted file with random access
# and an unknown length.
#
# INTERVIEW NOTES
# The smallest index with arr[i] >= k is a lower bound:
# binary search for the boundary between 'less than k' and
# 'at least k'. Keep lo and hi as the search range and
# move hi to mid when arr[mid] >= k, lo to mid + 1
# otherwise; the answer is lo if it is inside the array.
```

```
# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: main function ----
# Idea: the array is sorted, so the values below k come first and the
# values at least k come after. Halve the range while keeping that
# boundary inside it; when the range is one spot, that spot is the first
# value >= k. -1 if every value is smaller.
# Complexity: Time O(log n), Space O(1)
def first_index_at_least(arr, k):
    lo = 0
    hi = len(arr)
    while lo < hi:
        mid = (lo + hi) // 2
        if arr[mid] >= k:
            hi = mid                        # mid could be the answer: keep it
        else:
            lo = mid + 1                    # mid is too small: the answer is after it
    if lo < len(arr):
        return lo
    return -1


# ---- main question: tests ----
DATA = [1, 2, 2, 3, 5, 7]


def test_main_question():
    assert first_index_at_least(DATA, 2) == 1
    assert first_index_at_least(DATA, 4) == 4
    assert first_index_at_least(DATA, 8) == -1
    assert first_index_at_least([], 1) == -1


# ======================================================
# FOLLOW-UP 1: The array is a huge sorted file with random access and an unknown length
# ======================================================
# ---- follow-up 1: function ----
# Answer: probe positions 1, 2, 4, 8, ... (doubling) until the value
# there is at least k or the position is past the end, then binary search
# inside the last doubled range. O(log position) reads, and the data is
# never loaded. get(i) returns the i-th value or raises IndexError past the end.
# Complexity: Time O(log p) for the answer position p, Space O(1)
def first_at_least_unbounded(get, k):
    def value(i):
        try:
            return get(i)
        except IndexError:
            return float("inf")             # past the end counts as "bigger than anything"

    hi = 1
    while value(hi - 1) < k:
        hi *= 2
        if hi > (1 << 40):
            return -1
    lo = hi // 2
```

```
    while lo < hi:
```

```
        mid = (lo + hi) // 2
        if value(mid) >= k:
            hi = mid
        else:
            lo = mid + 1
    if value(lo) == float("inf"):
        return -1
    return lo


# ---- follow-up 1: tests ----
def get_from_data(i):
    return DATA[i]


def test_follow_up_1():
    assert first_at_least_unbounded(get_from_data, 2) == 1
    assert first_at_least_unbounded(get_from_data, 4) == 4
    assert first_at_least_unbounded(get_from_data, 1) == 0
    assert first_at_least_unbounded(get_from_data, 8) == -1
    assert first_at_least_unbounded(get_from_data, 7) == 5


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 6 OF 22 |

Greedy: scan once, carry the best choice so far, act only when forced

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Greedy: scan once, carry the best choice so far, act only when forced | 3 questions | tap a title to jump to it

READ FIRST: the code template for these 3 questions (tap here) >

E 9. Alarm for the Next Upcoming Birthday — ColorStack 2025-10-07

E 10. Coins on the Table — ColorStack 2025-09-20

E 11. Maximum Profit — Exponent no date

CODE TEMPLATE | GROUP 6 OF 22 | 3 QUESTIONS, ONE PATTERN

Greedy: scan once, carry the best choice so far, act only when forced

The one idea these questions share

Decide locally and never look back: sort or scan, carry one value (how far you can reach, how far you are covered, the largest seen so far), and take an action only at the moment the current choice stops being enough.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def min_cover(points, length):
    count, covered_to = 0, float("-inf")
    for p in sorted(set(points)):               # CHANGE 1: the order you scan in
        if p > covered_to:                      # CHANGE 2: not covered yet, so you must act
            count += 1
            covered_to = p + length - 1         # CHANGE 3: how far the new choice reaches
    return count
```

2. Helper functions that usually get written next to it

```
def circular_gap(day, today, cycle=365):        # how far ahead a date is, wrapping around the year
    return (day - today) % cycle


def farthest_reach(nums):
    reach = 0
    for i, x in enumerate(nums):
        if i > reach:
            return False
        reach = max(reach, i + x)
    return True
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
9	Alarm for the Next Upcoming Birthday	minimum over people of the circular gap (their day - today) modulo 365, ties by name
10	Coins on the Table	scan the events with one counter: add on '+', subtract on '-' but never below 0
11	Maximum Profit	pick the cheapest and the dearest city with a fixed tie-break (min and max once)

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 9 of 38 | topic 6 of 22: Greedy: scan once, carry the best choice so far, act only when forced | 1 of 3

Template fit: minimum over people of the circular gap (their day - today) modulo 365, ties by name | asked 378 days ago | frequency Single report

EASY | ColorStack | technical interview | 9 of 38

Alarm for the Next Upcoming Birthday

ASKED 2025-10-07 | 378 DAYS AGO | OLDER

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 236/288**

ABOUT THIS QUESTION

Topic: Greedy: scan once, carry the best choice so far, act only when forced

Optimal answer from: written and tested here for this exact question

LeetCode match: Date arithmetic / cyclic minimum (related: LC 1360 Number of Days Between Two Dates)

Frequency: Single report | Last asked: 2025-10-07 | Roles: SWE Intern

QUESTION

You maintain a list of people and their birthdays. Given today's date and a list of (name, month, day) records, return the name of the person whose birthday comes soonest starting from today. Birthdays recur every year, so if no birthday remains later this calendar year, the soonest one is the earliest birthday next year. A birthday that falls exactly on today counts as 0 days away (soonest). Ignore leap years; treat every date simply as a (month, day) pair. If two people share the soonest birthday, return the one that is alphabetically first by name.

Function signature

```
def next_birthday(today: tuple[int, int], people: list[tuple[str, int, int]]) -> str:
    ...
```

EXAMPLES

Example 1

Input	`today = (10, 7), people = [("Alice", 10, 9), ("Bob", 1, 15), ("Carol", 12, 25)]`
Output	`"Alice"`
Why	`Alice's birthday is 2 days away; Carol's is later this year; Bob's is next January. Alice is soonest.`

Example 2

Input	`today = (12, 31), people = [("Alice", 1, 1), ("Dan", 12, 31)]`
Output	`"Dan"`
Why	`Dan's birthday is today (0 days away), which beats Alice's tomorrow.`

Example 3

Input	`today = (6, 1), people = [("Eve", 5, 30), ("Finn", 5, 30)]`
Output	`"Eve"`
Why	`Both birthdays already passed this year, so both are ~next-year and tie; alphabetical order picks "Eve".`

CONSTRAINTS

- 1 <= len(people) <= 10**4
- 1 <= month <= 12 , 1 <= day <= 31 (assume valid calendar dates). today is a valid (month, day) pair.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
MONTH_DAYS = [
    31,
    28,
    31,
    30,
    31,
    30,
    31,
    31,
    30,
    31,
    30,
    31,
]
def brute_next(today, people):
    month, day = today
    for _ in range(365):
        hits = [
            name
            for name, m, d in people
            if (m, d) == (month, day)
        ]
        if hits:
            return min(hits)
        day += 1
        if day > MONTH_DAYS[month - 1]:
            month, day = month % 12 + 1, 1
```

TIME	SPACE
O(365 * p)	O(p)

WHY IT IS SLOW

It checks every day of the year against every person.

OPTIMAL SOLUTION · Day of the year and a cyclic minimum

TIME	SPACE
O(p)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Many queries with different dates. How do you make each one fast?

Sort the birthdays by (day of year, name) once. For a query binary search the first birthday at or after today's day; if there is none, wrap around to the first birthday in the list. O(log p) per query after an O(p log p) setup.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Alarm for the Next Upcoming Birthday
#
# You maintain a list of people and their birthdays. Given
```

```
# today's date and a list of (name, month, day) records,
# return the name of the person whose birthday comes
# soonest starting from today. Birthdays recur every year,
# so if no birthday remains later this calendar year, the
# soonest one is the earliest birthday next year. A
# birthday that falls exactly on today counts as 0 days
# away (soonest). Ignore leap years; treat every date
# simply as a (month, day) pair. If two people share the
# soonest birthday, return the one that is alphabetically
# first by name. Function signature: def
# next_birthday(today: tuple[int, int], people:
# list[tuple[str, int, int]]) -> str: ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Many queries with different dates. How do you make
# each one fast?
#
# INTERVIEW NOTES
# Turn each (month, day) into a day of the year with fixed
# month lengths (no leap years), then the number of days
# until the next occurrence is (their day - today's day)
# mod 365. The soonest birthday is the smallest of those,
# with the name as the tie-break.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from bisect import bisect_left
import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
MONTH_DAYS = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]      # a non-leap year
```

```
# Complexity: Time O(1) (at most 12 months), Space O(1)
def day_of_year(month, day):
    # 1 January is day 1
    total = day
    for m in range(month - 1):
        total += MONTH_DAYS[m]
    return total


# ---- main question: main function ----
# Idea: the number of days from today until a birthday is
# (its day of the year - today's day of the year) mod 365, so a birthday
# that already passed this year wraps around to next year. Take the
# smallest number of days; on a tie the name that comes first.
# today is (month, day); people are (name, month, day).
# Complexity: Time O(p) for p people, Space O(1)
def next_birthday(today, people):
    now = day_of_year(today[0], today[1])
    best_name = None
    best_days = None
    for name, month, day in people:
        days_away = (day_of_year(month, day) - now) % 365
        if best_days is None or days_away < best_days or (days_away == best_days and name < best_name):
```

```
            best_days = days_away
            best_name = name
    return best_name


# ---- main question: tests ----
def test_main_question():
    people = [("Alice", 10, 9), ("Bob", 1, 15), ("Carol", 12, 25)]
    assert next_birthday((10, 7), people) == "Alice"
    assert next_birthday((12, 31), [("Alice", 1, 1), ("Dan", 12, 31)]) == "Dan"     # today counts (0 days
away)
    assert next_birthday((6, 1), [("Eve", 5, 30), ("Finn", 5, 30)]) == "Eve"        # a tie: the first name


# ======================================================
# FOLLOW-UP 1: Many queries with different dates. How do you make each one fast?
# ======================================================
# ---- follow-up 1: helper functions ----
# Answer: sort the birthdays by (day of the year, name) ONCE. For a query
# binary-search the first birthday at or after today's day of the year;
# if there is none, wrap around to the first birthday in the list.
# O(log p) per query after an O(p log p) setup.
# Complexity: Time build O(p log p), each query O(log p), Space O(p)
```

```
class BirthdayBook:
    def __init__(self, people):
        self.items = []
        for name, month, day in people:
            self.items.append((day_of_year(month, day), name))
        self.items.sort()
        self.days = []
        for d, name in self.items:
            self.days.append(d)

    def next_birthday(self, today):
        at = bisect_left(self.days, day_of_year(today[0], today[1]))
        return self.items[at % len(self.items)][1]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    random.seed(50)
    names = ["Ann", "Bo", "Cy", "Di", "Ed"]
    for round_number in range(200):
        people = []
        for name in names:
            people.append((name, random.randint(1, 12), random.randint(1, 28)))
        book = BirthdayBook(people)
        today = (random.randint(1, 12), random.randint(1, 28))
        assert book.next_birthday(today) == next_birthday(today, people)


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 10 of 38 | topic 6 of 22: Greedy: scan once, carry the best choice so far, act only when forced | 2 of 3

Template fit: scan the events with one counter: add on '+', subtract on '-' but never below 0 | asked 395 days ago | frequency Single report

EASY | ColorStack | technical interview | 10 of 38

Coins on the Table

ASKED 2025-09-20 | 395 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 234/288

ABOUT THIS QUESTION

Topic: Greedy: scan once, carry the best choice so far, act only when forced

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1700 / simulation (related: clamped counter)

Frequency: Single report | Last asked: 2025-09-20 | Roles: SWE Intern

QUESTION

There is a table that starts with 0 coins on it. You are given a string events , where each character is a single event applied left to right: '+' places one coin on the table. '-' removes one coin from the table. The table can never hold a negative number of coins: a '-' event when the table is empty has no effect (it is ignored). After processing every event, return the number of coins on the table.

Function signature

```
def coins_on_table(events: str) -> int:
    ...
```

Follow-up (as reported): Generalize so each event is an integer delta (e.g. "+3" , "-2"), still clamping the running total at 0. Hint: parse tokens instead of single characters.

EXAMPLES

Example 1	
Input	`events = "+++-"`
Output	`2`
Why	`+1, +1, +1, then -1 -> 2 coins remain.`

Example 2	
Input	`events = "----"`
Output	`0`
Why	`Every removal happens on an empty table and is ignored.`

Example 3	
Input	`events = "+-+-+"`
Output	`1`
Why	`1, 0, 1, 0, 1 -> 1 coin remains.`

CONSTRAINTS

- 0 <= len(events) <= 10^5
- Every character of events is either '+' or '-' .
- An empty string returns 0 .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_coins(events):
    coins = []
    for e in events:
        if e == "+":
            coins.append(1)
        elif coins:
            coins.pop()
    return len(coins)
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

It stores every coin although only their number matters.

OPTIMAL SOLUTION · One clamped counter

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Each event is an integer delta like +3 or -2, still never below zero.

Split the string into tokens, convert each with int(), and apply total = max(0, total + delta). The clamp must happen after every event, not once at the end.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Coins on the Table
#
# There is a table that starts with 0 coins on it. You are
# given a string events , where each character is a single
# event applied left to right: '+' places one coin on the
# table. '-' removes one coin from the table. The table
# can never hold a negative number of coins: a '-' event
# when the table is empty has no effect (it is ignored).
# After processing every event, return the number of coins
# on the table. Function signature: def
# coins_on_table(events: str) -> int: ... Follow-up (as
# reported): Generalize so each event is an integer delta
# (e.g. "+3" , "-2" ), still clamping the running total at
# 0. Hint: parse tokens instead of single characters.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Each event is an integer delta like +3 or -2, still
# never below zero.
#
# INTERVIEW NOTES
# Each event changes a counter by one, with a floor at
# zero: total = max(0, total + delta). One pass is enough.
```

```
# For the integer-delta follow-up the only change is
# parsing tokens such as +3 and -2 instead of single
# characters.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: one counter that can never go below zero. "+" puts a coin on the
# table; "-" takes one, but only if there is one (otherwise it stays 0):
# total = max(0, total + delta) after every event.
# Complexity: Time O(n), Space O(1)
def coins_on_table(events):
    total = 0
    for event in events:
        if event == "+":
            total += 1
        elif total > 0:
            total -= 1
    return total


# ---- main question: tests ----
def test_main_question():
    assert coins_on_table("+++-") == 2
    assert coins_on_table("----") == 0
    assert coins_on_table("+-+-+") == 1
    assert coins_on_table("") == 0


# ============================================================
# FOLLOW-UP 1: Each event is an integer delta like +3 or -2, still never below zero
# ============================================================
# ---- follow-up 1: function ----
# Answer: split the text into tokens, turn each into a number with int()
# and apply total = max(0, total + delta). The clamp must happen after
# EVERY event, not once at the end.
# Complexity: Time O(n), Space O(n) for the split tokens
def coins_with_deltas(events):
    total = 0
    for token in events.split():
        total += int(token)
        if total < 0:
            total = 0
    return total


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert coins_with_deltas("+3 -2 -5 +4") == 4
    random.seed(16)
    for round_number in range(300):                # +1 / -1 deltas must agree with the plus/minus version
        s = ""
        for i in range(random.randint(0, 12)):
```

```
            s += random.choice("+-")
        tokens = []
        for c in s:
            if c == "+":
                tokens.append("+1")
            else:
                tokens.append("-1")
        assert coins_with_deltas(" ".join(tokens)) == coins_on_table(s)


# ================================================
# RUN EVERYTHING
# ================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 11 of 38 | topic 6 of 22: Greedy: scan once, carry the best choice so far, act only when forced | 3 of 3

Template fit: pick the cheapest and the dearest city with a fixed tie-break (min and max once) | no date

EASY | Exponent | 11 of 38

★ TOP 10 EASY: rank 2 among Easy problems

★ Maximum Profit

ASKED (ANY CO.) ~4 MONTHS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 72** | **30 DAYS 61** | **PRIORITY 6/288**

ABOUT THIS QUESTION

Topic: Greedy: scan once, carry the best choice so far, act only when forced

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #121 Best Time to Buy and Sell Stock

LeetCode match: LC 121 Best Time to Buy and Sell Stock (likely) (inferred from short statement)

Last asked (any company): 4 months ago, about 2026-06 (Exponent shows only a rough age) | 4 interviews

Original: https://www.aced.io/questions/3630/maximum-profit

QUESTION

You and your friend have decided to start an international trading business. You want to earn a profit through some ol' fashioned arbitrage: You'll buy widgets in one country and sell them in another at a higher price. Your friend has already found the current market prices for widgets in cities around the world, and you'd like to use this data to pick which cities to buy and sell widgets in. Write a function max_profit(prices) that efficiently finds the two cities that maximize your profit (i.e. largest difference in prices).

Input: A dictionary with cities as keys and prices as values Output: An array containing the names of the cities (min, max) If the prices dictionary is empty, return None/null (depending on the programming language used). If there is 0 profit to be earned, return None/null as well. If you are using C++, return {"", ""} in such cases. Example Pythonprices = {'London': 72, 'New York': 70, 'Tokyo': 67, 'Miami': 62} max_profit(prices)

=> ['Miami', 'London']Hint 1You can think of this as two sub-problems

: Finding the lowest price, and finding the highest price.Hint 2While you could compare every pair of cities, it's more efficient to find the min and max values.Hint 3You can keep track of the lowest and highest values and update them as you iterate through the list.Challenge 1Does your solution work in O(N) time and O(1) space? If not, you can be more efficient!Challenge 2What happens if the price is the same in every city? Does your function still return

EXAMPLES

Example 1	
Input	`{'A': 3, 'B': 10, 'C': 1}`
Output	`9`
Why	`Buy in the cheapest city (C, price 1) and sell in the dearest (B, price 10): profit 9.`

Example 2	
Input	`{'A': 5}`
Output	`0`
Why	`With a single city there is nothing to trade.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(price, cities):
    best = (-INF, None, None)
    for buy in cities:
        for sell in cities:
            if buy == sell:
                continue
            profit = price[sell] - price[buy]
            best = max(best, (profit, buy, sell))
    return best
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

n^2 pairs. Since city order does not matter, the best pair is the cheapest city and the dearest city, found in one O(n) scan.

OPTIMAL SOLUTION · One pass for the cheapest and the dearest city

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The prices are a time series: you must buy before you sell (LeetCode 121).

Keep the minimum price seen so far and the best difference: O(n), O(1). The dictionary version ignores order, so the two extremes are enough.

FOLLOW-UP 2 Allow many trades (LeetCode 122) or a limit of k trades.

Many trades: add every positive day-to-day rise. At most k trades: DP over (trades used, holding or not), O(n * k).

FOLLOW-UP 3 Transaction fee or cooldown.

A two-state DP (holding, not holding) per day: hold = max(hold, free - price), free = max(free, hold + price - fee).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Maximum Profit
#
# You and your friend have decided to start an
# international trading business. You want to earn a
# profit through some ol' fashioned arbitrage: You'll buy
# widgets in one country and sell them in another at a
# higher price. Your friend has already found the current
# market prices for widgets in cities around the world,
# and you'd like to use this data to pick which cities to
# buy and sell widgets in. Write a function
# max_profit(prices) that efficiently finds the two cities
```

```
# that maximize your profit (i.e. largest difference in
# prices). Input: A dictionary with cities as keys and
# prices as values Output: An array containing the names
# of the cities (min, max) If the prices dictionary is
# empty, return None/null (depending on the programming
# language used). If there is 0 profit to be earned,
# return None/null as well. If you are using C++, return
# {"", ""} in such cases.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The prices are a time series: you must buy before you
# sell (LeetCode 121).
# 2. Allow many trades (LeetCode 122) or a limit of k
# trades.
# 3. Transaction fee or cooldown.
#
# INTERVIEW NOTES
# Check whether order matters. A dictionary has none, so
# the profit is largest price minus smallest price, and
# the answer is just the two extremes. If this were a list
# of prices over time you would have to buy before you
# sell, which is a different one-pass problem with a
# running minimum.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
def bigger(a, b):
    if a > b:
        return a
    return b


def smaller(a, b):
    if a < b:
        return a
    return b


# ---- main question: main function ----
# Idea: prices is a dictionary city -> price. Buy in the cheapest city
# and sell in the dearest one: the order of the cities does not matter,
# so the best profit is highest price - lowest price (0 with one city).
# Complexity: Time O(n), Space O(1)
def max_profit_prices(prices):
    if len(prices) < 2:
        return 0
    lowest = None
    highest = None
    for city in prices:
        price = prices[city]
        if lowest is None or price < lowest:
            lowest = price
        if highest is None or price > highest:
            highest = price
    return highest - lowest
```

```
# ---- main question: tests ----
def test_main_question():
    assert max_profit_prices({"a": 10, "b": 4, "c": 9}) == 6
    assert max_profit_prices({"a": 3}) == 0
    assert max_profit_prices({}) == 0


# ======================================================
# FOLLOW-UP 1: The prices are a time series: you must buy before you sell (LeetCode 121)
# ======================================================
# ---- follow-up 1: function ----
# Answer: keep the lowest price seen so far and the best difference
# (today's price minus that lowest): O(n) time, O(1) space. The
```

```
# dictionary version could ignore the order; a time series cannot.
# Complexity: Time O(n), Space O(1)
def best_one_trade(series):
    lowest = float("inf")
    best = 0
    for price in series:
        lowest = smaller(lowest, price)
        best = bigger(best, price - lowest)
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert best_one_trade([7, 1, 5, 3, 6, 4]) == 5
    assert best_one_trade([7, 6, 4, 3, 1]) == 0


# ======================================================
# FOLLOW-UP 2: Allow many trades (LeetCode 122) or a limit of k trades
# ======================================================
# ---- follow-up 2: functions ----
# Answer: many trades: add up every rise from one day to the next. At
# most k trades: a DP over (trades used, holding a share or not): O(n * k).
# Complexity: Time O(n), Space O(1)
def best_many_trades(series):
    total = 0
    for i in range(1, len(series)):
        if series[i] > series[i - 1]:
            total += series[i] - series[i - 1]
    return total


# Complexity: Time O(n * k), Space O(k)
def best_k_trades(series, k):
    if len(series) == 0 or k == 0:
        return 0
    hold = [float("-inf")] * (k + 1)       # best money while holding a share, with t trades started
    cash = [0] * (k + 1)                   # best money with no share, with t trades finished
    for price in series:
        for t in range(1, k + 1):
            hold[t] = bigger(hold[t], cash[t - 1] - price)      # buy, using the cash of t - 1 finished tra
des
            cash[t] = bigger(cash[t], hold[t] + price)          # or sell
    return cash[k]
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert best_many_trades([7, 1, 5, 3, 6, 4]) == 7
```

```
    assert best_k_trades([3, 2, 6, 5, 0, 3], 2) == 7
    assert best_k_trades([1, 2, 3], 1) == 2
    assert best_k_trades([], 3) == 0


# =====================================================
# FOLLOW-UP 3: Transaction fee or cooldown
# =====================================================
# ---- follow-up 3: function ----
# Answer: a two-state DP per day: hold = best money while holding a
# share, cash = best money without one.
#   hold = max(hold, cash - price)        cash = max(cash, hold + price - fee)
# Complexity: Time O(n), Space O(1)
def best_with_fee(series, fee):
    hold = float("-inf")
    cash = 0
    for price in series:
        hold = bigger(hold, cash - price)
        cash = bigger(cash, hold + price - fee)
    return cash


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert best_with_fee([1, 3, 2, 8, 4, 9], 2) == 8


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

You and your friend have decided to start an international trading business. You want to earn a profit through some ol' fashioned arbitrage: You'll buy widgets in one country and sell them in another at a higher price. Your friend has already found the current market prices for widgets in cities around the world, and you'd like to use this data to pick which cities to buy and sell widgets in. Write a function max_profit(prices) that efficiently finds the two cities that maximize your profit (i.e. largest difference in prices).

Input: A dictionary with cities as keys and prices as values Output: An array containing the names of the cities (min, max) If the prices dictionary is empty, return None/null (depending on the programming language used). If there is 0 profit to be earned, return None/null as well. If you are using C++, return {"", ""} in such cases. Example

```
Pythonprices = {'London': 72, 'New York': 70, 'Tokyo': 67, 'Miami': 62}

max_profit(prices) # => ['Miami', 'London']
```

OFFICIAL PAGE TEXT

Hint 1You can think of this as two sub-problems: Finding the lowest price, and finding the highest price.Hint 2While you could compare every pair of cities, it's more efficient to find the min and max values.Hint 3You can keep track of the lowest and highest values and update them as you iterate through the list.Challenge 1Does your solution work in O(N) time and O(1) space? If not, you can be more efficient!Challenge 2What happens if the price is the same in every city? Does your function still return different cities?For our solution, we'll simply iterate through the prices list and simultaneously keep track of the minimum and maximum values we've seen so far. This only requires one iteration, so it's more efficient than the 'brute force' approach of calculating the profit for each pair of cities, which would require O(n^2) time. A more concise—but slightly less efficient solution—would be to sort the cities according to their prices and then return the first and last elements. In some languages like Python, you can also use the built-in min and max functions to compare dictionary values:

```
Pythondef max_profit(prices):

  # Check if the dictionary is empty
  if not prices:
    return None

  min_city = min(prices, key=prices.get)
  max_city = max(prices, key=prices.get)

  # Check if no profits earned
  if min_city == max_city:
    return None

  return [min_city, max_city]
```

Time Complexity: Iterating over prices.items() takes O(n) time, where n is the number of cities in the prices dictionary. Within the loop, there are constant-time operations, so the overall time complexity is O(n). Space Complexity: The space complexity is also O(1) because the size of the auxiliary array returned ([min_city, max_city]) is constant and independent of the input size.No answers contributed by the community yet.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 7 OF 22 |

Sets and counters: remember the first collection, check the second against it

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Sets and counters: remember the first collection, check the second against it | 4 questions | tap a title to jump to it

READ FIRST: the code template for these 4 questions (tap here) >

E	12. Exclude First-K-Unique From Another List	ColorStack 2025-10-17
E	13. Positional Matches Between Two Lists	ColorStack 2025-10-15
E	14. Contains Duplicate	ColorStack 2025-09-16
E	15. Find the Duplicates	Exponent no date

CODE TEMPLATE | GROUP 7 OF 22 | 4 QUESTIONS, ONE PATTERN

Sets and counters: remember the first collection, check the second against it

The one idea these questions share

Build a set (or a Counter) from one collection, then walk the other and ask 'is this in it?'. Membership in a set is O(1), which is the whole trick. Duplicates, intersections, filters, and missing values are the same loop with a different question.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import Counter


def keep_not_in(first, second):
    seen = set(first)                          # CHANGE 1: a set, or a Counter if counts matter
    return [x for x in second if x not in seen]     # CHANGE 2: keep, count, or stop at the first hit
```

2. Helper functions that usually get written next to it

```
def common_counts(a, b):                   # how many of each value the two lists share
    ca, cb = Counter(a), Counter(b)
    return {v: min(ca[v], cb[v]) for v in ca if v in cb}


def first_repeat(items):
    seen = set()
    for x in items:
        if x in seen:
            return x
        seen.add(x)
    return None
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
12	Exclude First-K-Unique From Another List	the set holds the first k distinct values of list 1; keep list 2's values that are not in it
13	Positional Matches Between Two Lists	no set needed: zip both lists and keep a where a == b
14	Contains Duplicate	add values to a set as you go; the first value already in the set means a duplicate
15	Find the Duplicates	values that appear in both lists, each reported once (set intersection, or two pointers on sorted lists)

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 12 of 38 | topic 7 of 22: Sets and counters: remember the first collection, check the second against it | 1 of 4

Template fit: the set holds the first k distinct values of list 1; keep list 2's values that are not in it | asked 368 days ago | frequency Single report

EASY | ColorStack | technical interview | 12 of 38

Exclude First-K-Unique From Another List

ASKED 2025-10-17 | 368 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 50.4** **30 DAYS 46.9** **PRIORITY 29/288**

ABOUT THIS QUESTION

Topic: Sets and counters: remember the first collection, check the second against it

Optimal answer from: written and tested here for this exact question

LeetCode match: Set filtering (related: LC 349 Intersection of Two Arrays)

Frequency: Single report | Last asked: 2025-10-17 | Roles: SWE Intern

QUESTION

You are given two lists list1 and list2 and an integer k . Scan list1 left to right and collect the first k distinct values it contains (in order of first appearance; if list1 has fewer than k distinct values, take all of them). Then return list2 with every occurrence of any of those collected values removed, preserving the order of the remaining elements.

Function signature

```
def exclude_first_k_unique(list1: list[int], list2: list[int], k: int) -> list[int]:
    ...
```

EXAMPLES

Example 1	
Input	`list1 = [3, 3, 1, 4, 1, 5], list2 = [1, 2, 3, 4, 3, 5], k = 2`
Output	`[2, 4, 5]`
Why	`First 2 distinct values of list1 are {3, 1}. Removing all 3s and 1s from list2 leaves [2, 4, 5].`

Example 2	
Input	`list1 = [7], list2 = [7, 8, 9], k = 3`
Output	`[8, 9]`
Why	`list1 has only one distinct value (7); remove all 7s from list2.`

CONSTRAINTS

- 0 <= len(list1), len(list2) <= 10**5 , 1 <= k .
- Values are integers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute_exclude(list1, list2, k):
    picked = []
    for x in list1:
        if len(picked) == k:
            break
        if x not in picked:
            picked.append(x)
    return [
        y for y in list2 if y not in picked
    ]
```

TIME	SPACE
O(n + m * k)	O(k)

WHY IT IS SLOW

Each element of list2 searches the whole collected list linearly.

OPTIMAL SOLUTION · A set of the first k distinct values

TIME	SPACE
O(n + m)	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 list2 is a huge stream. How do you avoid loading it?

Build the blocked set from list1 first (O(k) memory), then yield the stream's values lazily, skipping the blocked ones. The output is a generator.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Exclude First-K-Unique From Another List
#
# You are given two lists list1 and list2 and an integer k
# . Scan list1 left to right and collect the first k
# distinct values it contains (in order of first
# appearance; if list1 has fewer than k distinct values,
# take all of them). Then return list2 with every
# occurrence of any of those collected values removed,
# preserving the order of the remaining elements. Function
# signature: def exclude_first_k_unique(list1: list[int],
# list2: list[int], k: int) -> list[int]: ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. list2 is a huge stream. How do you avoid loading it?
#
# INTERVIEW NOTES
# Two steps: collect the first k distinct values of list1
# in a set (stopping as soon as the set has k values),
# then filter list2 by membership in that set. The set
# makes the filtering O(len(list2)).

# ======================================================
```

```
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def first_k_unique(list1, k):
    # the first k DISTINCT values of list1, as a set
    blocked = set()
    for x in list1:
        if len(blocked) == k:
            break
        blocked.add(x)
    return blocked


# ---- main question: main function ----
# Take the first k distinct values of list1 and remove them from list2.
# Idea: put those values in a set; a set lookup is O(1), so filtering
```

```
# list2 takes one pass.
# Complexity: Time O(n + m), Space O(k)
def exclude_first_k_unique(list1, list2, k):
    blocked = first_k_unique(list1, k)
    result = []
    for y in list2:
        if y not in blocked:
            result.append(y)
    return result


# ---- main question: tests ----
def test_main_question():
    assert exclude_first_k_unique([3, 3, 1, 4, 1, 5], [1, 2, 3, 4, 3, 5], 2) == [2, 4, 5]
    assert exclude_first_k_unique([7], [7, 8, 9], 3) == [8, 9]


# ============================================================
# FOLLOW-UP 1: list2 is a huge stream. How do you avoid loading it?
# ============================================================
# ---- follow-up 1: function ----
# Answer: build the blocked set from list1 first (O(k) memory), then
# yield the stream's values one at a time, skipping the blocked ones. The
# result is a generator, so list2 is never loaded.
# Complexity: Time O(n) to prepare, then O(1) per streamed item, Space O(k)
def exclude_stream(list1, stream, k):
    blocked = first_k_unique(list1, k)
    for y in stream:
        if y not in blocked:
            yield y


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert list(exclude_stream([3, 3, 1, 4, 1, 5], iter([1, 2, 3, 4, 3, 5]), 2)) == [2, 4, 5]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 13 of 38 | topic 7 of 22: Sets and counters: remember the first collection, check the second against it | 2 of 4

Template fit: no set needed: zip both lists and keep a where a == b | asked 370 days ago | frequency Single report

EASY | ColorStack | technical interview | 13 of 38

Positional Matches Between Two Lists

ASKED 2025-10-15 | 370 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 238/288

ABOUT THIS QUESTION

Topic: Sets and counters: remember the first collection, check the second against it

Optimal answer from: written and tested here for this exact question

LeetCode match: Two-list comparison (related: LC 1346 / zip scan)

Frequency: Single report | Last asked: 2025-10-15 | Roles: SWE Intern

QUESTION

You are given two lists of equal length, first and second . Return, in order, the values at every position where the two lists agree — that is, all first[i] such that first[i] == second[i] .

Function signature

def positional_matches(first: list[int], second: list[int]) -> list[int]:

...

EXAMPLES

Example 1	
Input	`first = [3, 1, 4, 1, 5, 5, 7, 9], second = [1, 2, 3, 4, 5, 6, 7, 8]`
Output	`[5, 7]`
Why	`Positions 4 (5 == 5) and 6 (7 == 7) match. (The report described "matches" between a digits-of-pi list and an index list.)`

Example 2	
Input	`first = [1, 2, 3], second = [4, 5, 6]`
Output	`[]`
Why	`No position matches.`

CONSTRAINTS

- len(first) == len(second) , 0 <= length <= 10**6 .
- Values are integers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_positional(first, second):
    out = []
    for i in range(len(first)):
        if first[i] == second[i]:
            out.append(first[i])
    return out
```

BRUTE FORCE

TIME	SPACE
O(n)	O(matches)

WHY IT IS SLOW

Correct and linear; it is only longer than the zip form.

OPTIMAL SOLUTION · zip and a filter

TIME	SPACE
O(n)	O(matches)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the positions instead of the values, lazily for a huge input.

Use enumerate over zip inside a generator: nothing is stored, and the caller can stop early. Memory is O(1).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Positional Matches Between Two Lists
#
# You are given two lists of equal length, first and
# second . Return, in order, the values at every position
# where the two lists agree — that is, all first[i] such
# that first[i] == second[i] . Function signature: def
# positional_matches(first: list[int], second: list[int])
# -> list[int]: ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the positions instead of the values, lazily
# for a huge input.
#
# INTERVIEW NOTES
# One pass over both lists at the same time: keep first[i]
# whenever first[i] == second[i]. zip walks them together,
# so there is no indexing and no extra structure.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: walk both lists together and keep the value wherever they agree at
# the same position. The shorter list decides how far to go.
# Complexity: Time O(min(n, m)), Space O(number of matches)
def positional_matches(first, second):
```

```
    matches = []
    for i in range(min(len(first), len(second))):
        if first[i] == second[i]:
            matches.append(first[i])
    return matches


# ---- main question: tests ----
def test_main_question():
```

```
    assert positional_matches([3, 1, 4, 1, 5, 5, 7, 9], [1, 2, 3, 4, 5, 6, 7, 8]) == [5, 7]
    assert positional_matches([1, 2, 3], [4, 5, 6]) == []


# ======================================================
# FOLLOW-UP 1: Return the positions instead of the values, lazily for a huge input
# ======================================================
# ---- follow-up 1: function ----
# Answer: write a GENERATOR that yields each matching position as it finds
# it. Nothing is stored, so memory is O(1), and the caller can stop early.
# Complexity: Time O(min(n, m)), Space O(1)
def matching_positions(first, second):
    for i in range(min(len(first), len(second))):
        if first[i] == second[i]:
            yield i


# ---- follow-up 1: tests ----
def test_follow_up_1():
    random.seed(64)
    for round_number in range(300):
        n = random.randint(0, 9)
        a = []
        b = []
        for i in range(n):
            a.append(random.randint(0, 3))
            b.append(random.randint(0, 3))
        values = []
        for position in matching_positions(a, b):
            values.append(a[position])
        assert values == positional_matches(a, b)


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 14 of 38 | topic 7 of 22: Sets and counters: remember the first collection, check the second against it | 3 of 4

Template fit: add values to a set as you go; the first value already in the set means a duplicate | asked 399 days ago | frequency Single report

EASY | IN APP LC 217 | ColorStack | technical interview | 14 of 38

Contains Duplicate

ASKED 2025-09-16 | 399 DAYS AGO | OLDER

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 232/288**

ABOUT THIS QUESTION

Topic: Sets and counters: remember the first collection, check the second against it

Optimal answer from: your app's interview approach (LeetCode #217)

LeetCode match: LC 217 Contains Duplicate

Frequency: Single report | Last asked: 2025-09-16 | Roles: SWE Intern

QUESTION

Given an integer array nums , return True if any value appears at least twice in the array, and False if every element is distinct.

Function signature

```
def contains_duplicate(nums: list[int]) -> bool:
    """Return True if any value appears more than once."""
    ...
```

EXAMPLES

Example 1	
Input	`nums = [1, 2, 3, 1]`
Output	`True`

Example 2	
Input	`nums = [1, 2, 3, 4]`
Output	`False`

Example 3	
Input	`nums = [1, 1, 1, 3, 3, 4, 3, 2, 4, 2]`
Output	`True`

CONSTRAINTS

- 1 <= len(nums) <= 10^5
- -10^9 <= nums[i] <= 10^9

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_duplicate(nums):
    for i in range(len(nums)):
        for j in range(i_+_1,_len(nums)):
            if nums[i] == nums[j]:
                return True
    return False
```

BRUTE FORCE

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Each element is compared with all the later ones.

OPTIMAL SOLUTION · A set of seen values

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if you cannot use extra memory?

Sort in place and compare neighbours: duplicates become adjacent. O(n log n) time and O(1) extra space (not counting the sort's own stack).

FOLLOW-UP 2 Only count it if the two equal values are at most k positions apart.

Keep a set of the last k values (a sliding window): add each new value, drop the one that left the window, and check membership before adding. O(n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Contains Duplicate
#
# Given an integer array nums , return True if any value
# appears at least twice in the array, and False if every
# element is distinct. Function signature: def
# contains_duplicate(nums: list[int]) -> bool: """Return
# True if any value appears more than once.""" ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if you cannot use extra memory?
# 2. Only count it if the two equal values are at most k
# positions apart.
#
# INTERVIEW NOTES
# A repeated value means the number of distinct values is
# smaller than the length. A set gives the answer in one
# pass, and sorting trades the memory for time.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: remember the values seen so far in a set; meeting a value that is
# already in the set means a duplicate. O(n) time, O(n) memory.
# Complexity: Time O(n), Space O(n)
def contains_duplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return True
        seen.add(x)
    return False


# ---- main question: tests ----
```

```
def test_main_question():
    assert contains_duplicate([1, 2, 3, 1]) is True
    assert contains_duplicate([1, 2, 3, 4]) is False
    assert contains_duplicate([]) is False


# ============================================================
# FOLLOW-UP 1: What if you cannot use extra memory?
# ============================================================
# ---- follow-up 1: function ----
# Answer: sort (in place if allowed) and compare neighbours: duplicates
# become adjacent. O(n log n) time and no extra set.
# Complexity: Time O(n log n), Space O(n) for the sorted copy
def contains_duplicate_sorted(nums):
    ordered = sorted(nums)
    for i in range(len(ordered) - 1):
        if ordered[i] == ordered[i + 1]:
            return True
    return False


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert contains_duplicate_sorted([1, 2, 3, 1]) is True
    assert contains_duplicate_sorted([1, 2, 3, 4]) is False


# ============================================================
# FOLLOW-UP 2: Only count it if the two equal values are at most k positions apart
# ============================================================
# ---- follow-up 2: function ----
# Answer: keep a set of just the last k values (a sliding window): check
# the new value against it, add it, and drop the value that just left the
# window. O(n).
# Complexity: Time O(n), Space O(k)
def duplicate_within_k(nums, k):
    window = set()
    for i in range(len(nums)):
        if nums[i] in window:
            return True
        window.add(nums[i])
        if len(window) > k:
            window.remove(nums[i - k])
    return False
```

```
# ---- follow-up 2: tests ----
```

```
def test_follow_up_2():
    random.seed(7)
    for round_number in range(300):
        nums = []
        for i in range(random.randint(0, 9)):
            nums.append(random.randint(0, 8))
        k = random.randint(1, 5)
        slow = False
        for i in range(len(nums)):                  # compare every pair that is at most k apart
            for j in range(i + 1, min(len(nums), i + k + 1)):
                if nums[i] == nums[j]:
                    slow = True
        assert duplicate_within_k(nums, k) == slow


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 15 of 38 | topic 7 of 22: Sets and counters: remember the first collection, check the second against it | 4 of 4

Template fit: values that appear in both lists, each reported once (set intersection, or two pointers on sorted lists) | no date

EASY | Exponent | 15 of 38

Find the Duplicates

ASKED (ANY CO.) ~7 MONTHS AGO | OLDER

STUDY: TALK | ALL TIME 28.2 | 30 DAYS 0 | PRIORITY 151/288

ABOUT THIS QUESTION

Topic: Sets and counters: remember the first collection, check the second against it

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #350 Intersection of Two Arrays II

LeetCode match: LC 350 Intersection of Two Arrays II

Last asked (any company): 7 months ago, about 2026-03 (Exponent shows only a rough age) | +15 interviews

Original: https://www.aced.io/questions/3056/find-the-duplicates

QUESTION

Given two sorted arrays arr1 and arr2 of passport numbers, implement a function findDuplicates that returns an array of all passport numbers that are both arrays. Note that the output array should be sorted in ascending order. Let N and M be the lengths of arr1 and arr2, respectively. Solve for two cases and analyze the time & space complexities of your solutions: M ~ N - the array lengths are approximately the same M >> N - arr2 is much bigger than arr1. Example input: arr1 = [1, 2, 3, 5, 6, 7], arr2 = [3, 6, 7, 8, 20] output: [3, 6, 7]

since only these three values

are both in arr1 and arr2Hint 1Start with a brute force solution and optimize from there.Hint 2Note that the question asks for a sorted array. What techniques work well on sorted arrays?Hint 3If M >> N (case #2) is giving you trouble, consider how you might use the values of N, the shorter array, to increase the efficiency of your answer.Challenge 1Have you taken care to avoid array out-of-bound runtime errors?First, the brute force solution. Try two nested loops. For every value arr1[i] in the outer loop, we look it up in arr2 in the inner loop. The time complexity of this algorithm in the worst case scenario is O(N*M). Not very efficient. We can do better than that. Case 1 (M ~ N) We can use the fact the arrays are sorted to traverse both of them in an in-order manner at the same time. The general idea of the algorithm is to use two indices, i and j, for arr1 and arr2, respectively. Every time one of indices,

EXAMPLES

Example 1	
Input	`[1, 3, 5, 7], [3, 4, 5, 8]`
Output	`[3, 5]`
Why	`Two sorted lists of passport numbers: 3 and 5 appear in both.`

Example 2	
Input	`[1, 2], [3, 4]`
Output	`[]`
Why	`Nothing in common.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(arr1, arr2):
    found = []
    for x in arr1:
        for y in arr2:
            if x == y and x not in found:
                found.append(x)
    return sorted(found)
```

TIME	SPACE
O(N * M)	**O(1)** extra

WHY IT IS SLOW

N * M comparisons. Two pointers on the sorted arrays are O(N + M), and binary searching the big array for each element of the small one is O(N log M).

OPTIMAL SOLUTION · Case 1 (M close to N): two pointers

TIME	SPACE
O(N + M)	**O(1)** extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Case 2 (M much bigger than N): binary search the big array

TIME	SPACE
O(N log M)	**O(1)** extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Three or more arrays.

Intersect pairwise, smallest arrays first so the running result shrinks quickly: O(total size). With K arrays you can also keep K pointers and advance the smallest.

FOLLOW-UP 2 The arrays are too big for memory.

Read both as streams (two pointers need only the current element of each), or partition both by value range into files that fit in memory and intersect partition by partition. **Cost:** O(N + M) time and O(1) memory with two streaming pointers; partitioning by value range multiplies the disk passes by the number of partitions.

FOLLOW-UP 3 Which one do you choose when N = 1,000 and M = 10^9?

Binary search: N log M is about 30,000 steps while two pointers need 10^9. The break-even is when N log M is around M.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the Duplicates
#
# Given two sorted arrays arr1 and arr2 of passport
# numbers, implement a function findDuplicates that
# returns an array of all passport numbers that are both
# arrays. Note that the output array should be sorted in
# ascending order. Let N and M be the lengths of arr1 and
# arr2, respectively. Solve for two cases and analyze the
# time & space complexities of your solutions: M ~ N - the
# array lengths are approximately the same M >> N - arr2 is
# much bigger than arr1. Example input: arr1 = [1, 2, 3,
# 5, 6, 7], arr2 = [3, 6, 7, 8, 20] output: [3, 6, 7] #
# since only these three values are both in arr1 and
# arr2Hint 1Start with a brute force solution and optimize
# from there.Hint 2Note that the question asks for a
# sorted array.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Three or more arrays.
# 2. The arrays are too big for memory.
# 3. Which one do you choose when N = 1,000 and M = 10^9?
#
# INTERVIEW NOTES
# Both arrays are sorted, so the question is how the two
# sizes compare. If they are about the same size, walk
# both with two pointers and advance the smaller one. If
# one is much smaller, do not walk the big one: binary
# search each element of the small array in the big array.
# Always say the cost in terms of both sizes N and M.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import random

END = object()


# ========================================================
# MAIN QUESTION
# ========================================================
```

```
# ---- main question: helper functions ----
def first_at_least(values, x, start):
    # binary search in a sorted list, looking only from index `start` on:
    # the first index whose value is >= x
    low = start
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] < x:
            low = middle + 1
        else:
            high = middle
    return low
```

```python
def bit_length(n):
    # how many binary digits n has (roughly log2 of n)
    bits = 0
    while n > 0:
        bits += 1
        n = n // 2
    return bits


# ---- main question: main functions ----
# Find the values that appear in BOTH sorted arrays (each result once).
# CASE 1: the arrays have similar sizes: two pointers. Walk both arrays
# together and move on the pointer that points at the smaller value; when
# they are equal, record the value once and move both. O(N + M) time.
# Complexity: Time O(n + m), Space O(1) plus the answer
def common_two_pointers(a, b):
    result = []
    i = 0
    j = 0
    while i < len(a) and j < len(b):
        if a[i] < b[j]:
            i += 1
        elif a[i] > b[j]:
            j += 1
        else:
            if not result or result[len(result) - 1] != a[i]:
                result.append(a[i])
            i += 1
            j += 1
    return result
```

```python
# CASE 2: one array is much shorter: binary-search every item of the short
# array in the long one. O(N log M); each search starts where the previous
# one stopped.
# Complexity: Time O(s log b) for the smaller list of size s and the bigger of size b, Space O(1) plus the
answer
def common_binary_search(small, big):
    result = []
    position = 0
    for x in small:
        position = first_at_least(big, x, position)
        if position == len(big):
            break
        if big[position] == x and (not result or result[len(result) - 1] != x):
            result.append(x)
    return result


# Choose by cost: N log M against N + M.
# Complexity: Time O(min(n + m, s log b)), Space O(1) plus the answer
def common(a, b):
    if len(a) <= len(b):
        small = a
        big = b
    else:
        small = b
        big = a
    if len(small) * bit_length(len(big)) < len(a) + len(b):
        return common_binary_search(small, big)
    return common_two_pointers(a, b)
```

```
# ---- main question: tests ----
def test_main_question():
    a = [1, 2, 3, 5, 6, 7]
    b = [3, 6, 7, 8, 20]
    assert common_two_pointers(a, b) == [3, 6, 7]
    assert common_binary_search(b, a) == [3, 6, 7]
    assert common(a, b) == [3, 6, 7]
    assert common([], [1]) == []
    assert common([1, 1, 2], [1, 1, 3]) == [1]
    assert common([5, 999999], list(range(0, 1000000, 3))) == [999999]      # one short array against a ver
y long one


# ========================================================
```

```
# FOLLOW-UP 1: Three or more arrays
# ========================================================
# ---- follow-up 1: function ----
# Answer: intersect pairwise, the smallest arrays first so the running
# result shrinks quickly: O(total size). With K arrays you can also keep K
# pointers and move the one at the smallest value.
# Complexity: Time O(total length of the lists) when started with the shortest, Space O(shortest list)
def common_many(arrays):
    ordered = sorted(arrays, key=len)
    result = ordered[0]
    for other in ordered[1:]:
        result = common(result, other)
        if not result:
            break
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert common_many([[1, 2, 3], [7, 8], [3]]) == []
    assert common_many([[1, 2, 3, 4], [2, 3, 9], [2, 3, 5, 6, 7]]) == [2, 3]


# ========================================================
# FOLLOW-UP 2: The arrays are too big for memory
# ========================================================
# ---- follow-up 2: function ----
# Answer: read both as streams (two pointers need only the CURRENT item of
# each), or split both by value range into files that fit in memory and
# intersect them part by part.
# Complexity: Time O(n + m), Space O(1)
def common_stream(a_items, b_items):
    a_stream = iter(a_items)
    b_stream = iter(b_items)
    x = next(a_stream, END)
    y = next(b_stream, END)
    last = END
    while x is not END and y is not END:
        if x < y:
            x = next(a_stream, END)
        elif x > y:
            y = next(b_stream, END)
        else:
            if x != last:
                yield x
```

```
                last = x
            x = next(a_stream, END)
            y = next(b_stream, END)
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert list(common_stream(iter([1, 2, 3, 5, 6, 7]), iter([3, 6, 7, 8, 20]))) == [3, 6, 7]
    random.seed(23)
    for round_number in range(300):
        x = sorted(random.sample(range(40), random.randint(0, 9)))
        y = sorted(random.sample(range(40), random.randint(0, 9)))
        expected = []
        for value in x:
            if value in y:
                expected.append(value)
        assert common_two_pointers(x, y) == expected
        assert common_binary_search(x, y) == expected
        assert common(x, y) == expected
        assert list(common_stream(x, y)) == expected


# ========================================================
# FOLLOW-UP 3: Which one do you choose when N = 1,000 and M = 10^9?
# ========================================================
# Answer: binary search: 1,000 * 30 steps is far fewer than 10^9, so the
# long array is hardly read at all (common() makes this choice by comparing
# the costs).


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given two sorted arrays arr1 and arr2 of passport numbers, implement a function findDuplicates that returns an array of all passport numbers that are both arrays. Note that the output array should be sorted in ascending order. Let N and M be the lengths of arr1 and arr2, respectively. Solve for two cases and analyze the time & space complexities of your solutions: M ~ N - the array lengths are approximately the same M >> N - arr2 is much bigger than arr1. Example

```
input:  arr1 = [1, 2, 3, 5, 6, 7], arr2 = [3, 6, 7, 8, 20]

output: [3, 6, 7] # since only these three values are both in arr1 and arr2
```

OFFICIAL PAGE TEXT

Hint 1Start with a brute force solution and optimize from there.Hint 2Note that the question asks for a sorted array. What techniques work well on sorted arrays?Hint 3If M >> N (case #2) is giving you trouble, consider how you might use the values of N, the shorter array, to increase the efficiency of your answer.Challenge 1Have you taken care to avoid array out-of-bound runtime errors?First, the brute force solution. Try two nested loops. For every value arr1[i] in the outer loop, we look it up in arr2 in the inner loop. The time complexity of this algorithm in the worst case scenario is O(N*M). Not very efficient. We can do better than that. Case 1 (M ~ N) We can use the fact the arrays are sorted to traverse both of them in an in-order manner at the same time. The general idea of the algorithm is to use two indices, i and j, for arr1 and arr2, respectively. Every time one of indices, let's say i without any loss of generality, points to a value that is smaller than the value pointed by other index, we increment i. If arr1[i] == arr2[j], we add the value to the output array and increment both indices. Pseudocode:

```
function findDuplicates(arr1, arr2):
   duplicates = []
   i = 0
   j = 0

   while (i < arr1.length AND j < arr2.length):
      if (arr1[i] == arr2[j]):
         duplicates.push(arr1[i])
         i = i + 1
         j = j + 1
      else if (arr1[i] < arr2[j]):
         i = i + 1
      else:
         j = j + 1

   return duplicates
```

Time Complexity: O(N+M) since in the worst case scenario we traverse both arrays once. Note that O(N+M) is a linear time complexity because the input size is O(N+M) as well. Space Complexity: the variable duplicates is the only dynamic auxiliary space we're using in the algorithm. In the worst case scenario, the size of duplicates is going to be as big as big as the smaller input array. For instance, when the smaller array is fully contained within the bigger one. The space complexity is therefore O(N), where N <= M. Case 2 (M >> N) When one array is substantially longer than the other, we should try to avoid traversing the longer one. Instead, we can traverse the shorter array and look up its values in the longer array by using the binary search algorithm. We explain why this approach is superior in this case to the previous one in the complexity analysis section. Pseudocode:

OFFICIAL PAGE TEXT

```
function findDuplicates(arr1, arr2):
    duplicates = []

    for number in arr1:
        if binarySearch(arr2, number) != -1:
            duplicates.push(number);

    return duplicates

function binarySearch(arr, num):
    begin = 0
    end = arr.length - 1

    while (begin <= end):
        mid = begin + floor((end-begin)/2)
        if arr[mid] < num:
            begin = mid + 1
        else if arr[mid] == num:
            return mid
        else:
            end = mid - 1

    return -1
```

Time Complexity: we running a binary search on arr2 N times. Hence the time complexity is O(N*log(M)). So why is this algorithm better than the previous one when M >> N? To demonstrate that let's analyze the case that M = N^C, where C is a constant such that C >= 2. In this case, the time complexity of the second algorithm is:

```
O(N*log(M)) = O(N*log(N^C)) = O(C*N*log(N)) = O(N*log(N))
```

And the time complexity of the first algorithm is:

```
O(N + M) = O(N + N^C) = O(N^C)
```

As we all know O(N^C) > O(N*log(N)). Space Complexity: the variable duplicates is the only dynamic auxiliary space we're using in the algorithm. In the worst case scenario, the size of duplicates is going to be as big as big as the smaller input array. For instance, when the smaller array is fully contained within the bigger one. The space complexity is therefore O(N), where N <= M. No answers contributed by the community yet.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 8 OF 22 |

bisect on a sorted list: find a position, count a range, keep it sorted

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

bisect on a sorted list: find a position, count a range, keep it sorted | 1 questions | tap a title to jump to it

READ FIRST: the code template for these 1 questions (tap here) >

E 16. Count People in a Postal Code Range **ColorStack 2025-10-19**

CODE TEMPLATE | GROUP 8 OF 22 | 1 QUESTIONS, ONE PATTERN

bisect on a sorted list: find a position, count a range, keep it sorted

The one idea these questions share

bisect_left and bisect_right give the position of a value in a sorted list in O(log n). Subtract two positions to count a range, slice between them to list a range, insort to keep inserting in sorted order, and replace an element at a position for the 'longest increasing' trick.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from bisect import bisect_left, bisect_right, insort


def count_in_range(sorted_list, low, high):
    hi = bisect_right(sorted_list, high)       # CHANGE: which positions you read
    lo = bisect_left(sorted_list, low)
    return hi - lo


# insort(sorted_list, x)                    keeps the list sorted as items arrive
# sorted_list[i]                            rank queries: i-th smallest, sorted_list[-1 - k] = k-th largest
```

2. Helper functions that usually get written next to it

```
def lis_length(values):                         # the tails list for "longest increasing"
    tails = []
    for v in values:
        i = bisect_left(tails, v)
        if i == len(tails):
            tails.append(v)
        else:
            tails[i] = v
    return len(tails)


def nearest(sorted_list, x):
    i = bisect_left(sorted_list, x)
    near = [sorted_list[j] for j in (i - 1, i) if 0 <= j < len(sorted_list)]
    return min(near, key=lambda v: abs(v - x))
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
16	Count People in a Postal Code Range	sort the codes once; the answer is the template as written, never below 0

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 16 of 38 | topic 8 of 22: bisect on a sorted list: find a position, count a range, keep it sorted | 1 of 1

Template fit: sort the codes once; the answer is the template as written, never below 0 | asked 366 days ago | frequency Single report

EASY | ColorStack | technical interview | 16 of 38

Count People in a Postal Code Range

ASKED 2025-10-19 | 366 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 239/288

ABOUT THIS QUESTION

Topic: bisect on a sorted list: find a position, count a range, keep it sorted

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #327 Count of Range Sum

LeetCode match: LC 327 Count of Range Sum (related: sorted codes and bisect)

Frequency: Single report | Last asked: 2025-10-19 | Roles: SWE Intern

QUESTION

You are given a list of people, each as a tuple (name, postal_code) . Build a structure that can answer many queries of the form: "how many people have a postal code within the inclusive range [low, high] ?" The interviewer explicitly wanted the optimal query implementation — not a linear scan of the people (or a scan over the range and a hash-map lookup for each value). Preprocess once, then answer each query in O(log n) . Implement a class with: a constructor that accepts the list of people, and count_in_range(low, high) returning the number of people whose postal code is in [low, high] inclusive.

Function signature

```
from typing import List, Tuple
class PostalRegistry:
    def __init__(self, people: List[Tuple[str, int]]) -> None:
        """Preprocess the (name, postal_code) pairs for fast range counts."""
        ...
    def count_in_range(self, low: int, high: int) -> int:
        """Return how many people have postal_code in [low, high] inclusive."""
        ...
```

Follow-up (as reported): Also return the list of matching names. Hint: sort by (postal_code, name) and slice between bisect_left(low) and bisect_right(high) .

EXAMPLES

Example 1	
Input	`people = [("Alice", 90210), ("Bob", 10001), ("Cara", 90211), ("Dan", 30301)]` `count_in_range(90000, 99999)`
Output	`2`
Why	`Alice (90210) and Cara (90211) fall in the range.`

Example 2	
Input	`same people; count_in_range(10001, 30301)`
Output	`2`
Why	`Bob (10001) and Dan (30301); range is inclusive on both ends.`

QUESTION

Example 3	
Input	`same people; count_in_range(50000, 60000)`
Output	`0`
Why	`No postal codes in that range.`

CONSTRAINTS

- 0 <= number of people <= 10^5 ; the same postal code may appear for multiple people (duplicates allowed).
- Postal codes are integers, 0 <= postal_code <= 10^9 .
- 0 <= low <= high <= 10^9 .
- Each count_in_range call must run in O(log n) after an O(n log n) preprocessing step.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
class ScanRegistry:
    def __init__(self, people):
        self.people = list(people)
    def count_in_range(self, low, high):
        return sum(
            1
            for _, code in self.people
            if low <= code <= high
        )
```

TIME	SPACE
O(n) per query	**O(1)**

WHY IT IS SLOW

Every query touches every person.

OPTIMAL SOLUTION · Sorted codes and two bisects

TIME	SPACE
O(log n) per query	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Also return the names of the people in the range.

Store the (code, name) pairs sorted. Find the two positions with bisect (using (low, '') and (high, a sentinel after every name)) and slice: the names come out ordered by code, then name. O(log n + answer size).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count People in a Postal Code Range
#
# You are given a list of people, each as a tuple (name,
# postal_code) . Build a structure that can answer many
# queries of the form: "how many people have a postal code
# within the inclusive range [low, high] ?" The
# interviewer explicitly wanted the optimal query
# implementation — not a linear scan of the people (or a
# scan over the range and a hash-map lookup for each
# value). Preprocess once, then answer each query in O(log
# n) . Implement a class with: a constructor that accepts
# the list of people, and count_in_range(low, high)
# returning the number of people whose postal code is in
# [low, high] inclusive. Function signature: from typing
# import List, Tuple class PostalRegistry: def
# __init__(self, people: List[Tuple[str, int]]) -> None:
# """Preprocess the (name, postal_code) pairs for fast
# range counts.""" ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Also return the names of the people in the range.
#
# INTERVIEW NOTES
# Sort the postal codes once. The number of codes in [low,
# high] is the position just after the last code <= high
# minus the position of the first code >= low, two binary
# searches with bisect, O(log n) per query. Sorting (code,
# name) pairs lets you also return the names by slicing
# between the same two positions.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_left, bisect_right


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: keep only the postal codes, sorted. The codes in the range
```

```
# [low, high] form one block of the sorted list: bisect_left finds where
# it starts, bisect_right where it ends. The count is the block's size.
# people is a list of (name, postal code).
# Complexity: Time build O(n log n), each query O(log n), Space O(n)
class PostalRegistry:
    def __init__(self, people):
        self.codes = []
        for name, code in people:
            self.codes.append(code)
        self.codes.sort()

    def count_in_range(self, low, high):
        count = bisect_right(self.codes, high) - bisect_left(self.codes, low)
        if count < 0:
            return 0                        # low is bigger than high
        return count


# ---- main question: tests ----
PEOPLE = [("Alice", 90210), ("Bob", 10001), ("Cara", 90211), ("Dan", 30301)]
```

```
def test_main_question():
    registry = PostalRegistry(PEOPLE)
    assert registry.count_in_range(90000, 99999) == 2
    assert registry.count_in_range(10001, 30301) == 2
    assert registry.count_in_range(50000, 60000) == 0
    assert registry.count_in_range(99999, 90000) == 0


# =============================================================
# FOLLOW-UP 1: Also return the names of the people in the range
# =============================================================
# ---- follow-up 1: function ----
# Answer: keep (code, name) pairs sorted. Find the two ends with bisect
# (from (low, "") up to just before (high + 1, "")) and slice: the names
# come out ordered by code, then by name. O(log n + size of the answer).
# Complexity: Time build O(n log n), each query O(log n + size of the answer), Space O(n)
class NamedPostalRegistry:
    def __init__(self, people):
        self.items = []
        for name, code in people:
            self.items.append((code, name))
        self.items.sort()
```

```
    def names_in_range(self, low, high):
        start = bisect_left(self.items, (low, ""))
        end = bisect_left(self.items, (high + 1, ""))
        names = []
        for code, name in self.items[start:end]:
            names.append(name)
        return names


# ---- follow-up 1: tests ----
def test_follow_up_1():
    registry = NamedPostalRegistry(PEOPLE)
    assert registry.names_in_range(10001, 30301) == ["Bob", "Dan"]
    assert registry.names_in_range(90000, 99999) == ["Alice", "Cara"]
    assert registry.names_in_range(1, 2) == []


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 9 OF 22 |

Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 3 questions | tap a title to jump to it

READ FIRST: the code template for these 3 questions (tap here) >

E	17. Cake Balance Line	ColorStack	2025-09-26
E	18. Find the Pivot Index (Balanced Split) in O(n)	ColorStack	2025-09-21
E	19. Parity Alternating Subarray Query	HackTheRounds	no date

CODE TEMPLATE | GROUP 9 OF 22 | 3 QUESTIONS, ONE PATTERN

Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

The one idea these questions share

pre[i] is the total of the first i items. Any range sum is pre[r + 1] - pre[l], and a balance point is an index where the left total equals the right total. Count something other than values (equal pairs, ones, repeats) by accumulating that instead.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def build_prefix(nums):
    pre = [0]
    for x in nums:                          # CHANGE 1: what you accumulate (value, 1 per match, 1 per one)
        pre.append(pre[-1] + x)
    return pre


def range_sum(pre, l, r):                   # inclusive l..r
    return pre[r + 1] - pre[l]


def pivot_index(nums):
    pre = build_prefix(nums)
    for i in range(len(nums)):              # CHANGE 2: the test at each index
        if pre[i] == pre[-1] - pre[i + 1]:
            return i
    return -1
```

2. Helper functions that usually get written next to it

```
def first_last(values):                     # first and last position of every value
    first, last = {}, {}
    for i, v in enumerate(values):
        first.setdefault(v, i)
        last[v] = i
    return first, last
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
17	Cake Balance Line	the same balance test on the cake heights
18	Find the Pivot Index (Balanced Split) in O(n)	the template as written: left total equals total - left - x
19	Parity Alternating Subarray Query	pre counts neighbouring pairs of equal parity; a range alternates when pre[r] - pre[l] == 0

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 17 of 38 | topic 9 of 22: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 1 of 3

Template fit: the same balance test on the cake heights | asked 389 days ago | frequency Single report

EASY | IN APP LC 724 | ColorStack | technical interview | 17 of 38

Cake Balance Line

ASKED 2025-09-26 | 389 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 33.2 | 30 DAYS 0 | PRIORITY 135/288

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

Optimal answer from: your app's interview approach (LeetCode #724)

LeetCode match: LC 724 Find Pivot Index (cake version)

Frequency: Single report | Last asked: 2025-09-26 | Roles: SWE Intern

QUESTION

A cake is described by a list heights , where heights[i] is the amount of cake sitting at position i (positions are left to right). You want to place a single vertical cut exactly at some position i so that the total amount of cake strictly to the left of the cut equals the total amount strictly to the right of the cut (the cake at position i itself is consumed by the cut and counts for neither side). Return the smallest index i where sum(heights[:i]) == sum(heights[i+1:]) , or -1 if no such position exists.

Function signature

```
def cake_balance_index(heights: list[int]) -> int:
    ...
```

Follow-up (as reported): Do it in one pass and O(1) extra space by tracking a running left sum against total - left - heights[i] .

EXAMPLES

Example 1	
Input	`heights = [1, 7, 3, 6, 5, 6]`
Output	`3`
Why	`Left of index 3 is 1+7+3=11; right of index 3 is 5+6=11.`

Example 2	
Input	`heights = [1, 2, 3]`
Output	`-1`
Why	`No cut position balances the two sides.`

Example 3	
Input	`heights = [2, 1, -1]`
Output	`0`
Why	`Left of index 0 is empty (0); right is 1+(-1)=0.`

CONSTRAINTS

- 1 <= len(heights) <= 10^5 .
- -10^4 <= heights[i] <= 10^4 (heights may be negative in this abstracted version).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_cake(heights):
    n = len(heights)
    prefix, total = [0] * n, 0
    for i, h in enumerate(heights):
        prefix[i] = total
        total += h
    suffix, total = [0] * n, 0
    for i in range(n_-_1,_-1,_-1):
        suffix[i] = total
        total += heights[i]
    for i in range(n):
        if prefix[i] == suffix[i]:
            return i
    return -1
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

It stores both running sums although one number is enough.

OPTIMAL SOLUTION · Running left sum

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Which cut makes the two sides as equal as possible?

Same scan, but track the smallest absolute difference |left - (total - left - h)| and the first index that reaches it. If a perfectly balanced cut exists it wins with difference 0.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Cake Balance Line
#
# A cake is described by a list heights , where heights[i]
# is the amount of cake sitting at position i (positions
# are left to right). You want to place a single vertical
# cut exactly at some position i so that the total amount
# of cake strictly to the left of the cut equals the total
# amount strictly to the right of the cut (the cake at
# position i itself is consumed by the cut and counts for
# neither side). Return the smallest index i where
# sum(heights[:i]) == sum(heights[i+1:]) , or -1 if no
# such position exists. Function signature: def
# cake_balance_index(heights: list[int]) -> int: ...
# Follow-up (as reported): Do it in one pass and O(1)
# extra space by tracking a running left sum against total
# - left - heights[i] .
```

```
#
# FOLLOW-UPS (answered below, in this order):
# 1. Which cut makes the two sides as equal as possible?
#
# INTERVIEW NOTES
# Same as the pivot index: the cut at i balances when the
# left sum equals total - left - heights[i]. Keep the left
# sum as a running value; no prefix array is needed.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Find a column of the cake whose left side weighs as much as its right
# side. Idea: the right side is total - left - heights[i], so keep a
# running left sum and compare in one pass.
# Complexity: Time O(n), Space O(1)
def cake_balance_index(heights):
    total = 0
    for h in heights:
        total += h
    left = 0
```

```
    for i in range(len(heights)):
        if left == total - left - heights[i]:
            return i
        left += heights[i]
    return -1


# ---- main question: tests ----
def test_main_question():
    assert cake_balance_index([1, 7, 3, 6, 5, 6]) == 3
    assert cake_balance_index([1, 2, 3]) == -1
    assert cake_balance_index([2, 1, -1]) == 0


# ========================================================
# FOLLOW-UP 1: Which cut makes the two sides as equal as possible?
# ========================================================
# ---- follow-up 1: function ----
# Answer: the same scan, but track the smallest difference
# |left - right| and the first index that reaches it. A perfectly
# balanced cut wins with difference 0.
# Complexity: Time O(n), Space O(1)
def closest_cut(heights):
    total = 0
    for h in heights:
        total += h
    left = 0
    best_gap = None
    best_index = -1
    for i in range(len(heights)):
        gap = left - (total - left - heights[i])
        if gap < 0:
            gap = -gap
        if best_gap is None or gap < best_gap:
            best_gap = gap
```

```
            best_index = i
        left += heights[i]
    return best_index


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert closest_cut([1, 7, 3, 6, 5, 6]) == 3
    assert closest_cut([5, 1]) == 0
    assert closest_cut([]) == -1
```

```
    for code in range(8 ** 6):                          # every list of 6 heights from -3..4: compare with a
dding up both sides
        h = []
        for i in range(6):
            h.append(code // 8 ** i % 8 - 3)
        best = None
        expected = -1
        for i in range(6):
            left = 0
            for j in range(i):
                left += h[j]
            right = 0
            for j in range(i + 1, 6):
                right += h[j]
            gap = abs(left - right)
            if best is None or gap < best:
                best = gap
                expected = i
        assert closest_cut(h) == expected


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 18 of 38 | topic 9 of 22: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 2 of 3

Template fit: the template as written: left total equals total - left - x | asked 394 days ago | frequency Single report

EASY | IN APP LC 724 | ColorStack | technical interview | 18 of 38

Find the Pivot Index (Balanced Split) in O(n)

ASKED 2025-09-21 | 394 DAYS AGO | OLDER

STUDY: TALK ALL TIME 33.2 30 DAYS 0 PRIORITY 134/288

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

Optimal answer from: your app's interview approach (LeetCode #724)

LeetCode match: LC 724 Find Pivot Index

Frequency: Single report | Last asked: 2025-09-21 | Roles: SWE Intern

QUESTION

Given an integer array nums , return the leftmost index i such that the sum of all elements strictly to the left of i equals the sum of all elements strictly to the right of i . The element at i itself is not counted on either side. For the first index, the left sum is 0 ; for the last index, the right sum is 0 . If no such index exists, return -1 . Your solution must run in O(n) time and O(1) extra space.

Function signature

```
def pivot_index(nums: list[int]) -> int:
    ...
```

Follow-up (as reported): If several pivots exist, return them all. Hint: keep scanning after the first match, maintaining the running left sum.

EXAMPLES

Example 1	
Input	`nums = [1, 7, 3, 6, 5, 6]`
Output	`3`
Why	`Left of index 3: 1+7+3 = 11. Right of index 3: 5+6 = 11.`

Example 2	
Input	`nums = [1, 2, 3]`
Output	`-1`
Why	`No index balances the two sides.`

Example 3	
Input	`nums = [2, 1, -1]`
Output	`0`
Why	`Left of index 0 is 0; right is 1 + (-1) = 0.`

CONSTRAINTS

- 1 <= len(nums) <= 10^4
- -1000 <= nums[i] <= 1000

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_pivot(nums):
    for i in range(len(nums)):
        if sum(nums[:i]) == sum(nums[i + 1 :]):
            return i
    return -1
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Each index re-adds the whole array.

OPTIMAL SOLUTION · Running left sum

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Several pivots can exist: return all of them.

Do not return at the first match; collect every index where the left sum equals the right sum while the scan continues. Still O(n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the Pivot Index (Balanced Split) in
# O(n)
#
# Given an integer array nums , return the leftmost index
# i such that the sum of all elements strictly to the left
# of i equals the sum of all elements strictly to the
# right of i . The element at i itself is not counted on
# either side. For the first index, the left sum is 0 ;
# for the last index, the right sum is 0 . If no such
# index exists, return -1 . Your solution must run in O(n)
# time and O(1) extra space. Function signature: def
# pivot_index(nums: list[int]) -> int: ... Follow-up (as
# reported): If several pivots exist, return them all.
# Hint: keep scanning after the first match, maintaining
# the running left sum.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Several pivots can exist: return all of them.
#
# INTERVIEW NOTES
# At the pivot, left sum equals total minus left sum minus
# the element. So sweep once keeping the running left sum
# and compare it with total - left - nums[i]; no second
# array is needed, which gives O(1) extra space.
```

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: the sum on the right of index i is total - left - nums[i], so one
# pass with a running left sum is enough.
# Complexity: Time O(n), Space O(1)
def pivot_index(nums):
    total = 0
    for x in nums:
        total += x
    left = 0
    for i in range(len(nums)):
        if left == total - left - nums[i]:
            return i
        left += nums[i]
    return -1


# ---- main question: tests ----
def test_main_question():
    assert pivot_index([1, 7, 3, 6, 5, 6]) == 3
    assert pivot_index([1, 2, 3]) == -1
    assert pivot_index([2, 1, -1]) == 0


# ========================================================
# FOLLOW-UP 1: Several pivots can exist: return all of them
# ========================================================
# ---- follow-up 1: function ----
# Answer: do not return at the first match; collect every index whose left
# sum equals its right sum while the scan goes on. Still O(n).
# Complexity: Time O(n), Space O(1) plus the answer
def all_pivots(nums):
    total = 0
    for x in nums:
        total += x
    left = 0
    found = []
    for i in range(len(nums)):
        if left == total - left - nums[i]:
            found.append(i)
        left += nums[i]
    return found


# ---- follow-up 1: tests ----
def slow_pivots(nums):
    # check every index by adding up both sides from scratch
    found = []
    for i in range(len(nums)):
        left = 0
        for j in range(i):
            left += nums[j]
        right = 0
        for j in range(i + 1, len(nums)):
            right += nums[j]
```

```
        if left == right:
```

```
            found.append(i)
    return found


def test_follow_up_1():
    assert all_pivots([0, 0, 0]) == [0, 1, 2]
    for code in range(7 ** 6):                          # every array of 6 numbers from -3..3
        nums = []
        for i in range(6):
            nums.append(code // 7 ** i % 7 - 3)
        assert all_pivots(nums) == slow_pivots(nums)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 19 of 38 | topic 9 of 22: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 3 of 3

Template fit: pre counts neighbouring pairs of equal parity; a range alternates when pre[r] - pre[l] == 0 | no date

EASY | HackTheRounds | Onsite | 19 of 38

Parity Alternating Subarray Query

STUDY: TALK **ALL TIME 34.9** **30 DAYS 0** **PRIORITY 129/288**

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #303 Range Sum Query - Immutable

LeetCode match: No direct match; prefix-sum range queries (idea of LC 303)

QUESTION

Given an array nums, answer many queries alternates(l, r): do the elements nums[l], ..., nums[r] (both ends included) alternate in parity (odd, even, odd, ... or even, odd, even, ...)? Preprocess the array once so that every query takes O(1).

Function signature

class ParityQueries:
  def __init__(self, nums: list)
  def alternates(self, l: int, r: int) -> bool

EXAMPLES

Example 1

Input	`p = ParityQueries([1, 2, 3, 5, 4])`
	`p.alternates(0, 2)`
	`p.alternates(0, 3)`
	`p.alternates(3, 4)`

Output	`p.alternates(0, 2) -> true`
	`p.alternates(0, 3) -> false`
	`p.alternates(3, 4) -> true`

Why	`1, 2, 3 alternates; the neighbours 3 and 5 are both odd, so any range containing both fails; 5, 4 alternates.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= queries.length <= 10^5
- 0 <= l <= r < nums.length

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def alternates(nums, l, r):
    for i in range(l, r):
        if nums[i] % 2 == nums[i + 1] % 2:
            return False
    return True
```

BRUTE FORCE

TIME	SPACE
O(length) per query	**O(1)**

WHY IT IS SLOW

Each query scans up to n elements. A prefix count of equal-parity neighbour pairs answers every query in O(1) after one O(n) pass.

OPTIMAL SOLUTION · Prefix count of equal-parity neighbour pairs

TIME	SPACE
O(n) to build, O(1) per query	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The array also changes (point updates).

Keep the bad-pair indicators in a Fenwick tree: an update changes at most two indicators, each an O(log n) update, and a query is two prefix sums.

FOLLOW-UP 2 Find the longest alternating subarray.

One pass with a running length that resets to 1 on a bad pair. O(n), O(1).

FOLLOW-UP 3 Offline queries, different property (for example all elements distinct).

The prefix-sum trick applies whenever the property is 'no violating neighbour pair'. Properties about distinctness need Mo's algorithm or an offline sweep with last-occurrence tracking. **Cost:** Mo's algorithm is O((n + q) sqrt(n)); the prefix trick for 'no violating neighbour pair' stays O(n + q).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Parity Alternating Subarray Query
#
# Given an array nums, answer many queries alternates(l,
# r): do the elements nums[l], ..., nums[r] (both ends
# included) alternate in parity (odd, even, odd, ... or
# even, odd, even, ...)? Preprocess the array once so that
# every query takes O(1).
#
# Signature:
#   class ParityQueries:
#       def __init__(self, nums: list)
#       def alternates(self, l: int, r: int) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. The array also changes (point updates).
# 2. Find the longest alternating subarray.
# 3. Offline queries, different property (for example all
# elements distinct).
#
```

```
# INTERVIEW NOTES
# A subarray alternates in parity exactly when it contains
# no pair of neighbours with the same parity. 'Count of
# bad pairs in a range' is a prefix sum question, so
# precompute once and answer every query with one
# subtraction. Mark each position i by whether (nums[i],
# nums[i + 1]) is a bad pair.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main class ----
# Does the subarray nums[l..r] alternate between even and odd numbers?
# Idea: write down, for the first i neighbouring pairs, how many pairs have
# the SAME parity (bad[i]). The pairs inside nums[l..r] are the pairs
# l, l+1, ..., r-1, so the subarray alternates when bad[r] - bad[l] == 0.
```

```
# Building is O(n); every query is O(1).
# Complexity: Time build O(n), each query O(1), Space O(n)
class ParityQueries:
    def __init__(self, nums):
        self.bad = [0]
        for i in range(len(nums) - 1):
            same = 0
            if nums[i] % 2 == nums[i + 1] % 2:
                same = 1
            self.bad.append(self.bad[len(self.bad) - 1] + same)

    def alternates(self, l, r):
        return self.bad[r] - self.bad[l] == 0


# ---- main question: tests ----
def test_main_question():
    q = ParityQueries([1, 2, 3, 4, 5])
    assert q.alternates(0, 4) and q.alternates(1, 3)
    q = ParityQueries([1, 3, 2, 4])
    assert not q.alternates(0, 1)
    assert not q.alternates(0, 3)
    assert q.alternates(1, 2)
    assert q.alternates(2, 2)                          # a single element always alternates


# ========================================================
# FOLLOW-UP 1: The array also changes (point updates)
# ========================================================
# ---- follow-up 1: helper functions ----
class Fenwick:
    # a Fenwick tree: add to one position and ask for the sum of the first i
    # positions, both in O(log n). (i & -i is the lowest set bit of i.)
    def __init__(self, size):
        self.size = size
        self.tree = [0] * (size + 1)

    def add(self, i, delta):
```

```
        i += 1
        while i <= self.size:
            self.tree[i] += delta
            i += i & -i

    def prefix(self, i):
        total = 0
```

```
        while i > 0:
            total += self.tree[i]
            i -= i & -i
        return total


# ---- follow-up 1: function ----
# Answer: keep the "same parity" flags in a Fenwick tree. Changing one
# number changes at most two flags (the pairs on its left and on its
# right), each an O(log n) update; a query is two prefix sums.
# Complexity: Time build O(n log n), update O(log n), query O(log n), Space O(n)
class ParityWithUpdates:
    def __init__(self, nums):
        self.nums = list(nums)
        self.flags = [0] * max(len(nums) - 1, 0)
        self.tree = Fenwick(len(self.flags))
        for i in range(len(self.flags)):
            self.refresh(i)

    def refresh(self, i):
        # recompute the flag of the pair (i, i + 1)
        now = 0
        if self.nums[i] % 2 == self.nums[i + 1] % 2:
            now = 1
        self.tree.add(i, now - self.flags[i])
        self.flags[i] = now

    def update(self, i, value):
        self.nums[i] = value
        for pair in (i - 1, i):
            if 0 <= pair < len(self.flags):
                self.refresh(pair)

    def alternates(self, l, r):
        return self.tree.prefix(r) - self.tree.prefix(l) == 0


# ---- follow-up 1: tests ----
def test_follow_up_1():
    random.seed(11)
    for round_number in range(200):
        nums = []
        for i in range(random.randint(1, 10)):
            nums.append(random.randint(0, 9))
        live = ParityWithUpdates(nums)
```

```
        for step in range(20):
            if random.random() < 0.4:
                i = random.randrange(len(nums))
                nums[i] = random.randint(0, 9)
                live.update(i, nums[i])
            l = random.randrange(len(nums))
            r = random.randrange(l, len(nums))
            assert live.alternates(l, r) == ParityQueries(nums).alternates(l, r)


# ============================================================
```

```python
# FOLLOW-UP 2: Find the longest alternating subarray
# ============================================================
# ---- follow-up 2: function ----
# Answer: one pass with a running length that restarts at 1 after a pair
# of equal parity. O(n) time, O(1) space.
# Complexity: Time O(n), Space O(1)
def longest_alternating(nums):
    if not nums:
        return 0
    best = 1
    run = 1
    for i in range(1, len(nums)):
        if nums[i - 1] % 2 != nums[i] % 2:
            run += 1
        else:
            run = 1
        if run > best:
            best = run
    return best


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert longest_alternating([1, 3, 2, 4]) == 2
    assert longest_alternating([1, 2, 3, 4, 5]) == 5
    assert longest_alternating([]) == 0
    assert longest_alternating([7]) == 1


# ============================================================
# FOLLOW-UP 3: Offline queries, different property (for example all elements distinct)
# ============================================================
# Answer: a property that is NOT a sum of local pair flags needs another
# tool: sort the queries by their right end and sweep (offline), keeping the
```

```python
# last position of every value in a Fenwick tree.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

ALSO REPORTED · 2 more

Answer parity-alternation range queries | PracHub | Technical Screen

https://prachub.com/coding-questions/answer-parity-alternation-range-queries

Answer range queries for alternating parity subarrays | PracHub | Technical Screen

https://prachub.com/coding-questions/answer-range-queries-for-alternating-parity-subarrays

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 10 OF 22

Sorted intervals: merge while they touch, then read the gaps or search

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Sorted intervals: merge while they touch, then read the gaps or search | 1 questions | tap a title to jump to it

READ FIRST: the code template for these 1 questions (tap here) >

E 20. Check an Interval Against Existing Intervals for Conflicts **PracHub 2026-08-08**

CODE TEMPLATE | GROUP 10 OF 22 | 1 QUESTIONS, ONE PATTERN

Sorted intervals: merge while they touch, then read the gaps or search

The one idea these questions share

Sort by start. Walk once: if the next interval starts before the last one ends, extend the last; otherwise start a new one. The merged list is sorted and disjoint, so gaps are the spaces between neighbours and any point can be found with bisect.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def merge(intervals):
    out = []
    for s, e in sorted(intervals):
        if out and s <= out[-1][1]:            # CHANGE 1: overlap or touch? (< to keep touching ones apart)
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

2. Helper functions that usually get written next to it

```
from bisect import bisect_left, bisect_right


def gaps(merged, lo=float("-inf"), hi=float("inf")):
    edges = [lo] + [x for iv in merged for x in iv] + [hi]
    return [(edges[i], edges[i + 1]) for i in range(0, len(edges), 2)]


def containing(merged, x):
    starts = [s for s, _ in merged]
    i = bisect_right(starts, x) - 1
    return tuple(merged[i]) if i >= 0 and merged[i][1] >= x else None
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
20	Check an Interval Against Existing Intervals for Conflicts	bisect on the starts; a new booking conflicts if the previous one ends after its start or the next one starts before its end

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 20 of 38 | topic 10 of 22: Sorted intervals: merge while they touch, then read the gaps or search | 1 of 1

Template fit: bisect on the starts; a new booking conflicts if the previous one ends after its start or the next one starts before its end | asked 73 days ago

EASY | IN APP LC 729 | PracHub | Technical Screen | 20 of 38

Check an Interval Against Existing Intervals for Conflicts

POSTED 2026-08-08 | 73 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 26.5 30 DAYS 0 PRIORITY 153/288

ABOUT THIS QUESTION

Topic: Sorted intervals: merge while they touch, then read the gaps or search

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #729 My Calendar I

LeetCode match: LC 729 My Calendar I (inferred from overview)

Posted on PracHub: 2026-08-08 (updated 2026-08-29)

Original: https://prachub.com/coding-questions/check-an-interval-against-existing-intervals-for-conflicts

QUESTION

Design a calendar of half-open intervals [start, end). book(start, end) books the interval and returns True when it does not overlap any interval booked so far, and returns False (booking nothing) when it does. Intervals that only touch, one ending exactly where the next starts, do not conflict.

Function signature

class Calendar:

    def book(self, start: int, end: int) -> bool

EXAMPLES

Example 1

Input	`c = Calendar()`
	`c.book(10, 20)`
	`c.book(15, 25)`
	`c.book(20, 30)`
	`c.book(5, 10)`
	`c.book(12, 14)`

Output	`c.book(10, 20) -> true`
	`c.book(15, 25) -> false`
	`c.book(20, 30) -> true`
	`c.book(5, 10) -> true`
	`c.book(12, 14) -> false`

Why	`[15, 25) overlaps [10, 20) and [12, 14) lies inside it, so both fail. [20, 30) and [5, 10) only touch the booked interval at an endpoint, so they succeed.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
booked = []
def book(s, e):
    for s2, e2 in booked:
        if s < e2 and s2 < e:
            return False  # overlap
    booked.append((s, e))
    return True
```

BRUTE FORCE

TIME	SPACE
O(n) per booking	O(n)

WHY IT IS SLOW

Each booking scans all earlier bookings, O(n). With the intervals sorted, binary search finds the only two neighbours that could conflict.

OPTIMAL SOLUTION · Sorted starts with bisect: check only two neighbours

TIME	SPACE
O(log n) search, O(n) list insert	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Allow double booking but never triple booking (LeetCode 731).

Keep a difference map (+1 at start, -1 at end) and sweep it in sorted order; if the running count ever exceeds 2, undo the change and reject. O(n log n) per booking, or use a segment tree for faster.

FOLLOW-UP 2 Report the maximum number of overlapping bookings after each booking (LeetCode 732).

The same difference map, taking the maximum running count in the sweep. A segment tree with lazy range increments gives O(log n) per booking.

FOLLOW-UP 3 Closed intervals instead of half-open.

Then touching endpoints conflict, so the tests become <= instead of <. Say it, and keep the choice in one comparison.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Check an Interval Against Existing
# Intervals for Conflicts
#
# Design a calendar of half-open intervals [start, end).
# book(start, end) books the interval and returns True
# when it does not overlap any interval booked so far, and
# returns False (booking nothing) when it does. Intervals
# that only touch, one ending exactly where the next
# starts, do not conflict.
#
# Signature:
#   class Calendar:
#       def book(self, start: int, end: int) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. Allow double booking but never triple booking
# (LeetCode 731).
# 2. Report the maximum number of overlapping bookings
# after each booking (LeetCode 732).
# 3. Closed intervals instead of half-open.
```

```
#
# INTERVIEW NOTES
# Two half-open intervals [s1, e1) and [s2, e2) overlap
# exactly when s1 < e2 and s2 < e1. If the booked
# intervals are kept sorted and disjoint, only the
# neighbours at the insertion point can conflict, so a
# binary search replaces the scan.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
# Complexity: Time O(log n), Space O(1)
def first_greater(values, x):
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] <= x:
            low = middle + 1
        else:
            high = middle
    return low


# Complexity: Time O(n), Space O(1)
def conflicts(candidate, intervals):
    # the simple check against every booking: half-open [start, end), so
    # touching ends do NOT conflict
    start, end = candidate
    for a, b in intervals:
        if a < end and start < b:
            return True
    return False


# ---- main question: main class ----
# book(start, end) says whether the interval can be booked without
# overlapping an existing booking (and books it if so). Half-open intervals
# [start, end): a booking may start exactly when another ends.
# Idea: keep the bookings sorted by start. Then only the TWO neighbours of
# the new interval can conflict: the booking just before (it must end by the
# new start) and the booking just after (it must start at or after the new
# end). One binary search finds the place: O(log n) per check.
# Complexity: Time book is O(log n) to search plus O(n) for the list insert, Space O(n)
class Calendar:
    def __init__(self):
        self.starts = []
        self.ends = []

    def book(self, start, end):
        i = first_greater(self.starts, start)            # the new booking would go at position i
        if i > 0 and self.ends[i - 1] > start:
            return False                                 # the booking before runs into it
        if i < len(self.starts) and self.starts[i] < end:
            return False                                 # the booking after starts too early
```

```
        self.starts.insert(i, start)
        self.ends.insert(i, end)
        return True


# ---- main question: tests ----
```

```
def test_main_question():
    taken = [(10, 20), (30, 40)]
    assert conflicts((15, 25), taken)
    assert not conflicts((20, 30), taken)
    assert conflicts((19, 31), taken)
    calendar = Calendar()
    assert calendar.book(10, 20) and calendar.book(20, 30)
    assert not calendar.book(15, 25) and calendar.book(5, 10)


# ========================================================
# FOLLOW-UP 1: Allow double booking but never triple booking (LeetCode 731)
# ========================================================


# ========================================================
# FOLLOW-UP 2: Report the maximum number of overlapping bookings after each booking (LeetCode 732)
# ========================================================
# ---- follow-up 1: function ----
# Answer: keep a difference map (+1 at every start, -1 at every end) and
# sweep it in time order; the running total is the number of overlapping
# bookings. If the biggest total ever exceeds the limit (2 for LeetCode 731),
# undo the booking and reject it. O(n log n) per booking (a segment tree makes
# it faster). peak() is also the answer to LeetCode 732.
# Complexity: Time book is O(n log n) (it re-sorts the boundary times), Space O(n)
class DoubleBooking:
    def __init__(self, limit=2):
        self.delta = {}
        self.limit = limit

    def peak(self):
        # sweep the +1 / -1 changes in time order
        running = 0
        best = 0
        for t in sorted(self.delta):
            running += self.delta[t]
            if running > best:
                best = running
        return best

    def book(self, start, end):
        self.delta[start] = self.delta.get(start, 0) + 1
        self.delta[end] = self.delta.get(end, 0) - 1
        if self.peak() > self.limit:
            self.delta[start] -= 1                      # undo the booking and reject it
```

```
            self.delta[end] += 1
            return False
        return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    d = DoubleBooking()
    assert d.book(10, 20) and d.book(15, 25)            # double booking is allowed ...
    assert not d.book(17, 18)                           # ... triple booking is not
    assert d.peak() == 2
    one = DoubleBooking(limit=1)                        # limit 1 is the plain no-overlap calendar
```

```
    assert one.book(1, 5) and not one.book(4, 6)


# ============================================================
# FOLLOW-UP 3: Closed intervals instead of half-open
# ============================================================
# Answer: with closed intervals [start, end] two bookings that share an end
# DO conflict, so compare with <= instead of < (the previous booking must end
# before the new start; the next must start after the new end).


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def hasIntervalConflict(candidate: list[int], intervals: list[list[int]]) -> bool:
```

Sample call

```
hasIntervalConflict([4,7], [[1,3],[7,9]])
```

Test cases

Input	Expected
`([4,7],[[1,3],[7,9]])`	`False`
`([4,7],[[1,5],[8,9]])`	`True`

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 11 OF 22

A dict for lookup plus an ordered container for who is next (lazy deletion)

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

A dict for lookup plus an ordered container for who is next (lazy deletion) | 1 questions | tap a title to jump to it

READ FIRST: the code template for these 1 questions (tap here) >

E 21. Implement Batched Undo/Redo Layer **PracHub 2026-02-04**

CODE TEMPLATE | GROUP 11 OF 22 | 1 QUESTIONS, ONE PATTERN

A dict for lookup plus an ordered container for who is next (lazy deletion)

The one idea these questions share

Keep a dict for O(1) lookup and a deque, list or heap for the order. When something is removed, delete it from the dict only and skip stale entries when you read from the order. LRU caches, waitlists, undo stacks and schedulers all fit.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import deque


class OrderedStore:
    def __init__(self):
        self.order = deque()                  # CHANGE 1: the container that says who is next
        self.live = {}                        # key -> ticket, for O(1) lookup and lazy deletion
        self.ticket = 0

    def add(self, key):
        self.ticket += 1
        self.live[key] = self.ticket
        self.order.append((self.ticket, key))

    def remove(self, key):
        self.live.pop(key, None)              # lazy: the entry stays in the deque

    def next(self):
        while self.order and self.live.get(self.order[0][1]) != self.order[0][0]:
            self.order.popleft()              # CHANGE 2: skip entries that were removed
        return self.order[0][1] if self.order else None
```

2. Helper functions that usually get written next to it

```
from collections import OrderedDict


class LRU:                                    # the short version of the same idea
    def __init__(self, cap):
        self.cap, self.d = cap, OrderedDict()

    def get(self, k):
        if k not in self.d:
            return -1
        self.d.move_to_end(k)
        return self.d[k]

    def put(self, k, v):
        self.d[k] = v
        self.d.move_to_end(k)
        if len(self.d) > self.cap:
            self.d.popitem(last=False)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
21	Implement Batched Undo/Redo Layer	two stacks (undo, redo) of batches; a batch records (old value, new value) per property; undo applies the old values and moves the batch

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 21 of 38 | topic 11 of 22: A dict for lookup plus an ordered container for who is next (lazy deletion) | 1 of 1

Template fit: two stacks (undo, redo) of batches; a batch records (old value, new value) per property; undo applies the old values and moves the batch | asked 258 days ago

EASY | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 21 of 38

Implement Batched Undo/Redo Layer

POSTED 2026-02-04 | 258 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 179/288

ABOUT THIS QUESTION

Topic: A dict for lookup plus an ordered container for who is next (lazy deletion)

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #1472 Design Browser History

LeetCode match: LC 1472 Design Browser History (related: undo/redo stacks)

Posted on PracHub: 2026-02-04 (updated 2026-05-06)

Original: https://prachub.com/coding-questions/implement-batched-undo-redo-layer

QUESTION

You are implementing a simplified document layer for a design tool. The layer stores properties as a **`map<string, string>`**. Implement a class that supports the following operations:

- **`beginBatch()`**: start a batch of edits.
- **`setProperty(key, value)`**: set or overwrite a property inside the current batch.
- **`endBatch()`**: commit the batch as one undoable action.
- **`undo()`**: revert the most recently committed batch.
- **`redo()`**: reapply the most recently undone batch. Requirements:

1. A batch may update the same key multiple times.
2. **`undo()`** must restore the exact document state from before that batch.
3. **`redo()`** must restore the exact document state after that batch.
4. If a property did not exist before the batch, then **`undo()`** should remove that property.
5. If **`undo()`** has been called and then a new batch is committed, the redo history must be cleared.
6. Follow-up: make the history memory-efficient. Instead of storing every intermediate edit in a batch, compact the batch so that each key appears at most once in history while still preserving enough information for correct **`undo()`** and **`redo()`**. A useful compact record for one key is:

```
Change { string key; string value; // final value after the batch string old_value; // value before the ba
tch bool has_old_value; // whether the key existed before the batch }
```

Explain how you would build and use such a compact history so that repeated updates to the same key inside one batch do not waste memory.

QUESTION

Part 1: Implement Batched Undo/Redo for Document PropertiesYou are given a sequence of operations for an initially empty document represented as a map<string, string>. A batch starts with ('begin',), contains zero or more ('set', key, value) operations, and ends with ('end',). Each committed batch must be undoable and redoable as one unit. Operation ('snapshot',) asks you to append the current document to the answer; snapshots may appear while a batch is open and should reflect the live document state. If a key is set multiple times in one batch, undo must restore the value from before the batch began. If a key did not exist before the batch, undo must remove it. If a new non-empty batch is committed after an undo, the redo history is cleared. If undo or redo is impossible, do nothing. If end closes an empty batch, treat it as a no-op.

Constraints

0 <= len(operations) <= 200000The total number of ('set', key, value) operations is at most 200000Keys and values are strings of length 1 to 50Batches are not nestedIf a batch contains no set operations, committing it does not create history

EXAMPLES

```
Input: [('begin',), ('set', 'color', 'red'), ('end',), ('snapshot',), ('undo',), ('snapshot',),
    ('redo',), ('snapshot',)]
Expected Output: [{'color': 'red'}, {}, {'color': 'red'}]
```

Explanation: A single batch is committed, undone, and redone.

```
Input: [('begin',), ('set', 'a', '1'), ('end',), ('begin',), ('set', 'a', '2'), ('set', 'a', '3'),
    ('set', 'b', '9'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]
Expected Output: [{'a': '3', 'b': '9'}, {'a': '1'}, {'a': '3', 'b': '9'}]
```

Explanation: The second batch updates 'a' twice and adds 'b'. Undo must restore the exact pre-batch state.

Hints

For each key touched in the current batch, record its old value only the first time that key is edited in the batch.Use two stacks: one for committed batches that can be undone, and one for undone batches that can be redone. Clear the redo stack when a new non-empty batch is committed.

Part 2: Compact a Batch into Memory-Efficient Change RecordsYou are given the document state before a batch and the raw setProperty edits that happened inside that batch. Multiple edits may target the same key. Build a compact history so that each key appears at most once, while still storing enough informa

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
undo, redo = [], []
def commit_batch():
    undo.append(copy.deepcopy(props))
    redo.clear()
def do_undo():
    redo.append(copy.deepcopy(props))
    props.clear()
    props.update(undo.pop())
def do_redo():
    undo.append(copy.deepcopy(props))
    props.clear()
    props.update(redo.pop())
```

TIME	SPACE
O(size of document) per batch	**O(size * batches)**

WHY IT IS SLOW

BRUTE FORCE

A one-key change copies the whole document, O(size) per batch and O(size * batches) memory. Storing only the old and new value of each changed key makes each batch O(keys changed).

OPTIMAL SOLUTION · Store only the changed keys per batch

TIME	SPACE
O(keys changed in the batch) per operation	**O(total changed keys)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Limit the history to the last N batches.

Keep the undo stack in a deque with maxlen N so the oldest batch is dropped automatically. Redo is unaffected.

FOLLOW-UP 2 Nested batches.

Keep a stack of open batches and merge a child into its parent on endBatch (keep the parent's old value if the key was already touched). Only the outermost endBatch commits an undoable action.

FOLLOW-UP 3 Undo of individual setProperty calls, not whole batches.

Treat every setProperty outside a batch as a one-key batch (the code does), so undo granularity equals commit granularity.

FOLLOW-UP 4 Limit the history to the last N batches.

Cap the undo stack: when it grows beyond N drop the oldest batch (use a deque with maxlen, or pop(0) as here). The redo stack is unaffected. Memory is then O(N batches).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement Batched Undo/Redo Layer
#
# You are implementing a simplified document layer for a
# design tool. The layer stores properties as a
# map<string, string>. Implement a class that supports the
# following operations: - beginBatch(): start a batch of
# edits. - setProperty(key, value): set or overwrite a
# property inside the current batch. - endBatch(): commit
# the batch as one undoable action. - undo(): revert the
# most recently committed batch. - redo(): reapply the
# most recently undone batch. Requirements: 1. A batch may
# update the same key multiple times. 2. undo() must
# restore the exact document state from before that batch.
# 3. redo() must restore the exact document state after
# that batch. 4. If a property did not exist before the
# batch, then undo() should remove that property. 5. If
# undo() has been called and then a new batch is
# committed, the redo history must be cleared. 6. Follow-
# up: make the history memory-efficient.
#
# FOLLOW-UPS (answered below, in this order):
```

```
# 1. Limit the history to the last N batches.
# 2. Nested batches.
# 3. Undo of individual setProperty calls, not whole
# batches.
# 4. Limit the history to the last N batches.
#
# INTERVIEW NOTES
# Undo and redo are two stacks of actions. The question is
# what an action is: a batch is one action, so record per
# batch only what is needed to reverse it (the old value
# of each key touched, and the new one), not a whole copy
# of the document. A new edit after an undo kills the redo
# history.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Change:
    # one key changed inside one batch: its value before and after
    def __init__(self, key, old, had_old, new):
        self.key = key
        self.old = old              # the value before the batch (if it had one)
        self.had_old = had_old      # did the key exist before the batch?
        self.new = new              # the value at the end of the batch


def merge_batch(parent, child):
    # a nested batch folds into its parent: keep the parent's OLD value
    # when the key was already touched there, but take the newest value
    for key in child:
        if key in parent:
            parent[key].new = child[key].new
        else:
            parent[key] = child[key]


# ---- main question: main function ----
# Idea: while a batch is open, remember for every key touched the value
# it had BEFORE the batch (recorded only the first time the key is
# touched) and its latest value. end_batch puts the batch on the undo
# stack as ONE action and clears the redo stack. Undo restores the old
# values; redo applies the new ones.
# `history` limits how many batches are remembered (follow-up 1): a deque
# with a maximum length drops the oldest batch by itself.
# `open_batches` is a stack, so batches can be nested (follow-up 2).
# Complexity: Time set_property O(1), end_batch O(keys in the batch), undo and redo O(keys in the batch), S
pace O(total changes kept)
class Layer:
    def __init__(self, history=None):
        self.props = {}
        self.undo_stack = deque(maxlen=history)       # committed batches, newest last
        self.redo_stack = []
        self.open_batches = []                        # each open batch: key -> Change
```

```
    def begin_batch(self):
        self.open_batches.append({})

    def set_property(self, key, value):
        if len(self.open_batches) == 0:               # outside a batch: it is a batch of one change
            self.begin_batch()
            self.set_property(key, value)
            self.end_batch()
            return
        batch = self.open_batches[-1]
        if key in batch:
            batch[key].new = value                    # touched again: keep the ORIGINAL old value
        else:
            batch[key] = Change(key, self.props.get(key), key in self.props, value)
        self.props[key] = value

    def end_batch(self):
        batch = self.open_batches.pop()
        if len(self.open_batches) > 0:
            merge_batch(self.open_batches[-1], batch)     # a nested batch folds into its parent
        elif len(batch) > 0:
            self.undo_stack.append(list(batch.values()))  # only the OUTERMOST batch is one undoable action
            self.redo_stack.clear()                       # a new action clears the redo history

    def undo(self):
        if len(self.undo_stack) == 0:
            return False
        changes = self.undo_stack.pop()
        for change in changes:
            if change.had_old:
                self.props[change.key] = change.old
            else:
                del self.props[change.key]                # the key did not exist before: remove it
        self.redo_stack.append(changes)
        return True

    def redo(self):
        if len(self.redo_stack) == 0:
            return False
        changes = self.redo_stack.pop()
        for change in changes:
            self.props[change.key] = change.new
        self.undo_stack.append(changes)
        return True


# ---- main question: tests ----
def test_main_question():
    layer = Layer()
    layer.begin_batch()
    layer.set_property("color", "red")
    layer.set_property("color", "blue")
    layer.set_property("size", "4")
    layer.end_batch()
    assert layer.props == {"color": "blue", "size": "4"}
    assert len(layer.undo_stack[0]) == 2              # each key is stored once, however many edits
    layer.undo()
    assert layer.props == {}
    layer.redo()
    assert layer.props == {"color": "blue", "size": "4"}
    layer.begin_batch()
    layer.set_property("color", "green")
    layer.end_batch()
```

```
    layer.undo()
    assert layer.props["color"] == "blue"             # the old value comes back, not the first value of th
e batch
    layer.set_property("x", "1")
    assert not layer.redo()                           # a new batch clears the redo history


# ============================================================
# FOLLOW-UP 1: Limit the history to the last N batches
# ============================================================


# ============================================================
# FOLLOW-UP 4: Limit the history to the last N batches
# ============================================================
# ---- follow-up 1: tests ----
# Answer: keep the undo stack in a deque with maxlen N so the oldest
# batch is dropped automatically (the `history` parameter). Redo is not
# affected.
def test_follow_up_1():
    small = Layer(history=2)
    for i in range(4):
        small.set_property("k", str(i))
    assert len(small.undo_stack) == 2
    small.undo()
    small.undo()
    assert not small.undo() and small.props["k"] == "1"


# ============================================================
# FOLLOW-UP 2: Nested batches
# ============================================================
# ---- follow-up 2: tests ----
```

```
# Answer: keep a STACK of open batches and merge a child batch into its
# parent when it ends (keeping the parent's old value if the key was
# already touched). Only the outermost end_batch commits an undoable action.
def test_follow_up_2():
    layer = Layer()
    layer.begin_batch()
    layer.begin_batch()
    layer.set_property("y", "2")
    layer.end_batch()
    layer.set_property("z", "3")
    layer.end_batch()
    layer.undo()                                      # one undo removes both properties
    assert "y" not in layer.props
    assert "z" not in layer.props


# ============================================================
# FOLLOW-UP 3: Undo of individual setProperty calls, not whole batches
# ============================================================
# Answer: make every set_property its own batch (call begin_batch and
# end_batch around each one), or record one Change per call instead of
# one per key.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[list[str]]) -> list[dict[str, str]]:
SAMPLE INPUT
[('begin',), ('set', 'color', 'red'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snaps
hot',)]
```

Test cases

Input	Expected
`[('begin',), ('set', 'color', 'red'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]`	`[{'color': 'red'}, {}, {'color': 'red'}]`
`[('begin',), ('set', 'a', '1'), ('end',), ('begin',), ('set', 'a', '2'), ('set', 'a', '3'), ('set', 'b', '9'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]`	`[{'a': '3', 'b': '9'}, {'a': '1'}, {'a': '3', 'b': '9'}]`

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 12 OF 22

Rolling state: the next state comes only from the previous one

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

2
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Rolling state: the next state comes only from the previous one | 2 questions | tap a title to jump to it

READ FIRST: the code template for these 2 questions (tap here) >

E 22. Climbing Stairs — **Exponent no date**

E 23. Fibonacci Numbers — **Exponent no date**

CODE TEMPLATE | GROUP 12 OF 22 | 2 QUESTIONS, ONE PATTERN

Rolling state: the next state comes only from the previous one

The one idea these questions share

You never need the whole table, only the previous state: one number, a pair, or one row. Write the state, then one line that builds the new state from the old one, and loop.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def roll(items):
    take, skip = 0, 0                          # CHANGE 1: the state
    for x in items:
        take, skip = skip + x, max(take, skip)     # CHANGE 2: new state from the old state
    return max(take, skip)
```

2. Helper functions that usually get written next to it

```
def two_smallest(row):                    # the trick behind "best of the others"
    order = sorted(range(len(row)), key=lambda j: row[j])
    return order[0], order[1] if len(order) > 1 else order[0]


def fib_pair(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
22	Climbing Stairs	state = the last two counts: a, b = b, a + b, repeated n - 1 times
23	Fibonacci Numbers	the same pair update starting from (0, 1)

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 22 of 38 | topic 12 of 22: Rolling state: the next state comes only from the previous one | 1 of 2

Template fit: state = the last two counts: a, b = b, a + b, repeated n - 1 times | no date

EASY | IN APP LC 70 | Exponent | 22 of 38

★ TOP 10 EASY: rank 6 among Easy problems

★ Climbing Stairs

ASKED (ANY CO.) ~4 MONTHS AGO | LAST 6 MONTHS

STUDY: TYPE **ALL TIME 62.9** **30 DAYS 51.4** **PRIORITY 9/288**

ABOUT THIS QUESTION

Topic: Rolling state: the next state comes only from the previous one

Optimal answer from: your app's interview approach (LeetCode #70)

LeetCode match: LC 70 Climbing Stairs

Last asked (any company): 4 months ago, about 2026-06 (Exponent shows only a rough age) | +38 interviews

Original: https://www.aced.io/questions/3255/climbing-stairs

QUESTION

Given an integer n, representing the number of steps in a staircase, write a function climbStairs that returns the number of distinct ways to climb to the top of the staircase. Each time you can either climb 1 step or 2 steps. You may assume that n is a non-negative integer. Examples Input: n = 1 Output: 1 Explanation: There is only 1 way to climb a staircase with 1 step.

Input: n = 2 Output: 2 Explanation: There are 2 ways to climb a staircase with 2 steps:

1. Climb 1 step + 1 step 2. Climb 2 steps directly Input: n = 3 Output: 3 Explanation: There are 3 ways to climb a staircase with 3 steps:

1. Climb 1 step + 1 step + 1 step 2. Climb 1 step + 2 steps 3. Climb 2 steps + 1 step Input: n = 4 Output: 5 Explanation: There are 5 ways to climb a staircase with 4 steps:

1. Climb 1 step + 1 step + 1 step + 1 step 2. Climb 1 step + 1 step + 2 steps 3. Climb 1 step + 2 steps + 1 step 4. Climb 2 steps + 1 step + 1 step 5. Climb 2 steps + 2 stepsChallenge 1Can you come up with a solution that has a O(n) time complexity?Challenge 2Can you come up with a solution that has a O(1) space complexity?Solution 1: Dynamic Programming Approach In this solution, we use dynamic programming to compute the number of distinct ways to climb n stairs. The idea is to build up the solution incrementally by solving subproblems. We initialize a list ways where ways[i] represents the number of ways to reach the ith step. We start by setting the base cases: ways[0] = 1 (1 way to stay at the ground) ways[1] =

EXAMPLES

Example 1	
Input	**2**
Output	**2**
Why	**Two ways to climb 2 steps: 1+1 or 2.**

Example 2	
Input	**3**
Output	**3**
Why	**Three ways to climb 3 steps: 1+1+1, 1+2, 2+1.**

QUESTION

Example 3	
Input	**5**
Output	**8**
Why	`Eight ways for 5 steps.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def ways(n):
    if n <= 1:
        return 1
    return ways(n - 1) + ways(n - 2)
```

TIME	SPACE
O(2^n)	**O(n)**

WHY IT IS SLOW

The same values are computed over and over: O(2^n). Keeping the last two values in a loop is O(n) time and O(1) space (and a matrix power gives O(log n)).

OPTIMAL SOLUTION · Two running values

TIME	SPACE
O(n)	**O(1)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 n is huge (10^18).

Use matrix exponentiation of [[1, 1], [1, 0]] (or fast doubling) for O(log n) multiplications; take the result modulo a prime if asked.

FOLLOW-UP 2 You can take 1 to k steps (or a given set of step sizes).

ways[i] = sum of ways[i - s] over allowed step sizes s; for 1..k keep a sliding-window sum for O(n).

FOLLOW-UP 3 Some steps are broken (cannot be stood on).

Set ways[i] = 0 for broken steps while filling the table; everything else is unchanged.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Climbing Stairs
#
# Given an integer n, representing the number of steps in
# a staircase, write a function climbStairs that returns
# the number of distinct ways to climb to the top of the
```

```
# staircase. Each time you can either climb 1 step or 2
# steps. You may assume that n is a non-negative integer.
# Examples Input: n = 1 Output: 1 Explanation: There is
# only 1 way to climb a staircase with 1 step. Input: n =
# 2 Output: 2 Explanation: There are 2 ways to climb a
# staircase with 2 steps: 1. Climb 1 step + 1 step 2.
# Climb 2 steps directly Input: n = 3 Output: 3
# Explanation: There are 3 ways to climb a staircase with
# 3 steps: 1. Climb 1 step + 1 step + 1 step 2. Climb 1
# step + 2 steps 3. Climb 2 steps + 1 step Input: n = 4
# Output: 5 Explanation: There are 5 ways to climb a
# staircase with 4 steps: 1. Climb 1 step + 1 step + 1
# step + 1 step 2. Climb 1 step + 1 step + 2 steps 3.
# Climb 1 step + 2 steps + 1 step 4.
#
# FOLLOW-UPS (answered below, in this order):
# 1. n is huge (10^18).
# 2. You can take 1 to k steps (or a given set of step
# sizes).
# 3. Some steps are broken (cannot be stood on).
#
# INTERVIEW NOTES
# The last move was either a 1-step from n - 1 or a 2-step
# from n - 2, so ways(n) = ways(n - 1) + ways(n - 2): the
# Fibonacci recurrence. Only the previous two values
# matter, so two variables replace the whole table.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: main function ----
# Idea: the ways to reach step n are the ways to reach step n - 1 (then
# one step) plus the ways to reach step n - 2 (then two steps). Keep just
```

```
# the last two numbers: (a, b) becomes (b, a + b). Start with
# ways(0) = ways(1) = 1. O(n) time, O(1) space, no table, no recursion.
# Complexity: Time O(n), Space O(1)
def climb(n):
    a = 1
    b = 1
    for i in range(n - 1):
        a, b = b, a + b
    if n >= 1:
        return b
    return 1


# ---- main question: tests ----
def test_main_question():
    answers = []
    for n in range(1, 6):
        answers.append(climb(n))
    assert answers == [1, 2, 3, 5, 8]
    assert climb(0) == 1


# ======================================================
# FOLLOW-UP 1: n is huge (10^18)
```

```python
# ============================================================
# ---- follow-up 1: helper functions ----
def mat_mult(x, y, mod):
    # multiply two 2 x 2 matrices, keeping every number below `mod`
    out = [[0, 0], [0, 0]]
    for i in range(2):
        for j in range(2):
            total = 0
            for k in range(2):
                total += x[i][k] * y[k][j]
            out[i][j] = total % mod
    return out


# ---- follow-up 1: function ----
# Answer: raise the matrix [[1, 1], [1, 0]] to the n-th power by
# repeated squaring: O(log n) multiplications (fast doubling is the same
# idea). Take every product modulo a prime if the answer is huge.
# Complexity: Time O(log n) (matrix power by squaring), Space O(1)
def climb_huge(n, mod=10 ** 9 + 7):
    result = [[1, 0], [0, 1]]                           # the identity matrix
    base = [[1, 1], [1, 0]]
    power = n
    while power > 0:
        if power % 2 == 1:
            result = mat_mult(result, base, mod)
        base = mat_mult(base, base, mod)
        power = power // 2
    return result[0][0] % mod                           # result = [[F(n+1), F(n)], [F(n), F(n-1)]]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert climb_huge(5) == 8 and climb_huge(1) == 1
    assert 0 <= climb_huge(10 ** 18) < 10 ** 9 + 7
    assert climb_huge(50) == climb(50) % (10 ** 9 + 7)


# ============================================================
# FOLLOW-UP 2: You can take 1 to k steps (or a given set of step sizes)
# ============================================================
# ---- follow-up 2: function ----
# Answer: ways[i] = the sum of ways[i - s] over every allowed step size s.
# For steps 1..k keep a sliding-window sum instead, which makes it O(n).
# Complexity: Time O(n * number of step sizes), Space O(n)
def climb_steps(n, steps):
    ways = [1] + [0] * n
    for i in range(1, n + 1):
        for s in steps:
            if i >= s:
                ways[i] += ways[i - s]
    return ways[n]


# Complexity: Time O(n), Space O(n)
def climb_up_to_k(n, k):
    ways = [1] + [0] * n
    window = 1                                          # the sum of the last k values of ways
    for i in range(1, n + 1):
        ways[i] = window
        window += ways[i]
        if i - k >= 0:
            window -= ways[i - k]
    return ways[n]
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert climb_steps(5, (1, 2)) == 8
    assert climb_steps(4, (1, 3)) == 3
    assert climb_up_to_k(5, 2) == 8
    assert climb_up_to_k(4, 3) == 7


# ============================================================
# FOLLOW-UP 3: Some steps are broken (cannot be stood on)
# ============================================================
# ---- follow-up 3: function ----
# Answer: set ways[i] = 0 for a broken step while filling the table;
# everything else stays the same.
# Complexity: Time O(n + number of broken steps), Space O(n)
def climb_broken(n, broken):
    bad = set(broken)
    ways = [1] + [0] * n
    for i in range(1, n + 1):
        if i in bad:
            continue
        two_back = 0
        if i > 1:
            two_back = ways[i - 2]
        ways[i] = ways[i - 1] + two_back
    return ways[n]


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert climb_broken(5, [3]) == 2
    assert climb_broken(3, [1, 2]) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given an integer n, representing the number of steps in a staircase, write a function climbStairs that returns the number of distinct ways to climb to the top of the staircase. Each time you can either climb 1 step or 2 steps. You may assume that n is a non-negative integer. Examples

OFFICIAL PAGE TEXT

```
Input: n = 1
Output: 1
Explanation: There is only 1 way to climb a staircase with 1 step.

Input: n = 2
Output: 2
Explanation: There are 2 ways to climb a staircase with 2 steps:
1. Climb 1 step + 1 step
2. Climb 2 steps directly

Input: n = 3
Output: 3
Explanation: There are 3 ways to climb a staircase with 3 steps:
1. Climb 1 step + 1 step + 1 step
2. Climb 1 step + 2 steps
3. Climb 2 steps + 1 step

Input: n = 4
Output: 5
Explanation: There are 5 ways to climb a staircase with 4 steps:
1. Climb 1 step + 1 step + 1 step + 1 step
2. Climb 1 step + 1 step + 2 steps
3. Climb 1 step + 2 steps + 1 step
4. Climb 2 steps + 1 step + 1 step
5. Climb 2 steps + 2 steps
```

Challenge 1Can you come up with a solution that has a O(n) time complexity?Challenge 2Can you come up with a solution that has a O(1) space complexity?Solution 1: Dynamic Programming Approach In this solution, we use dynamic programming to compute the number of distinct ways to climb n stairs. The idea is to build up the solution incrementally by solving subproblems. We initialize a list ways where ways[i] represents the number of ways to reach the ith step. We start by setting the base cases: ways[0] = 1 (1 way to stay at the ground) ways[1] = 1 (1 way to reach the first step, which is a single 1-step) For each subsequent step i, the number of ways to reach that step is the sum of the ways to reach the previous step (ways[i - 1]) and the step before that (ways[i - 2]). This is because you can get to step i either from step i - 1 with a 1-step or from step i - 2 with a 2-step. Time Complexity: The time complexity of this solution is O(n), where n is the number of steps. This is because we iterate through the steps from 2 to n, updating the number of ways for each step. Space Complexity: The space complexity is O(n) due to the additional array ways used to store the number of ways for each step. If space optimization is needed, this can be reduced to O(1) by keeping only the last two computed values. Solution 2: Combinatorial Approach Our solution leverages combinatorial mathematics to calculate the number of distinct ways to climb n stairs. The approach is based on counting the number of valid sequences of steps (1-step and 2-steps) that sum up to n. We use the binomial coefficient to count the number of ways to arrange a certain number of 1-steps and 2-steps. For each possible number of 2-steps (k), we compute the number of ways to choose k positions from n - k steps (the remaining steps will be 1-steps). This is done using the formula for combinations: C(n,k)C(n, k)C(n,k), where C(n,k)=n!k!*(n-k)!C(n, k) = \frac{n!}{k! \cdot (n-k)!}C(n,k)=k!*(n-k)!n!. The total number of ways is the sum of these combinations for all valid values of k. Time Complexity: The time complexity of this solution is O(n), where n is the number of steps. This is because we compute the combination for each k from 0 to n // 2, and each combination computation is efficient. Space Complexity: The space complexity is O(1) as we only use a few additional variables for storing intermediate results, regardless of the input size.No answers contributed by the community yet.

STUDY PATH: question 23 of 38 | topic 12 of 22: Rolling state: the next state comes only from the previous one | 2 of 2

Template fit: the same pair update starting from (0, 1) | no date

EASY | IN APP LC 509 | Exponent | 23 of 38

Fibonacci Numbers

ASKED (ANY CO.) ~9 MONTHS AGO | OLDER

STUDY: TYPE | ALL TIME 46.3 | 30 DAYS 0 | PRIORITY 89/288

ABOUT THIS QUESTION

Topic: Rolling state: the next state comes only from the previous one

Optimal answer from: your app's interview approach (LeetCode #509)

LeetCode match: LC 509 Fibonacci Number

Last asked (any company): 9 months ago, about 2026-01 (Exponent shows only a rough age) | +26 interviews

Original: https://www.aced.io/questions/3631/fibonacci

QUESTION

Write a function fib(n) that returns the nth Fibonacci number. The Fibonacci sequence is defined as F(n) = F(n-1) + F(n-2) with F(0) = 1 and F(1) = 1, which produces the following pattern: 1, 1, 2, 3, 5, 8, 13 ... Examples Pythonfib(0)

=> 1 fib(1)

=> 1 fib(2)

=> 2 fib(3)

=> 3 ... fib(10)

=> 89 fib(20)

=> 10946Hint 1The Fibonacci sequence

is defined recursively since it refers to itself.Hint 2We can start with a recursive implementation and optimize it.Hint 3Each call to F(n) has two subproblems: F(n-1) and F(n-2). Can we memoize these calculations?Challenge 1Does your solution work in O(n) time?This question is a classic example of how dynamic programming, or memoization, can help us solve problems faster by reducing repetitive computations. In this first version, we recursively call the fib function twice, and then we call it twice more in every subsequent recursive call, so the total number of operations expands to O(2^n)! To improve the runtime of our program, let's save our prior calculations in a dictionary called memo and pass it into our function. Now, if we've already found fib(n) before, our function will use this stored value instead of recursing again. Time Complexity: The time complexity of the memoized version is O(n). This is because each Fibonacci number from 0 to n is computed only once and then stored in the memo dictionary. The recursive calls will now simply look up the value in the dictionary if it has already been computed, reducing redundant

EXAMPLES

Example 1	
Input	`0`
Output	`1`
Why	`F(0) = 1 and F(1) = 1, as defined in this question.`

Example 2	
Input	`5`
Output	`8`
Why	`1, 1, 2, 3, 5, 8: F(5) = 8.`

QUESTION

Example 3	
Input	10
Output	89
Why	F(10) = 89.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def fib(n):
    if n < 2:
        return 1  # fib(0) = fib(1) = 1
    return fib(n - 1) + fib(n - 2)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

The call tree has about 1.6^n nodes because the same values are recomputed. A loop with two variables is O(n) time and O(1) space; fast doubling is O(log n).

OPTIMAL SOLUTION · Iteration with two variables

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 n is up to 10^18 and the answer is needed modulo a prime.

Fast doubling (or matrix power) with every multiplication reduced modulo the prime: O(log n) steps with small numbers.

FOLLOW-UP 2 Why is the recursive version slow, and how do you fix it without changing the structure?

It recomputes the same values exponentially often (the call tree has about phi^n nodes). Memoisation (lru_cache) makes each value computed once, so O(n) time with O(n) memory; bottom-up with two variables drops the memory to O(1).

FOLLOW-UP 3 Closed form (Binet's formula).

F(n) = round(phi^n / sqrt(5)) is exact only while floating point has enough precision (up to about n = 70); beyond that it gives wrong digits, so use the integer methods.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Fibonacci Numbers
#
# Write a function fib(n) that returns the nth Fibonacci
# number. The Fibonacci sequence is defined as F(n) =
```

```
# F(n-1) + F(n-2) with F(0) = 1 and F(1) = 1, which
# produces the following pattern: 1, 1, 2, 3, 5, 8, 13 ...
# Examples Pythonfib(0) # => 1 fib(1) # => 1 fib(2) # => 2
# fib(3) # => 3 ... fib(10) # => 89 fib(20) # => 10946Hint
# 1The Fibonacci sequence is defined recursively since it
# refers to itself.Hint 2We can start with a recursive
# implementation and optimize it.Hint 3Each call to F(n)
# has two subproblems: F(n-1) and F(n-2). Can we memoize
# these calculations?Challenge 1Does your solution work in
# O(n) time?This question is a classic example of how
# dynamic programming, or memoization, can help us solve
# problems faster by reducing repetitive computations.
#
# FOLLOW-UPS (answered below, in this order):
# 1. n is up to 10^18 and the answer is needed modulo a
# prime.
# 2. Why is the recursive version slow, and how do you fix
# it without changing the structure?
# 3. Closed form (Binet's formula).
#
# INTERVIEW NOTES
# Each term needs only the previous two, so a loop with
# two variables is O(n) time and O(1) space; the naive
# recursion does exponential repeated work. Since the
# recurrence is linear it can also be turned into a matrix
# power, or into the fast doubling identities, for O(log
# n).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main functions ----
# This question numbers the sequence 1, 1, 2, 3, 5, 8 ...: fib(0) = fib(1) = 1.
# The definition (slow: about 1.6 ** n calls; say it first):
```

```
# Complexity: Time O(2^n) (the same values are computed again and again), Space O(n) for the recursion
def fib_slow(n):
    if n < 2:
        return 1
    return fib_slow(n - 1) + fib_slow(n - 2)


# Idea: keep only the previous two values and move forward n times.
# O(n) time and O(1) memory (memoising the recursion gives the same time
# with O(n) memory).
# Complexity: Time O(n), Space O(1)
def fib(n):
    a = 1
    b = 1
    for i in range(n):
        a, b = b, a + b
    return a


# ---- main question: tests ----
def test_main_question():
    first = []
    for n in range(7):
```

```
        first.append(fib(n))
    assert first == [1, 1, 2, 3, 5, 8, 13]
    assert fib(10) == 89 and fib(20) == 10946
    assert fib_slow(10) == 89


# =====================================================
# FOLLOW-UP 1: n is up to 10^18 and the answer is needed modulo a prime
# =====================================================
# ---- follow-up 1: helper functions ----
MOD = 10 ** 9 + 7


def pair(n, mod):
    # fast doubling. Returns (F(n), F(n + 1)) of the standard sequence 0, 1, 1, 2, 3 ...
    # using F(2k) = F(k) * (2 * F(k + 1) - F(k)) and F(2k + 1) = F(k)^2 + F(k + 1)^2.
    if n == 0:
        return 0, 1
    a, b = pair(n // 2, mod)                # F(k) and F(k + 1) for k = n // 2
    c = a * (2 * b - a) % mod               # F(2k)
    d = (a * a + b * b) % mod               # F(2k + 1)
    if n % 2 == 1:
        return d, (c + d) % mod
    return c, d


# ---- follow-up 1: function ----
# Answer: fast doubling (or matrix power) with every multiplication
# reduced modulo the prime: O(log n) steps with small numbers. Our
# fib(n) is the standard F(n + 1).
# Complexity: Time O(log n) (fast doubling), Space O(log n) for the recursion
def fib_mod(n, mod=MOD):
    return pair(n + 1, mod)[0] % mod


# ---- follow-up 1: tests ----
def test_follow_up_1():
    for n in range(200):
        assert fib_mod(n, MOD) == fib(n) % MOD
        big = 10 ** 18 + 9
        assert fib_mod(n, big) == fib(n) % big
    assert 0 <= fib_mod(10 ** 18) < MOD                    # a huge n finishes at once


# =====================================================
# FOLLOW-UP 2: Why is the recursive version slow, and how do you fix it without changing the structure?
# =====================================================
# ---- follow-up 2: function ----
# Answer: it solves the same smaller problems again and again (the call
# tree has about 1.6 ** n nodes). Remember each answer in a dictionary
# (memoisation): every value is computed once, so O(n) time and O(n)
# memory. The loop with two variables brings the memory down to O(1).
# Complexity: Time O(n), Space O(n)
def fib_memo(n, memo=None):
    if memo is None:
        memo = {}
    if n < 2:
        return 1
    if n not in memo:
        memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]


# ---- follow-up 2: tests ----
```

```
def test_follow_up_2():
    assert fib_memo(20) == 10946
    assert fib(0) == 1 and fib(1) == 1
```

```
    assert fib_memo(90) == fib(90)


# ======================================================
# FOLLOW-UP 3: Closed form (Binet's formula)
# ======================================================
# ---- follow-up 3: function ----
# Answer: F(n) = round(phi^n / sqrt(5)) with phi = (1 + sqrt(5)) / 2 is
# exact only while floating point has enough precision (up to about n =
# 70); after that the last digits are wrong, so use the integer methods.
# Complexity: Time O(1), Space O(1) (floating point: only exact for small n)
def fib_binet(n):
    root5 = 5 ** 0.5
    phi = (1 + root5) / 2
    return round(phi ** (n + 1) / root5)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    for n in range(60):
        assert fib_binet(n) == fib(n)                    # exact for a while ...
    wrong = []
    for n in range(60, 200):
        if fib_binet(n) != fib(n):
            wrong.append(n)
    assert wrong and wrong[0] < 100                      # ... then floating point fails


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Write a function fib(n) that returns the nth Fibonacci number. The Fibonacci sequence is defined as F(n) = F(n-1) + F(n-2) with F(0) = 1 and F(1) = 1, which produces the following pattern:

```
1, 1, 2, 3, 5, 8, 13 ...
```

Examples

```
Pythonfib(0)  # => 1
fib(1)  # => 1
fib(2)  # => 2
fib(3)  # => 3
...
fib(10) # => 89
fib(20) # => 10946
```

OFFICIAL PAGE TEXT

Hint 1The Fibonacci sequence is defined recursively since it refers to itself.Hint 2We can start with a recursive implementation and optimize it.Hint 3Each call to F(n) has two subproblems: F(n-1) and F(n-2). Can we memoize these calculations?Challenge 1Does your solution work in O(n) time?This question is a classic example of how dynamic programming, or memoization, can help us solve problems faster by reducing repetitive computations. In this first version, we recursively call the fib function twice, and then we call it twice more in every subsequent recursive call, so the total number of operations expands to O(2^n)! To improve the runtime of our program, let's save our prior calculations in a dictionary called memo and pass it into our function. Now, if we've already found fib(n) before, our function will use this stored value instead of recursing again. Time Complexity: The time complexity of the memoized version is O(n). This is because each Fibonacci number from 0 to n is computed only once and then stored in the memo dictionary. The recursive calls will now simply look up the value in the dictionary if it has already been computed, reducing redundant calculations. Space Complexity: The memo dictionary stores each Fibonacci number from 0 to n, which requires O(n) space.No answers contributed by the community yet.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 13 OF 22

Two pointers: move i and j toward each other (or along two lists)

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Two pointers: move i and j toward each other (or along two lists) | 4 questions | tap a title to jump to it

READ FIRST: the code template for these 4 questions (tap here) >

E 24. Reverse a linked list. **Exponent no date**

E 25. Sort a binary array in linear time. **Exponent no date**

E 26. Is this a valid palindrome? **Exponent no date**

E 27. Reverse a Sentence **Exponent no date**

CODE TEMPLATE | GROUP 13 OF 22 | 4 QUESTIONS, ONE PATTERN

Two pointers: move i and j toward each other (or along two lists)

The one idea these questions share

Put one pointer at each end (or one at the start of each of two sorted lists). Look at the two items, decide which pointer moves, and stop when they meet. Most of these are O(n) because each pointer only moves one way.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def has_pair(nums, target):                    # nums sorted
    i, j = 0, len(nums) - 1
    while i < j:
        s = nums[i] + nums[j]                  # CHANGE 1: what you compute from the two items
        if s == target:
            return True
        if s < target:                         # CHANGE 2: which pointer moves
            i += 1
        else:
            j -= 1
    return False
```

2. Helper functions that usually get written next to it

```
def reverse_range(a, i, j):
    while i < j:
        a[i], a[j] = a[j], a[i]
        i, j = i + 1, j - 1


def walk_two(a, b):                            # merge order of two sorted lists
    i = j = 0
    while i < len(a) or j < len(b):
        if j == len(b) or (i < len(a) and a[i] <= b[j]):
            yield a[i]; i += 1
        else:
            yield b[j]; j += 1
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
24	Reverse a linked list.	three pointers prev, current, next: point current.next back at prev, then move all three one step
25	Sort a binary array in linear time.	left skips 0s, right skips 1s; when left sits on a 1 and right on a 0, swap them
26	Is this a valid palindrome?	compare the two ends; skip characters that are not letters or digits; stop when the pointers cross
27	Reverse a Sentence	swap from both ends to reverse the whole array, then reverse every word the same way

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 24 of 38 | topic 13 of 22: Two pointers: move i and j toward each other (or along two lists) | 1 of 4

Template fit: three pointers prev, current, next: point current.next back at prev, then move all three one step | no date

EASY | IN APP LC 206 | Exponent | 24 of 38

Reverse a linked list.

ASKED (ANY CO.) ~4 MONTHS AGO | LAST 6 MONTHS

STUDY: TYPE **ALL TIME 59.8** **30 DAYS 0** **PRIORITY 43/288**

ABOUT THIS QUESTION

Topic: Two pointers: move i and j toward each other (or along two lists)

Optimal answer from: your app's interview approach (LeetCode #206)

LeetCode match: LC 206 Reverse Linked List

Last asked (any company): 4 months ago, about 2026-06 (Exponent shows only a rough age) | +921 interviews

Original: https://www.aced.io/questions/3247/reverse-linked-list

QUESTION

Given the head of a singly linked list, reverse the list and return its head. Implement a function reverseList that takes the head of the linked list as input and returns the new head of the reversed list. Refer to the ListNode class/struct and do not comment it out. Example Input: 1 -> 2 -> 3 -> 4 Output: 4 -> 3 -> 2 -> 1Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(1)?

EXAMPLES

Example 1	
Input	`head = 1 -> 2 -> 3`
Output	`vals(reverse_list(head)) -> [3, 2, 1]`
Why	`1 -> 2 -> 3 becomes 3 -> 2 -> 1.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def reverse_list(head):
    values = []
    node = head
    while node:
        values.append(node.val)
        node = node.next
    new_head = None
    for v in values:
        node = ListNode(v)
        node.next = new_head
        new_head = node  # push to front
    return new_head
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Time is O(n), but it allocates a whole new list, O(n) extra space. Flipping each node's next pointer in place with three variables needs O(1) space.

OPTIMAL SOLUTION · Flip the pointers in one pass

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it recursively.

Reverse the rest first, then make the next node point back at this one and cut this node's own pointer. An inner helper that carries prev (as coded) keeps it tail-recursive in spirit; the call stack is O(n), so mention the depth limit.

FOLLOW-UP 2 Reverse only positions left to right (LeetCode 92).

Walk to the node before the segment, reverse right - left + 1 nodes with the same loop, then reconnect: the old first node of the segment must point to what followed the segment, and the node before it points to the new first.

FOLLOW-UP 3 Reverse in groups of k and leave a short last group alone (LeetCode 25).

Check that k nodes exist before reversing a group, reverse them with the same loop, reconnect both ends, and move the group pointer to the old first node of the group.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Reverse a linked list.
#
# Given the head of a singly linked list, reverse the list
# and return its head. Implement a function reverseList
# that takes the head of the linked list as input and
# returns the new head of the reversed list. Refer to the
# ListNode class/struct and do not comment it out. Example
# Input: 1 -> 2 -> 3 -> 4 Output: 4 -> 3 -> 2 ->
# 1Challenge 1Can you come up with a solution with a time
# complexity of O(n)?Challenge 2Can you come up with a
# solution with a space complexity of O(1)?
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it recursively.
# 2. Reverse only positions left to right (LeetCode 92).
# 3. Reverse in groups of k and leave a short last group
# alone (LeetCode 25).
#
# INTERVIEW NOTES
# Reversing means every next pointer has to flip, so hold
# three things: the node you are on, the one before it
# (what it will point to) and the one after it (so you do
# not lose the rest). Always save next before overwriting
# it. For 'reverse part of the list' remember the node
# before the part and the node that will end up last in
# it.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
```

```
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


def make_list(values):
    # [1, 2, 3] -> 1 -> 2 -> 3
    dummy = ListNode()
    tail = dummy
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next


def to_list(node):
    values = []
    while node:
        values.append(node.val)
        node = node.next
    return values


# ---- main question: main function ----
# Idea: walk with `prev` and `node`. Remember node.next FIRST (the step
# people forget), point node back at prev, then move both forward. When
# node is None, prev is the new head.
# Complexity: Time O(n), Space O(1)
def reverse_list(head):
    prev = None
    node = head
    while node:
        following = node.next
        node.next = prev
        prev = node
        node = following
    return prev


# ---- main question: tests ----
def test_main_question():
    assert to_list(reverse_list(make_list([1, 2, 3, 4]))) == [4, 3, 2, 1]
    assert reverse_list(None) is None
    assert to_list(reverse_list(make_list([7]))) == [7]


# ============================================================
# FOLLOW-UP 1: Do it recursively
# ============================================================
# ---- follow-up 1: function ----
# Answer: handle one node, then hand the rest to the same function: save
# the next node, point this one back at `prev`, and recurse with this
# node as the new prev. The call stack is O(n): mention the depth limit.
# Complexity: Time O(n), Space O(n) for the recursion
def reverse_recursive(head, prev=None):
    if head is None:
```

```
        return prev
    following = head.next
    head.next = prev
    return reverse_recursive(following, head)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert to_list(reverse_recursive(make_list([1, 2, 3]))) == [3, 2, 1]


# ========================================================
# FOLLOW-UP 2: Reverse only positions left to right (LeetCode 92)
# ========================================================
# ---- follow-up 2: function ----
# Answer: walk to the node before the segment, reverse right - left + 1
# nodes with the same loop, then reconnect: the old first node of the
# segment must point to what came after the segment, and the node before
# the segment points to the new first node.
# Complexity: Time O(n), Space O(1)
def reverse_between(head, left, right):
    dummy = ListNode(0, head)
    before = dummy
    for i in range(left - 1):
        before = before.next
    prev = None
    node = before.next
    for i in range(right - left + 1):
        following = node.next
        node.next = prev
        prev = node
        node = following
    before.next.next = node                          # the old first node points past the segment
    before.next = prev
    return dummy.next


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert to_list(reverse_between(make_list([1, 2, 3, 4, 5]), 2, 4)) == [1, 4, 3, 2, 5]
```

```
    assert to_list(reverse_between(make_list([1, 2]), 1, 2)) == [2, 1]


# ========================================================
# FOLLOW-UP 3: Reverse in groups of k and leave a short last group alone (LeetCode 25)
# ========================================================
# ---- follow-up 3: function ----
# Answer: check that k nodes exist before reversing a group, reverse them
# with the same loop, reconnect both ends, and move on to the old first
# node of the group (now its last).
# Complexity: Time O(n), Space O(1)
def reverse_k_group(head, k):
    dummy = ListNode(0, head)
    group_prev = dummy
    while True:
        end = group_prev                             # is there a full group of k nodes?
        for i in range(k):
            end = end.next
            if end is None:
                return dummy.next
        first = group_prev.next
        after = end.next
        prev = after
        node = first
```

```
        while node is not after:
            following = node.next
            node.next = prev
            prev = node
            node = following
        group_prev.next = prev
        group_prev = first


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert to_list(reverse_k_group(make_list([1, 2, 3, 4, 5]), 2)) == [2, 1, 4, 3, 5]
    assert to_list(reverse_k_group(make_list([1, 2, 3]), 3)) == [3, 2, 1]
    assert to_list(reverse_k_group(make_list([1]), 2)) == [1]


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given the head of a singly linked list, reverse the list and return its head. Implement a function reverseList that takes the head of the linked list as input and returns the new head of the reversed list. Refer to the ListNode class/struct and do not comment it out. Example

```
Input: 1 -> 2 -> 3 -> 4
Output: 4 -> 3 -> 2 -> 1
```

Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(1)?Our solution reverses the linked list by iterating through it while maintaining pointers to keep track of the previous node and the current node. For each node, it saves the next node, then reverses the link by pointing the current node's next to the previous node. It then moves the prev pointer to the current node and advances the current pointer to the saved next node. This process continues until the end of the list is reached. At the end, the prev pointer points to the new head of the reversed list, which is returned. The solution uses two pointers, prev and current, to achieve this reversal. This approach ensures that each node is processed exactly once, making it both efficient and straightforward. Time Complexity: The solution has a time complexity of O(n), where n is the number of nodes in the linked list. This is because we iterate through the list once, performing constant-time operations (pointer adjustments) for each node. Space Complexity: The solution uses O(1) space, as it only uses a fixed amount of extra space (three pointers: prev, current, and nextNode). No additional data structures are used, making it very space-efficient. NoteCheck out our lesson on the two pointer approach to learn more about what it is and how to use it!No answers contributed by the community yet.

STUDY PATH: question 25 of 38 | topic 13 of 22: Two pointers: move i and j toward each other (or along two lists) | 2 of 4

Template fit: left skips 0s, right skips 1s; when left sits on a 1 and right on a 0, swap them | no date

EASY | Exponent | 25 of 38

Sort a binary array in linear time.

ASKED (ANY CO.) ~5 MONTHS AGO | LAST 6 MONTHS

STUDY: TYPE **ALL TIME 53.6** **30 DAYS 0** **PRIORITY 74/288**

ABOUT THIS QUESTION

Topic: Two pointers: move i and j toward each other (or along two lists)

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #75 Sort Colors

LeetCode match: LC 75 Sort Colors (binary version) (inferred from short statement)

Last asked (any company): 5 months ago, about 2026-05 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/971/sort-binary-array-linear-time

QUESTION

Sort an array that contains only 0s and 1s in place, in one pass over the array (O(n) time) and with O(1) extra space.

Function signature

def sort_binary(arr: list) -> list

EXAMPLES

Example 1	
Input	`[1, 0, 1, 0, 0, 1]`
Output	`[0, 0, 0, 1, 1, 1]`
Why	`All the 0s move to the front.`

Example 2	
Input	`[0, 0, 1, 1]`
Output	`[0, 0, 1, 1]`
Why	`Already sorted arrays stay as they are.`

Example 3	
Input	`[1]`
Output	`[1]`
Why	`A single value.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(arr):
    return sorted(arr)  # O(n log n)
```

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

BRUTE FORCE

A comparison sort is O(n log n) and needs a copy; with only two values two pointers (swap a 1 on the left with a 0 on the right) finish in one O(n) pass in place.

OPTIMAL SOLUTION · Two pointers swapping from both ends

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why not just count zeros and fill?

Counting is also O(n) time and O(1) space and is fine for bare numbers. It needs two passes, and it cannot sort records that merely have a binary key, because it loses the objects. The swap version sorts the records themselves.

FOLLOW-UP 2 Count the zeros and rebuild (also O(n)).

Fine for bare numbers: one pass to count, one to fill. It cannot sort records that merely have a 0 or 1 key, because it loses the objects; the swapping version can.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Sort a binary array in linear time.
#
# Sort an array that contains only 0s and 1s in place, in
# one pass over the array (O(n) time) and with O(1) extra
# space.
#
# Signature:
#   def sort_binary(arr: list) -> list
#
# Examples:
# [1, 0, 1, 0, 0, 1]  ->  [0, 0, 0, 1, 1, 1]
# [0, 0, 1, 1]  ->  [0, 0, 1, 1]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why not just count zeros and fill?
# 2. Count the zeros and rebuild (also O(n)).
#
# INTERVIEW NOTES
# With only two values the sorted array is 'all zeros then
# all ones', so you only have to move zeros left and ones
# right. Two pointers from the two ends do it in one pass:
# swap whenever the left pointer sees a 1 and the right
# pointer a 0.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

# ======================================================
# MAIN QUESTION
```

```
# ============================================================
# ---- main question: main function ----
# Idea: two pointers. Zeros collect on the left, ones on the right.
# Move lo past a 0, move hi back past a 1; when lo sits on a 1 and hi on
# a 0, swap them. Every step moves a pointer: O(n), in place.
# Complexity: Time O(n), Space O(1)
def sort_binary(arr):
    lo = 0
    hi = len(arr) - 1
    while lo < hi:
        if arr[lo] == 0:
            lo += 1
        elif arr[hi] == 1:
            hi -= 1
        else:
            arr[lo], arr[hi] = arr[hi], arr[lo]       # a 1 on the left and a 0 on the right
            lo += 1
            hi -= 1
    return arr


# the same idea for records that only have a 0/1 key: swapping keeps the
# objects themselves (counting would lose them). Not stable.
# Complexity: Time O(n), Space O(1)
def sort_records(items, key):
    lo = 0
    hi = len(items) - 1
    while lo < hi:
        if key(items[lo]) == 0:
            lo += 1
        elif key(items[hi]) == 1:
            hi -= 1
        else:
            items[lo], items[hi] = items[hi], items[lo]
            lo += 1
            hi -= 1
    return items


# the same idea for 0, 1 and 2 (Dutch national flag)
# Complexity: Time O(n), Space O(1)
def sort_three(arr):
    low = 0              # arr[:low] are all 0
    mid = 0              # arr[low:mid] are all 1
    high = len(arr)      # arr[high:] are all 2; arr[mid:high] is not looked at yet
    while mid < high:
        if arr[mid] == 0:
            arr[low], arr[mid] = arr[mid], arr[low]
            low += 1
            mid += 1
        elif arr[mid] == 2:
            high -= 1
            arr[mid], arr[high] = arr[high], arr[mid]
        else:
            mid += 1
    return arr


# ---- main question: tests ----
def test_main_question():
    assert sort_binary([1, 0, 1, 0, 1, 0]) == [0, 0, 0, 1, 1, 1]
    assert sort_binary([]) == []
    assert sort_binary([1]) == [1]
    assert sort_binary([1, 1, 0]) == [0, 1, 1]
```

```
    assert sort_three([2, 0, 1, 2, 0, 1]) == [0, 0, 1, 1, 2, 2]
    assert sort_three([2, 2, 0]) == [0, 2, 2]


# ====================================================
# FOLLOW-UP 1: Why not just count zeros and fill?
# ====================================================
# Answer: it works for bare numbers, but it cannot sort RECORDS that only
# have a 0 or 1 key, because rewriting the array from a count loses the
# objects. The swapping version keeps them.


# ====================================================
# FOLLOW-UP 2: Count the zeros and rebuild (also O(n))
# ====================================================
# ---- follow-up 2: function ----
# Answer: one pass to count the zeros, then write that many 0s and the
# rest 1s.
# Complexity: Time O(n), Space O(n)
def sort_by_counting(arr):
    zeros = 0
    for x in arr:
        if x == 0:
            zeros += 1
    return [0] * zeros + [1] * (len(arr) - zeros)


# ---- follow-up 2: tests ----
def second(person):
    return person[1]


def test_follow_up_2():
    for arr in ([0, 0], [1, 1], [1, 0], [0, 1, 0, 0, 1]):
        assert sort_binary(arr[:]) == sorted(arr)
        assert sort_by_counting(arr) == sorted(arr)
    people = [("ann", 1), ("bob", 0), ("cy", 1), ("di", 0)]
    done = sort_records(people[:], second)
```

```
    assert [second(p) for p in done] == [0, 0, 1, 1]
    assert sorted(done) == sorted(people)             # no record was lost


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 26 of 38 | topic 13 of 22: Two pointers: move i and j toward each other (or along two lists) | 3 of 4

Template fit: compare the two ends; skip characters that are not letters or digits; stop when the pointers cross | no date

EASY | IN APP LC 125 | Exponent | 26 of 38

Is this a valid palindrome?

ASKED (ANY CO.) ~6 MONTHS AGO | OLDER

STUDY: TYPE ALL TIME 52.8 30 DAYS 51.4 PRIORITY 26/288

ABOUT THIS QUESTION

Topic: Two pointers: move i and j toward each other (or along two lists)

Optimal answer from: your app's interview approach (LeetCode #125)

LeetCode match: LC 125 Valid Palindrome

Last asked (any company): 6 months ago, about 2026-04 (Exponent shows only a rough age) | +612 interviews

Original: https://www.aced.io/questions/4056/validate-palindrome-check

QUESTION

Given a string s, return true if it is a palindrome, and false otherwise. A string is considered a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Write a function isPalindrome that implements this logic. Examples Input: s = "H123!@321h" Output: true Input: s = "race a car" Output: false Constraints 1 <= s.length <= 2 * 10^5 The input string s can include any printable ASCII characters. Challenge 1Can you come up with a solution that achieves O(n) time complexity?Challenge 2Can you come up with a solution that achieves O(1) space complexity?Solution 1: Filtering and Reversing Approach This solution processes the input string by first filtering out non-alphanumeric characters and converting all remaining characters to lowercase. It then checks if this filtered string reads the same forward and backward. Filtering: Iterate through the string and collect alphanumeric characters while converting them to lowercase. Reversing and Comparing: Create a reversed version of the filtered string and compare it with the original filtered string to check for palindrome properties. Time Complexity: The time complexity is O(n), where n is the length of the input string. This is because filtering the characters and creating the reversed string both require iterating through the input once. Space Complexity: The space complexity is O(n) for storing the filtered characters and their

EXAMPLES

Example 1	
Input	`'A man, a plan, a canal: Panama'`
Output	`true`
Why	`Ignoring punctuation, spaces and case it reads 'amanaplanacanalpanama'.`

Example 2	
Input	`'race a car'`
Output	`false`
Why	`'raceacar' is not a palindrome.`

Example 3	
Input	`' '`
Output	`true`
Why	`Nothing but a space: nothing to compare, so it is a palindrome.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_palindrome(s):
    t = [c.lower() for c in s if c.isalnum()]
    return t == t[::-1]
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but two extra copies of the string, O(n) space. Two pointers moving inward, skipping punctuation, need O(1) space and stop at the first mismatch.

OPTIMAL SOLUTION · Two pointers that skip ignored characters

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 You may delete at most one character (LeetCode 680).

Move inward until the first mismatch, then the answer is whether skipping the left character or skipping the right character leaves a palindrome. That is two more linear checks, so O(n) overall.

FOLLOW-UP 2 Unicode, accents, or locale-specific case.

Normalise first (unicodedata.normalize and casefold) and decide what counts as a letter. str.isalnum handles many scripts, but combining marks need normalisation.

FOLLOW-UP 3 Longest palindromic substring.

Expand around each of the 2n - 1 centers in O(n^2), or use Manacher's algorithm in O(n). The two-pointer comparison is the building block.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Is this a valid palindrome?
#
# Given a string s, return true if it is a palindrome, and
# false otherwise. A string is considered a palindrome if,
# after converting all uppercase letters into lowercase
# letters and removing all non-alphanumeric characters, it
# reads the same forward and backward. Write a function
# isPalindrome that implements this logic. Examples Input:
# s = "H123!@321h" Output: true Input: s = "race a car"
# Output: false Constraints 1 <= s.length <= 2 * 10^5 The
# input string s can include any printable ASCII
# characters. Challenge 1Can you come up with a solution
# that achieves O(n) time complexity?Challenge 2Can you
```

```
# come up with a solution that achieves O(1) space
# complexity?Solution 1: Filtering and Reversing Approach
# This solution processes the input string by first
# filtering out non-alphanumeric characters and converting
# all remaining characters to lowercase.
#
# FOLLOW-UPS (answered below, in this order):
# 1. You may delete at most one character (LeetCode 680).
# 2. Unicode, accents, or locale-specific case.
# 3. Longest palindromic substring.
#
# INTERVIEW NOTES
# A palindrome is symmetric, so compare the two ends and
# move inward; there is no need to build the reversed
# string. Decide what to ignore (non-alphanumerics, case)
# and skip those characters with the pointers instead of
# copying a cleaned string.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import unicodedata


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
```

```
# Idea: two pointers, one at each end, moving toward each other. Skip
# characters that are not letters or digits and compare the lowercase
# forms. The first mismatch ends it; nothing is copied.
# Complexity: Time O(n), Space O(1)
def is_palindrome(s):
    lo = 0
    hi = len(s) - 1
    while lo < hi:
        if not s[lo].isalnum():
            lo += 1
        elif not s[hi].isalnum():
            hi -= 1
        elif s[lo].lower() != s[hi].lower():
            return False
        else:
            lo += 1
            hi -= 1
    return True


# ---- main question: tests ----
def test_main_question():
    assert is_palindrome("A man, a plan, a canal: Panama")
    assert not is_palindrome("race a car")
    assert is_palindrome(" ")
    assert is_palindrome("")


# ============================================================
# FOLLOW-UP 1: You may delete at most one character (LeetCode 680)
# ============================================================
# ---- follow-up 1: helper functions ----
```

```
def letters_only(s):
    # the lowercase letters and digits of s, as a list
    chars = []
    for ch in s:
        if ch.isalnum():
            chars.append(ch.lower())
    return chars


def is_straight(chars, lo, hi):
    # is chars[lo..hi] a palindrome?
    while lo < hi:
        if chars[lo] != chars[hi]:
```

```
            return False
        lo += 1
        hi -= 1
    return True


# ---- follow-up 1: function ----
# Answer: move inward until the first mismatch. Then the answer is
# whether skipping the LEFT character or skipping the RIGHT character
# leaves a palindrome: two more linear checks, so O(n) overall.
# Complexity: Time O(n), Space O(n)
def almost_palindrome(s):
    chars = letters_only(s)
    lo = 0
    hi = len(chars) - 1
    while lo < hi:
        if chars[lo] != chars[hi]:
            if is_straight(chars, lo + 1, hi):
                return True
            return is_straight(chars, lo, hi - 1)
        lo += 1
        hi -= 1
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert almost_palindrome("abca")
    assert almost_palindrome("aba")
    assert not almost_palindrome("abc")


# ========================================================
# FOLLOW-UP 2: Unicode, accents, or locale-specific case
# ========================================================
# ---- follow-up 2: functions ----
# Answer: normalise first (unicodedata.normalize splits "e with an accent"
# into "e" plus an accent mark; drop the marks; casefold() is the proper
# way to ignore case) and decide what counts as a letter. isalnum() covers
# many scripts, but combining marks need the normalisation.
# Complexity: Time O(n), Space O(n)
def clean_unicode(s):
    text = unicodedata.normalize("NFKD", s)
    chars = []
    for ch in text:
```

```
        if not unicodedata.combining(ch):
            chars.append(ch)
    folded = "".join(chars).casefold()
    result = []
    for ch in folded:
```

```
        if ch.isalnum():
            result.append(ch)
    return result


# Complexity: Time O(n), Space O(n)
def is_palindrome_unicode(s):
    chars = clean_unicode(s)
    return is_straight(chars, 0, len(chars) - 1)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert is_palindrome_unicode("Été")
    assert is_palindrome_unicode("\u00c1a")             # "Aa" with an accent on the first A
    assert not is_palindrome("\u00c1a")                 # the plain version sees two different letters
    assert not is_palindrome_unicode("abc")


# ============================================================
# FOLLOW-UP 3: Longest palindromic substring
# ============================================================
# ---- follow-up 3: function ----
# Answer: expand around each of the 2n - 1 centres (n letters and n - 1
# gaps) while both ends match: O(n^2). Manacher's algorithm does it in
# O(n). The two-pointer comparison is the building block.
# Complexity: Time O(n^2), Space O(1) extra
def longest_palindromic_substring(s):
    best = ""
    for centre in range(2 * len(s) - 1):
        i = centre // 2
        j = i + centre % 2
        while i >= 0 and j < len(s) and s[i] == s[j]:
            i -= 1
            j += 1
        if j - i - 1 > len(best):
            best = s[i + 1:j]
    return best
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert longest_palindromic_substring("babad") in ("bab", "aba")
    assert longest_palindromic_substring("cbbd") == "bb"
    assert longest_palindromic_substring("") == ""


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given a string s, return true if it is a palindrome, and false otherwise. A string is considered a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Write a function isPalindrome that implements this logic. Examples

```
Input: s = "H123!@321h"
Output: true

Input: s = "race a car"
Output: false
```

Constraints 1 <= s.length <= 2 * 10^5 The input string s can include any printable ASCII characters. Challenge 1Can you come up with a solution that achieves O(n) time complexity?Challenge 2Can you come up with a solution that achieves O(1) space complexity?Solution 1: Filtering and Reversing Approach This solution processes the input string by first filtering out non-alphanumeric characters and converting all remaining characters to lowercase. It then checks if this filtered string reads the same forward and backward. Filtering: Iterate through the string and collect alphanumeric characters while converting them to lowercase. Reversing and Comparing: Create a reversed version of the filtered string and compare it with the original filtered string to check for palindrome properties. Time Complexity: The time complexity is O(n), where n is the length of the input string. This is because filtering the characters and creating the reversed string both require iterating through the input once. Space Complexity: The space complexity is O(n) for storing the filtered characters and their reversed copy. This space is needed to hold the filtered string and its reversed version. Solution 2: Two-Pointer Approach This solution uses a two-pointer technique to efficiently check if the string is a palindrome. The idea is to use one pointer starting from the beginning of the string (left) and another pointer from the end (right). The pointers move towards each other, skipping non-alphanumeric characters and comparing characters that are alphanumeric. Pointer Initialization: Set two pointers: left at the beginning and right at the end of the string. Skipping Non-Alphanumeric Characters: Move the left pointer to the right while skipping non-alphanumeric characters, and similarly move the right pointer to the left. Comparison: Compare the characters at left and right in a case-insensitive manner. If characters do not match, the string is not a palindrome. Update Pointers: Move the pointers closer to each other and repeat until the pointers meet or cross. Time Complexity: The time complexity is O(n), where n is the length of the string. Each character is processed at most twice (once by each pointer). Space Complexity: The space complexity is O(1) as it only uses a constant amount of extra space for the two pointers, regardless of the size of the input string. NoteCheck out our lesson on the two-pointer approach to learn more about it and its applications!No answers contributed by the community yet.

STUDY PATH: question 27 of 38 | topic 13 of 22: Two pointers: move i and j toward each other (or along two lists) | 4 of 4

Template fit: swap from both ends to reverse the whole array, then reverse every word the same way | no date

EASY | IN APP LC 186 | Exponent | 27 of 38

Reverse a Sentence

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 231/288

ABOUT THIS QUESTION

Topic: Two pointers: move i and j toward each other (or along two lists)

Optimal answer from: your app's interview approach (LeetCode #186)

LeetCode match: LC 186 Reverse Words in a String II

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age) | +38 interviews

Original: https://www.aced.io/questions/3062/reverse-a-sentence

QUESTION

You are given an array of characters arrthat consists of sequences of characters separated by space characters. Each space-delimited sequence of characters defines a word. Implement a function reverseWords that reverses the order of the words in the array in the most efficient manner. Explain your solution and analyze its time and space complexities. Example input: arr = ['p', 'e', 'r', 'f', 'e', 'c', 't', ' ', 'm', 'a', 'k', 'e', 's', ' ', 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e'] output: ['p', 'r', 'a', 'c', 't', 'i', 'c', 'e', ' ', 'm', 'a', 'k', 'e', 's', ' ', 'p', 'e', 'r', 'f', 'e', 'c', 't']Hint 1Even if your language of choice enables this, don't be tempted to use standard functions. Consider how inefficient it is to separately create a string, split the words, and reverse the split.Challenge 1As a general rule, if you're tempted to use a standard or library function, be sure to explain that function's time and space complexity before committing to that solution. You may find a more efficient way to solve the problem yourself. Oftentimes in coding interviews, this is the whole point of the exercise.Challenge 2How have you handled edge cases? For example, empty arrays, arrays with nothing but spaces, arrays with single words only, or with multiple spaces between words, etc.Challenge 3Were you able to achieve a linear time complexity and a constant space complexity?Challenge 4Bonus: how might the mirrorReverse function be implemented with a single index?One possible s

EXAMPLES

Example 1	
Input	`arr = ['t','h','e',' ','s','k','y',' ','i','s',' ','b','l','u','e']`
Output	`''.join(reverse_words(list('the sky is blue'))) -> 'blue is sky the'`
Why	The words come out in reverse order while each word keeps its letters in order (the array is reversed in place).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def reverse_words(arr):
    sentence = "".join(arr)
    words = sentence.split()
    return list(" ".join(reversed(words)))
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

BRUTE FORCE

O(n) time but extra space for the word list, and it collapses repeated spaces. Reversing the whole character array and then each word in place needs O(1) extra space.

OPTIMAL SOLUTION · Reverse the whole array, then reverse each word in place

TIME	SPACE
O(n)	**O(1)** extra on the character array

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The input may have leading, trailing or repeated spaces.

The split-based version already normalises them; the in-place version keeps them, so say which you want. To normalise in place add a two-pointer pass that writes single spaces.

FOLLOW-UP 2 Reverse the letters of each word but keep the word order (LeetCode 557).

Split on single spaces and reverse each word, or reverse each run in place without the first full reversal.

FOLLOW-UP 3 The sentence is a linked list of characters.

Reverse each word segment, then reverse the whole list, using the pointer-flip routine for linked lists.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Reverse a Sentence
#
# You are given an array of characters arrthat consists of
# sequences of characters separated by space characters.
# Each space-delimited sequence of characters defines a
# word. Implement a function reverseWords that reverses
# the order of the words in the array in the most
# efficient manner. Explain your solution and analyze its
# time and space complexities. Example input: arr = [ 'p',
# 'e', 'r', 'f', 'e', 'c', 't', ' ', 'm', 'a', 'k', 'e',
# 's', ' ', 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e' ]
# output: [ 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e', ' ',
# 'm', 'a', 'k', 'e', 's', ' ', 'p', 'e', 'r', 'f', 'e',
# 'c', 't' ]Hint 1Even if your language of choice enables
# this, don't be tempted to use standard functions.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The input may have leading, trailing or repeated
# spaces.
# 2. Reverse the letters of each word but keep the word
# order (LeetCode 557).
# 3. The sentence is a linked list of characters.
#
# INTERVIEW NOTES
# Reversing the order of words but not the letters is the
# same as reversing everything and then reversing each
# word back. That is two passes of the same helper and
# needs no extra memory when the sentence is a mutable
```

```
# character array. First decide how to treat repeated
# spaces.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
```

```
def reverse_range(arr, lo, hi):
    # reverse arr[lo..hi] in place with two pointers
    while lo < hi:
        arr[lo], arr[hi] = arr[hi], arr[lo]
        lo += 1
        hi -= 1


def find_words(arr):
    # the (start, end) of every word: a run of characters without a space
    words = []
    i = 0
    while i < len(arr):
        if arr[i] == " ":
            i += 1
            continue
        j = i
        while j + 1 < len(arr) and arr[j + 1] != " ":
            j += 1
        words.append((i, j))
        i = j + 1
    return words


# ---- main question: main function ----
# Idea: reverse the WHOLE array first (the words are now in the right
# order but spelled backwards), then reverse each word back. Everything
# happens inside the same array: O(n) time, O(1) extra space. Spaces stay
# where the reversal puts them.
# Complexity: Time O(n), Space O(1) extra
def reverse_words(arr):
    reverse_range(arr, 0, len(arr) - 1)
    for start, end in find_words(arr):
        reverse_range(arr, start, end)
    return arr


# ---- main question: tests ----
def test_main_question():
    assert "".join(reverse_words(list("perfect makes practice"))) == "practice makes perfect"
    assert reverse_words([]) == []
    assert reverse_words(list("one")) == list("one")
    assert "".join(reverse_words(list(" a  bc "))) == " bc  a "
```

```
# ============================================================
# FOLLOW-UP 1: The input may have leading, trailing or repeated spaces
```

```
# ========================================================
# ---- follow-up 1: function ----
# Answer: a split-based version already tidies them; the in-place version
# keeps them, so say which you want. To tidy in place, add a pass with a
# second pointer that WRITES the words with single spaces, then cut the
# array to the new length.
# Complexity: Time O(n), Space O(1) extra
def normalise_spaces(arr):
    write = 0
    for read in range(len(arr)):
        if arr[read] != " ":
            arr[write] = arr[read]
            write += 1
        elif write > 0 and arr[write - 1] != " ":
            arr[write] = " "
            write += 1
    if write > 0 and arr[write - 1] == " ":
        write -= 1                                  # no trailing space
    del arr[write:]
    return arr


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert "".join(normalise_spaces(list("  a   bc  "))) == "a bc"
    assert normalise_spaces(list("   ")) == []
    assert normalise_spaces(list("x")) == ["x"]
    tidy = reverse_words(normalise_spaces(list("  one   two three ")))
    assert "".join(tidy) == "three two one"
    random.seed(11)
    for round_number in range(300):                 # compare with the split-based answer
        s = ""
        for i in range(random.randint(0, 12)):
            s += random.choice("ab c")
        parts = s.split(" ")
        parts.reverse()
        assert "".join(reverse_words(list(s))) == " ".join(parts)


# ========================================================
# FOLLOW-UP 2: Reverse the letters of each word but keep the word order (LeetCode 557)
# ========================================================
# ---- follow-up 2: function ----
```

```
# Answer: skip the first full reversal and reverse each word in place.
# Complexity: Time O(n), Space O(1) extra
def reverse_each_word(arr):
    for start, end in find_words(arr):
        reverse_range(arr, start, end)
    return arr


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert "".join(reverse_each_word(list("let's code"))) == "s'tel edoc"


# ========================================================
# FOLLOW-UP 3: The sentence is a linked list of characters
# ========================================================
# Answer: reverse the whole list (the pointer flip), then reverse each
# word's nodes, finding each word by walking to the next space. Same
# idea, pointers instead of indices.
```

```
# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

You are given an array of characters arrthat consists of sequences of characters separated by space characters. Each space-delimited sequence of characters defines a word. Implement a function reverseWords that reverses the order of the words in the array in the most efficient manner. Explain your solution and analyze its time and space complexities. Example

```
input:  arr = [ 'p', 'e', 'r', 'f', 'e', 'c', 't', '  ',
                'm', 'a', 'k', 'e', 's', '  ',
                'p', 'r', 'a', 'c', 't', 'i', 'c', 'e' ]

output: [ 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e', '  ',
          'm', 'a', 'k', 'e', 's', '  ',
          'p', 'e', 'r', 'f', 'e', 'c', 't' ]
```

Hint 1Even if your language of choice enables this, don't be tempted to use standard functions. Consider how inefficient it is to separately create a string, split the words, and reverse the split.Challenge 1As a general rule, if you're tempted to use a standard or library function, be sure to explain that function's time and space complexity before committing to that solution. You may find a more efficient way to solve the problem yourself. Oftentimes in coding interviews, this is the whole point of the exercise.Challenge 2How have you handled edge cases? For example, empty arrays, arrays with nothing but spaces, arrays with single words only, or with multiple spaces between words, etc.Challenge 3Were you able to achieve a linear time complexity and a constant space complexity?Challenge 4Bonus: how might the mirrorReverse function be implemented with a single index?One possible solution is doing a linear iteration on arr while pushing a copy of every word to a stack and then pulling them in reverse order while copying the words back into arr. Another approach is initializing a new array at the same length, iterating over arr from right to left, and copying any sequence of characters to the new array from left to right. Both approaches take O(N) time and at least O(N) space. A more elegant and efficient approach is to reverse all the characters in arr and then reverse the characters in each word separately. While the first reverse gives us the words in the reverse order as we wanted, it also reverses the characters of each word. To fix that, we do the second reverse, which reverses each word separately. Reversing items in an array is done by a 'mirror' function, that swaps the items of every 2 indices with the same distance from the middle. Pseudocode:

OFFICIAL PAGE TEXT

```
function reverseWords(arr):
    # reverse all characters:
    n = arr.length
    mirrorReverse(arr, 0, n-1)

    # reverse each word:
    wordStart = null
    for i from 0 to n-1:
        if (arr[i] == ' '):
            if (wordStart != null):
                mirrorReverse(arr, wordStart, i-1)
                wordStart = null
        else if (i == n-1):
            if (wordStart != null):
                mirrorReverse(arr, wordStart, i)
        else:
            if (wordStart == null):
                wordStart = i

    return arr

# helper function - reverses the order of items in arr
# please note that this is language dependent:
# if are arrays sent by value, reversing should be done in place

function mirrorReverse(arr, start, end):
    tmp = null
    while (start < end):
        tmp = arr[start]
        arr[start] = arr[end]
```

```
        arr[end] = tmp
        start++
        end--
```

Time Complexity: traversing the array twice with a constant number of actions for each item is linear O(N). Space Complexity: using iteration indices and one temp variable takes constant O(1) memory. Answer to bonus question: The mirrorReverse function could be implemented with a single index with left to right linear iteration and swapping arr[i] and arr[n-1-i] as long as i < n-1-i. No answers contributed by the community yet.Related questionsReverse a linked list.Related coursesCourseCourseMachine Learning Engineer Interview PrepLand your dream machine learning role at Meta, Google, Amazon, Apple, Microsoft, Nvidia, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates. Explore ML system design, core concepts, coding, behavioral interviews, and more.Start learningCourseCourseData Science Interview PrepLand your dream data science role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates. Practice statistics, experimentation, coding, SQL, machine learning, behavioral interviews, and more.Start learningPythonJavaScriptC#JavaC++CGoRubySwiftPythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 14 OF 22

Walk two strings at once with two indices

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Walk two strings at once with two indices | 3 questions | tap a title to jump to it

READ FIRST: the code template for these 3 questions (tap here) >

E 28. Find the First Occurrence of a Substring — ColorStack 2025-09-16

E 29. Determine if two words are sorted in lexicographic order. — Exponent no date

E 30. Return all unique substrings of a given string. — Exponent no date

CODE TEMPLATE | GROUP 14 OF 22 | 3 QUESTIONS, ONE PATTERN

Walk two strings at once with two indices

The one idea these questions share

Keep i on one string and j on the other. Skip characters that do not matter, compare the rest, and decide what to do at the first difference. Substring search, ordering, similarity and move-based transforms are all this loop.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def first_difference(a, b, skip="."):
    i = j = 0
    while i < len(a) and j < len(b):
        while i < len(a) and a[i] == skip:      # CHANGE 1: skip characters that do not matter
            i += 1
        while j < len(b) and b[j] == skip:
            j += 1
        if i == len(a) or j == len(b):
            break
        if a[i] != b[j]:                        # CHANGE 2: the rule when they differ
            return i, j
        i, j = i + 1, j + 1
    return None
```

2. Helper functions that usually get written next to it

```
def runs(s):                                    # run-length encoding: "aaab" -> [("a", 3), ("b", 1)]
    out = []
    for ch in s:
        if out and out[-1][0] == ch:
            out[-1][1] += 1
        else:
            out.append([ch, 1])
    return [(c, n) for c, n in out]
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
28	Find the First Occurrence of a Substring	try every start i; inside, j walks the needle; a full match returns i
29	Determine if two words are sorted in lexicographic order.	walk both words; at the first differing letter compare the letters' ranks in the given alphabet; if one word ends first, it must be the shorter
30	Return all unique substrings of a given string.	sort all suffixes; the unique substrings are all prefixes minus the shared prefixes of neighbouring suffixes

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 28 of 38 | topic 14 of 22: Walk two strings at once with two indices | 1 of 3

Template fit: try every start i; inside, j walks the needle; a full match returns i | asked 399 days ago | frequency Single report

EASY | IN APP LC 28 | ColorStack | technical interview | 28 of 38

Find the First Occurrence of a Substring

ASKED 2025-09-16 | 399 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 233/288

ABOUT THIS QUESTION

Topic: Walk two strings at once with two indices

Optimal answer from: your app's interview approach (LeetCode #28)

LeetCode match: LC 28 Find the Index of the First Occurrence in a String

Frequency: Single report | Last asked: 2025-09-16 | Roles: SWE Intern

QUESTION

Reconstructing the "find substring in array" item as substring search: given two strings haystack and needle , return the index of the first occurrence of needle in haystack , or -1 if needle is not part of haystack .

Function signature

```
def str_str(haystack: str, needle: str) -> int:
    """Return the index of the first occurrence of needle in haystack, or -1."""
    ...
```

Follow-up (as reported): Achieve O(n + m) worst-case time with the Knuth-Morris-Pratt (KMP) failure-function approach instead of the naive O(n * m) scan.

EXAMPLES

Example 1	
Input	`haystack = "sadbutsad", needle = "sad"`
Output	`0`
Why	`"sad" occurs at indices 0 and 6; the first is index 0.`

Example 2	
Input	`haystack = "leetcode", needle = "leeto"`
Output	`-1`
Why	`"leeto" does not occur in "leetcode".`

Example 3	
Input	`haystack = "abc", needle = "c"`
Output	`2`

CONSTRAINTS

- 1 <= len(haystack), len(needle) <= 10^4 haystack and needle consist of only lowercase English letters.
- If needle is the empty string it is considered to occur at index 0 (though the constraints above make it non-empty).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_str_str(haystack, needle):
    n, m = len(haystack), len(needle)
    for i in range(n_-_m_+_1):
        if haystack[i:
            i + m] == needle:
            return i
    return -1
```

TIME	SPACE
O(n * m)	O(1)

WHY IT IS SLOW

Each of the n - m + 1 starts may compare up to m characters.

OPTIMAL SOLUTION · Naive scan with early exit

TIME	SPACE
O(n * m) worst case	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make it O(n + m) in the worst case.

Knuth-Morris-Pratt: table[i] is the length of the longest proper prefix of needle[:i + 1] that is also a suffix. While scanning the haystack, on a mismatch fall back to table[j - 1] instead of restarting, so the haystack pointer only moves forward.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the First Occurrence of a Substring
#
# Reconstructing the "find substring in array" item as
# substring search: given two strings haystack and needle
# , return the index of the first occurrence of needle in
# haystack , or -1 if needle is not part of haystack .
# Function signature: def str_str(haystack: str, needle:
# str) -> int: """Return the index of the first occurrence
# of needle in haystack, or -1.""" ... Follow-up (as
# reported): Achieve O(n + m) worst-case time with the
# Knuth-Morris-Pratt (KMP) failure-function approach
# instead of the naive O(n * m) scan.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Make it O(n + m) in the worst case.
#
# INTERVIEW NOTES
# Substring search compares the needle at every start. The
# naive scan restarts after a mismatch; KMP precomputes,
# for each prefix of the needle, the longest proper prefix
# that is also a suffix, so after a mismatch the haystack
# pointer never moves back.
```

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: try every start position i of the haystack; compare the needle
# letter by letter and stop at the first mismatch. The first start where
# all letters match is the answer; -1 if there is none.
# Complexity: Time O(n * m) in the worst case, Space O(1)
def str_str(haystack, needle):
    n = len(haystack)
    m = len(needle)
    for i in range(n - m + 1):
        j = 0
        while j < m and haystack[i + j] == needle[j]:
            j += 1
        if j == m:
            return i
    return -1


# ---- main question: tests ----
def test_main_question():
    assert str_str("sadbutsad", "sad") == 0
    assert str_str("leetcode", "leeto") == -1
    assert str_str("abc", "c") == 2
    assert str_str("abc", "") == 0


# ========================================================
# FOLLOW-UP 1: Make it O(n + m) in the worst case
# ========================================================
# ---- follow-up 1: function ----
# Answer: Knuth-Morris-Pratt. table[i] is the length of the longest proper
# prefix of needle[:i + 1] that is also its suffix. When a letter fails
# to match, fall back to table[j - 1] instead of starting again, so the
# haystack is never re-read: O(n + m).
# Complexity: Time O(n + m), Space O(m)
def str_str_kmp(haystack, needle):
    if needle == "":
        return 0
    table = [0] * len(needle)
    j = 0
    for i in range(1, len(needle)):                # build the table from the needle itself
        while j > 0 and needle[i] != needle[j]:
            j = table[j - 1]
        if needle[i] == needle[j]:
            j += 1
        table[i] = j
    j = 0                                          # how many needle letters match right now
    for i in range(len(haystack)):
        while j > 0 and haystack[i] != needle[j]:
            j = table[j - 1]
        if haystack[i] == needle[j]:
```

```
            j += 1
        if j == len(needle):
            return i - j + 1
    return -1
```

```


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert str_str_kmp("sadbutsad", "sad") == 0
    assert str_str_kmp("leetcode", "leeto") == -1
    assert str_str_kmp("abc", "") == 0
    random.seed(10)
    for round_number in range(500):                # both versions must agree
        haystack = ""
        for i in range(random.randint(0, 10)):
            haystack += random.choice("ab")
        needle = ""
        for i in range(random.randint(0, 4)):
            needle += random.choice("ab")
        assert str_str(haystack, needle) == str_str_kmp(haystack, needle)


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 29 of 38 | topic 14 of 22: Walk two strings at once with two indices | 2 of 3

Template fit: walk both words; at the first differing letter compare the letters' ranks in the given alphabet; if one word ends first, it must be the shorter | no date

EASY | IN APP LC 953 | Exponent | 29 of 38

Determine if two words are sorted in lexicographic order.

ASKED (ANY CO.) ~3 YEARS AGO | OLDER

STUDY: TALK ALL TIME 8.1 30 DAYS 0 PRIORITY 192/288

ABOUT THIS QUESTION

Topic: Walk two strings at once with two indices

Optimal answer from: your app's interview approach (LeetCode #953)

LeetCode match: LC 953 Verifying an Alien Dictionary (inferred from short statement)

Last asked (any company): 3 years ago, about 2023-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/3431/check-lexicographic-order-two-words

QUESTION

Words are ordered using a custom alphabet: rank maps each letter to its position in that alphabet. Return whether word a comes before word b (or is equal to it). The first letter where the words differ decides; if one word is a prefix of the other, the shorter one comes first.

Function signature

def in_order(a: str, b: str, rank: dict) -> bool

EXAMPLES

Example 1

Input	`alphabet = 'hlabcdefgijkmnopqrstuvwxyz' # 'h' comes first, then 'l', then a, b, c ...`
	`rank = {c: i for i, c in enumerate(alphabet)}`
	`in_order('hello', 'leetcode', rank)`
	`in_order('word', 'world', rank)`
	`in_order('app', 'apple', rank)`

Output	`in_order('hello', 'leetcode', rank) -> true`
	`in_order('word', 'world', rank) -> false`
	`in_order('app', 'apple', rank) -> true`

Why	`'h' ranks before 'l', so hello comes first. word vs world differ at d versus l, and d ranks after l, so the order is wrong. A prefix always comes first.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def sorted_pair(a, b, order):
    rank = {c: i for i, c in enumerate(order)}
    ranks_a = [rank[c] for c in a]
    ranks_b = [rank[c] for c in b]
    return ranks_a <= ranks_b
```

TIME	SPACE
O(L)	O(L)

WHY IT IS SLOW

BRUTE FORCE

It builds both full rank lists even when the first letters already decide the answer. Comparing character by character and stopping at the first difference is O(min(len a, len b)) with O(1) extra space.

OPTIMAL SOLUTION · Compare character by character, optionally with a custom alphabet

TIME	SPACE
O(min(len a, len b))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What about uppercase, accents, or case-insensitive order?

Normalize first (casefold, unicode normalization) or give the rank map entries for the extra characters. Python's own string comparison orders by code point, which is not locale order, so use locale.strxfrm or a collation library when real language order matters.

FOLLOW-UP 2 A whole list of words in an alien alphabet (LeetCode 953).

Check each adjacent pair; sortedness of a list is the AND of its adjacent pairs. Build the rank table once.

FOLLOW-UP 3 Case-insensitive or accented comparison.

Normalise (casefold, unicode normalisation) before building ranks, or use locale-aware collation keys; plain code-point order is not language order.

FOLLOW-UP 4 Prefix rule.

'app' comes before 'apple' but 'apple' does not come before 'app': compare lengths only after all shared positions are equal.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Determine if two words are sorted in
# lexicographic order.
#
# Words are ordered using a custom alphabet: rank maps
# each letter to its position in that alphabet. Return
# whether word a comes before word b (or is equal to it).
# The first letter where the words differ decides; if one
# word is a prefix of the other, the shorter one comes
# first.
#
# Signature:
#   def in_order(a: str, b: str, rank: dict) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. What about uppercase, accents, or case-insensitive
# order?
# 2. A whole list of words in an alien alphabet (LeetCode
# 953).
# 3. Case-insensitive or accented comparison.
# 4. Prefix rule.
#
```

```
# INTERVIEW NOTES
# Compare character by character until the first
# difference; whichever character comes first in the
# alphabet decides, and if one word is a prefix of the
# other the shorter word comes first. A custom alphabet is
# just a lookup table from letter to rank.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import unicodedata
import random

ALPHABET = "abcdefghijklmnopqrstuvwxyz"


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
```

```
def make_rank(order=ALPHABET):
    # the position of every letter in the alphabet: {"a": 0, "b": 1, ...}
    rank = {}
    for i in range(len(order)):
        rank[order[i]] = i
    return rank


def rank_of(rank, ch):
    # a character that is not in the alphabet is ranked after all the known ones
    if ch in rank:
        return rank[ch]
    return len(rank) + ord(ch)


# ---- main question: main function ----
# Idea: walk both words with one index. The first position where the
# letters differ decides: compare the two letters by their rank. If one
# word is a prefix of the other, the SHORTER one comes first ("app" is
# before "apple", not the other way round).
# Complexity: Time O(min(len(a), len(b))), Space O(1)
def in_order(a, b, rank):
    i = 0
    while i < len(a) and i < len(b):
        if a[i] != b[i]:
            return rank_of(rank, a[i]) < rank_of(rank, b[i])
        i += 1
    return len(a) <= len(b)                        # the prefix rule (and equal words are in order)


# ---- main question: tests ----
def test_main_question():
    rank = make_rank()
    assert in_order("apple", "banana", rank)
    assert not in_order("banana", "apple", rank)
    assert in_order("app", "apple", rank)            # the prefix rule
    assert not in_order("apple", "app", rank)
    assert in_order("same", "same", rank)
    assert in_order("", "a", rank)
    random.seed(10)
```

```
    for round_number in range(500):                # with the normal alphabet it is plain string compariso
n
        a = ""
        for i in range(random.randint(0, 4)):
            a += random.choice("abc")
        b = ""
```

```
        for i in range(random.randint(0, 4)):
            b += random.choice("abc")
        assert in_order(a, b, rank) == (a <= b)


# ========================================================
# FOLLOW-UP 1: What about uppercase, accents, or case-insensitive order?
# ========================================================


# ========================================================
# FOLLOW-UP 3: Case-insensitive or accented comparison
# ========================================================
# ---- follow-up 1: function ----
# Answer: normalise both words first (casefold ignores case; the unicode
# "NFKD" form splits an accented letter into the letter plus an accent mark,
# which is then dropped), or give the rank map entries for the extra characters.
# Complexity: Time O(L) for a word of length L, Space O(L)
def fold(word):
    text = unicodedata.normalize("NFKD", word.casefold())
    result = ""
    for ch in text:
        if not unicodedata.combining(ch):
            result += ch
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert fold("Great") == "great"
    assert fold("\u00e9clair") == "eclair"          # an accented e becomes a plain e


# ========================================================
# FOLLOW-UP 2: A whole list of words in an alien alphabet (LeetCode 953)
# ========================================================


# ========================================================
# FOLLOW-UP 4: Prefix rule
# ========================================================
# ---- follow-up 2: function ----
# Answer: a list is sorted when every ADJACENT pair is in order. Build the
# rank table once. The prefix rule (a longer word must not come before
# its own prefix) is already inside in_order.
```

```
# Complexity: Time O(total length of the words + A) for alphabet size A, Space O(A + L)
def words_sorted(words, order=ALPHABET, fold_case=False):
    rank = make_rank(order)
    if fold_case:
        folded = []
        for w in words:
            folded.append(fold(w))
        words = folded
    for i in range(len(words) - 1):
        if not in_order(words[i], words[i + 1], rank):
            return False
    return True
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    alien = "hlabcdefgijkmnopqrstuvwxyz"
    assert words_sorted(["hello", "leetcode"], alien)
    alien = "worldabcefghijkmnpqstuvxyz"
    assert not words_sorted(["word", "world", "row"], alien)
    assert not words_sorted(["apple", "app"], ALPHABET)
    assert words_sorted([], ALPHABET)
    assert words_sorted(["x"], ALPHABET)
    assert not words_sorted(["A", "b"], ALPHABET)         # case only matters when asked for
    assert words_sorted(["A", "b"], fold_case=True)
    assert words_sorted(["\u00e9clair", "ecole"], fold_case=True)


# =========================================================
# RUN EVERYTHING
# =========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 30 of 38 | topic 14 of 22: Walk two strings at once with two indices | 3 of 3

Template fit: sort all suffixes; the unique substrings are all prefixes minus the shared prefixes of neighbouring suffixes | no date

EASY | **Exponent** | **30 of 38**

Return all unique substrings of a given string.

ASKED (ANY CO.) ~6 MONTHS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 279/288

ABOUT THIS QUESTION

Topic: Walk two strings at once with two indices

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match (substring enumeration) (inferred from short statement)

Last asked (any company): 6 months ago, about 2026-04 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/1223/unique-substrings-given-string

QUESTION

Return every distinct substring of a string, sorted alphabetically (a substring is a contiguous piece, the whole string and single letters included).

Function signature

def unique_substrings(s: str) -> list

EXAMPLES

Example 1	
Input	`'abc'`
Output	`['a', 'ab', 'abc', 'b', 'bc', 'c']`
Why	`Six different substrings of 'abc'.`

Example 2	
Input	`'aaa'`
Output	`['a', 'aa', 'aaa']`
Why	`'a' appears three times but is listed once.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s):
    result = set()
    for i in range(len(s)):
        for j in range(i + 1, len(s) + 1):
            result.add(s[i:j])
    return result
```

TIME	SPACE
O(n^3)	O(n^3)

WHY IT IS SLOW

There are about n^2 / 2 substrings and each slice copies up to n characters: O(n^3) time and space. If only the count is needed, a suffix array with LCP counts them in O(n log n) without storing any.

OPTIMAL SOLUTION · Count with a suffix array and LCP

TIME	SPACE
O(n^2 log n) with naive sorting (O(n log n) with a real suffix array)	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 List them without building duplicates first.

Insert every suffix into a trie; a substring is new exactly when its trie node is created. O(n^2) time and nodes, and the substrings appear without duplicates.

FOLLOW-UP 2 Return the k-th lexicographic distinct substring, or the longest repeated substring.

Both use the suffix array with LCP: the longest repeated substring is the maximum LCP value, and the k-th substring is found by walking the sorted suffixes and subtracting the new prefixes each one contributes.

FOLLOW-UP 3 The string has a million characters.

Quadratic is out. Build a suffix automaton (O(n)) and count distinct substrings as the sum of len(v) - len(link(v)) over states, or use a suffix array with O(n log n) construction.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Return all unique substrings of a given
# string.
#
# Return every distinct substring of a string, sorted
# alphabetically (a substring is a contiguous piece, the
# whole string and single letters included).
#
# Signature:
#   def unique_substrings(s: str) -> list
#
# Examples:
# 'abc'  ->  ['a', 'ab', 'abc', 'b', 'bc', 'c']
# 'aaa'  ->  ['a', 'aa', 'aaa']
#
# FOLLOW-UPS (answered below, in this order):
# 1. List them without building duplicates first.
# 2. Return the k-th lexicographic distinct substring, or
# the longest repeated substring.
# 3. The string has a million characters.
#
# INTERVIEW NOTES
# If the output must list the substrings, its size already
# is quadratic, so a set over all slices is the honest
# answer. If only the count is needed, nothing needs to be
# stored: distinct substrings equal total suffix lengths
# minus the sum of longest-common-prefixes between
# adjacent sorted suffixes.

# ======================================================
# HOW THIS FILE IS ORGANISED
```

```
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea (the simplest): cut out every slice s[i:j] and put it in a set (a
# set keeps each text only once), then sort. That is O(n^3): fine for a
# short string; the follow-ups make it faster.
```

```
# Complexity: Time O(n^3) (n^2 substrings, each copied in O(n)), Space O(n^3) for the stored text
def unique_substrings(s):
    seen = set()
    for i in range(len(s)):
        for j in range(i + 1, len(s) + 1):
            seen.add(s[i:j])
    return sorted(seen)


# ---- main question: tests ----
def test_main_question():
    assert unique_substrings("abc") == ["a", "ab", "abc", "b", "bc", "c"]
    assert unique_substrings("aaa") == ["a", "aa", "aaa"]
    assert unique_substrings("") == []


# ============================================================
# FOLLOW-UP 1: List them without building duplicates first
# ============================================================
# ---- follow-up 1: function ----
# Answer: insert every suffix into a trie (a tree with one character per
# level). A substring is NEW exactly when its trie node is created while
# inserting. O(n^2) time and nodes, and the substrings come out once each.
# Complexity: Time O(n^2), Space O(n^2) trie nodes
def trie_substrings(s):
    found = []
    root = {}
    for start in range(len(s)):
        node = root
        for end in range(start, len(s)):
            ch = s[end]
            if ch not in node:
                node[ch] = {}
                found.append(s[start:end + 1])      # a new node: a substring we have not seen
            node = node[ch]
    return found


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert sorted(trie_substrings("aaa")) == ["a", "aa", "aaa"]


# ============================================================
# FOLLOW-UP 2: Return the k-th lexicographic distinct substring, or the longest repeated substring
```

```
# ============================================================
# ---- follow-up 2: helper functions ----
def suffix_array(s):
```

```
    # the start positions of all suffixes, in the sorted order of the suffixes
    pairs = []
    for i in range(len(s)):
        pairs.append((s[i:], i))
    pairs.sort()
    starts = []
    for suffix, i in pairs:
        starts.append(i)
    return starts


def lcp_array(s, sa):
    # lcp[i] = how many letters the suffix sa[i] shares with the one before it
    # in sorted order (Kasai's method: O(n) in total, because the shared
    # length drops by at most one from one start to the next)
    n = len(s)
    rank = [0] * n
    for i in range(n):
        rank[sa[i]] = i
    lcp = [0] * n
    h = 0
    for start in range(n):
        if rank[start] > 0:
            before = sa[rank[start] - 1]
            while start + h < n and before + h < n and s[start + h] == s[before + h]:
                h += 1
            lcp[rank[start]] = h
            if h > 0:
                h -= 1
        else:
            h = 0
    return lcp


# ---- follow-up 2: functions ----
# Answer: both use the suffix array with the LCP array. Walking the sorted
# suffixes, each suffix brings (its length - its LCP) NEW substrings, so
# the k-th one is found by subtracting those counts. The longest repeated
# substring is the largest LCP value.
# Complexity: Time O(n^2 log n) (sorting the suffixes), Space O(n^2)
def kth_substring(s, k):
    sa = suffix_array(s)
```

```
    lcp = lcp_array(s, sa)
    for i in range(len(sa)):
        fresh = len(s) - sa[i] - lcp[i]
        if k <= fresh:
            return s[sa[i]:sa[i] + lcp[i] + k]
        k -= fresh
    return None


# Complexity: Time O(n^2 log n), Space O(n^2)
def longest_repeated(s):
    if s == "":
        return ""
    sa = suffix_array(s)
    lcp = lcp_array(s, sa)
    best = 0
    for i in range(len(lcp)):
        if lcp[i] > lcp[best]:
            best = i
    return s[sa[best]:sa[best] + lcp[best]]
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert kth_substring("abc", 4) == "b"
    assert kth_substring("abc", 7) is None
    assert longest_repeated("banana") == "ana"
    assert longest_repeated("abcd") == ""
    random.seed(12)
    for round_number in range(200):
        s = ""
        for i in range(random.randint(0, 9)):
            s += random.choice("abc")
        everything = unique_substrings(s)
        for k in range(1, len(everything) + 1):
            assert kth_substring(s, k) == everything[k - 1]


# ======================================================
# FOLLOW-UP 3: The string has a million characters
# ======================================================
# ---- follow-up 3: function ----
# Answer: quadratic is out. Build a suffix AUTOMATON in O(n): every state
# stands for a group of substrings, and the number of distinct substrings
# is the sum of (length[v] - length[link[v]]) over its states. (Or use a
```

```
# suffix array built in O(n log n).)
# Complexity: Time O(n) for a small alphabet, Space O(n)
def count_distinct(s):
    length = [0]                    # the longest substring of each state
    link = [-1]                     # the suffix link of each state
    transitions = [{}]              # state -> {letter: next state}
    last = 0
    for ch in s:
        cur = len(length)
        length.append(length[last] + 1)
        link.append(0)
        transitions.append({})
        p = last
        while p != -1 and ch not in transitions[p]:
            transitions[p][ch] = cur
            p = link[p]
        if p != -1:
            q = transitions[p][ch]
            if length[p] + 1 == length[q]:
                link[cur] = q
            else:
                clone = len(length)                 # split q: a copy with the shorter length
                length.append(length[p] + 1)
                link.append(link[q])
                transitions.append(dict(transitions[q]))
                while p != -1 and transitions[p].get(ch) == q:
                    transitions[p][ch] = clone
                    p = link[p]
                link[q] = clone
                link[cur] = clone
        last = cur
    total = 0
    for v in range(1, len(length)):
        total += length[v] - length[link[v]]
    return total


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert count_distinct("aaa") == 3
    assert count_distinct("") == 0
```

```
    random.seed(13)
    for round_number in range(300):
        s = ""
        for i in range(random.randint(0, 9)):
            s += random.choice("abc")
        everything = unique_substrings(s)
        assert count_distinct(s) == len(everything)
        assert sorted(trie_substrings(s)) == everything


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 15 OF 22

Simulate a machine: a class with state and one branch per kind of event

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Simulate a machine: a class with state and one branch per kind of event | 1 questions | tap a title to jump to it

READ FIRST: the code template for these 1 questions (tap here) >

E 31. Mario-Style Score Tracker (OOP) **ColorStack 2025-10-11**

CODE TEMPLATE | GROUP 15 OF 22 | 1 QUESTIONS, ONE PATTERN

Simulate a machine: a class with state and one branch per kind of event

The one idea these questions share

Put the state in __init__, give it one method that takes an event, and write one branch per event kind. Board games keep a grid or counters and check for a win after each move; game-of-life style updates compute every cell from the old state before changing it.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
class Machine:
    def __init__(self):
        self.score, self.lives = 0, 3             # CHANGE 1: the state
        self.over = False

    def process(self, event):
        if self.over:
            return
        if event == "coin":                       # CHANGE 2: one branch per kind of event
            self.score += 100
        elif event == "damage":
            self.lives -= 1
            self.over = self.lives == 0
```

2. Helper functions that usually get written next to it

```
def neighbors8(r, c):
    return [(r + dr, c + dc) for dr in (-1, 0, 1) for dc in (-1, 0, 1) if dr or dc]


def run_through(board, r, c, dr, dc, player):    # length of the line of one player through a cell
    n, run = len(board), 1
    for sign in (1, -1):
        a, b = r + sign * dr, c + sign * dc
        while 0 <= a < n and 0 <= b < n and board[a][b] == player:
            run += 1
            a, b = a + sign * dr, b + sign * dc
    return run
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
31	Mario-Style Score Tracker (OOP)	the template as written: an event name decides how score, lives and size change

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 31 of 38 | topic 15 of 22: Simulate a machine: a class with state and one branch per kind of event | 1 of 1

Template fit: the template as written: an event name decides how score, lives and size change | asked 374 days ago | frequency Single report

EASY | ColorStack | technical interview | 31 of 38

Mario-Style Score Tracker (OOP)

ASKED 2025-10-11 | 374 DAYS AGO | OLDER

STUDY: TALK **ALL TIME 16.5** **30 DAYS 0** **PRIORITY 182/288**

ABOUT THIS QUESTION

Topic: Simulate a machine: a class with state and one branch per kind of event

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1472 Design Browser History

LeetCode match: State machine design (related: LC 1472 / simulation)

Frequency: Single report | Last asked: 2025-10-11 | Roles: SWE Intern

QUESTION

Design a class that tracks the state of a platformer character across a sequence of events. The character has a score (starts at 0), a lives count (starts at 3), and a big flag (starts False). Process events with these rules: "coin" : add 100 to the score. "mushroom" : add 1000 to the score and set big = True . "enemy" : add 200 to the score (the character stomps an enemy). "damage" : if big is True , set big = False (no life lost); otherwise decrease lives by 1. Score is unchanged. The character is alive while lives > 0 . Once lives reaches 0, the game is over and any further events are ignored. Implement the class and a method to run a batch of events.

Function signature

```
class MarioGame:
    def __init__(self) -> None:
        """score=0, lives=3, big=False."""
        ...
    def process(self, event: str) -> None:
        """Apply one event, ignoring it if the game is already over."""
        ...
    def run(self, events: list[str]) -> None:
        """Apply a sequence of events in order."""
        ...
    def get_score(self) -> int:
        ...
    def is_alive(self) -> bool:
        ...
```

EXAMPLES

Example 1	
Input	`g = MarioGame(); g.run(["coin", "mushroom", "enemy", "damage", "damage"]); (g.get_score(), g.is_alive())`
Output	`(1300, True)`
Why	`coin -> 100; mushroom -> 1100, big=True; enemy -> 1300; damage -> big=False (no life lost); damage -> small, lives 3->2. Score 1300, still alive.`

QUESTION

Example 2	
Input	`g = MarioGame(); g.run(["damage", "damage", "damage", "coin"]); (g.get_score(), g.is_alive())`
Output	`(0, False)`
Why	`Three small-state hits drop lives 3->0 (game over). The trailing "coin" is ignored.`

CONSTRAINTS

- Event strings are always one of the four listed above.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_mario(events):
    score, lives, big = 0, 3, False
    for e in events:
        if lives == 0:
            break
        if e == "coin":
            score += 100
        elif e == "mushroom":
            score, big = score + 1000, True
        elif e == "enemy":
            score += 200
        elif e == "damage":
            if big:
                big = False
            else:
                lives -= 1
    return score, lives > 0
```

TIME	SPACE
O(events)	O(1)

WHY IT IS SLOW

It is the same rule set written as a pure function; it is the oracle for the class.

OPTIMAL SOLUTION · A small state class

TIME	SPACE
O(events)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Add undo() to take back the last event.

Before applying an event push a snapshot (score, lives, big) on a stack; undo pops it and restores the three fields. Events ignored after game over should not push a snapshot.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Mario-Style Score Tracker (OOP)
#
# Design a class that tracks the state of a platformer
# character across a sequence of events. The character has
# a score (starts at 0), a lives count (starts at 3), and
# a big flag (starts False ). Process events with these
# rules: "coin" : add 100 to the score. "mushroom" : add
# 1000 to the score and set big = True . "enemy" : add 200
# to the score (the character stomps an enemy). "damage" :
# if big is True , set big = False (no life lost);
# otherwise decrease lives by 1. Score is unchanged. The
# character is alive while lives > 0 . Once lives reaches
# 0, the game is over and any further events are ignored.
# Implement the class and a method to run a batch of
# events. Function signature: class MarioGame: def
# __init__(self) -> None: """score=0, lives=3,
# big=False.""" ... def process(self, event: str) -> None:
# """Apply one event, ignoring it if the game is already
# over.""" ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Add undo() to take back the last event.
#
# INTERVIEW NOTES
# A small state machine: score, lives and a big flag
# change according to fixed rules, and once lives reach
# zero every later event is ignored. Keep the rules in one
# method and let run() call it for each event; never reach
# into the fields from outside.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: a small class with the state (score, lives, big or small) and ONE
```

```
# method that owns the rules, one branch per kind of event. The game ends
# when the lives reach zero; events after that are ignored.
# Complexity: Time O(1) per event, Space O(1)
class MarioGame:
    def __init__(self):
        self.score = 0
        self.lives = 3
        self.big = False

    def process(self, event):
        if self.lives == 0:
            return                                  # the game is over
        if event == "coin":
```

```
            self.score += 100
        elif event == "mushroom":
            self.score += 1000
            self.big = True
        elif event == "enemy":
            self.score += 200
        elif event == "damage":
            if self.big:
                self.big = False                    # damage first costs the mushroom power-up
            else:
                self.lives -= 1

    def run(self, events):
        for event in events:
            self.process(event)

    def get_score(self):
        return self.score

    def is_alive(self):
        return self.lives > 0


# ---- main question: tests ----
def test_main_question():
    g = MarioGame()
    g.run(["coin", "mushroom", "enemy", "damage", "damage"])
    assert (g.get_score(), g.is_alive()) == (1300, True)
    g = MarioGame()
    g.run(["damage", "damage", "damage", "coin"])
    assert (g.get_score(), g.is_alive()) == (0, False)      # the coin after game over is ignored
```

```
# ======================================================
# FOLLOW-UP 1: Add undo() to take back the last event
# ======================================================
# ---- follow-up 1: function ----
# Answer: before applying an event push a snapshot (score, lives, big) on a
# stack; undo pops the last snapshot and restores the three fields. Events
# that are ignored after the game is over must NOT push a snapshot.
# Complexity: Time O(1) per event and per undo, Space O(events) for the snapshots
class UndoMarioGame(MarioGame):
    def __init__(self):
        MarioGame.__init__(self)
        self.history = []

    def process(self, event):
        if self.lives == 0:
            return
        self.history.append((self.score, self.lives, self.big))
        MarioGame.process(self, event)

    def undo(self):
        if len(self.history) > 0:
            self.score, self.lives, self.big = self.history.pop()


# ---- follow-up 1: tests ----
def test_follow_up_1():
    random.seed(60)
    for round_number in range(300):
        events = []
        for i in range(random.randint(0, 12)):
            events.append(random.choice(["coin", "mushroom", "enemy", "damage"]))
        game = UndoMarioGame()
```

```
        game.run(events)
        for i in range(len(events)):
            game.undo()
        assert (game.get_score(), game.lives, game.big) == (0, 3, False)       # undoing everything restore
s the start


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 16 OF 22

Digits and characters: peel one at a time and carry

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Digits and characters: peel one at a time and carry | 1 questions | tap a title to jump to it

READ FIRST: the code template for these 1 questions (tap here) >

E 32. Caesar Cipher: Recover Key, Encrypt, and Decrypt **ColorStack 2026-02-08**

CODE TEMPLATE | GROUP 16 OF 22 | 1 QUESTIONS, ONE PATTERN

Digits and characters: peel one at a time and carry

The one idea these questions share

Work one digit or character at a time: n % 10 peels the last digit, n // 10 removes it, a carry moves overflow to the next place, and (ord(c) - 97 + k) % 26 shifts a letter. Build the answer in a list and reverse or join at the end.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def add_digits(a, b):                          # two digit lists, most significant first
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry + (a[i] if i >= 0 else 0) + (b[j] if j >= 0 else 0)
        out.append(total % 10)                 # CHANGE 1: what you build per place
        carry = total // 10                    # CHANGE 2: what you carry
        i, j = i - 1, j - 1
    return out[::-1] or [0]
```

2. Helper functions that usually get written next to it

```
def digits_of(n):
    out = []
    while n:
        n, d = divmod(n, 10)
        out.append(d)
    return out[::-1]


def shift_letter(ch, k):
    return chr((ord(ch) - 97 + k) % 26 + 97)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
32	Caesar Cipher: Recover Key, Encrypt, and Decrypt	per letter: (ord(ch) - 97 + k) % 26 + 97; the key comes from one letter pair; decrypt uses -k

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 32 of 38 | topic 16 of 22: Digits and characters: peel one at a time and carry | 1 of 1

Template fit: per letter: (ord(ch) - 97 + k) % 26 + 97; the key comes from one letter pair; decrypt uses -k | asked 254 days ago | frequency Single report

EASY | ColorStack | technical interview | 32 of 38

Caesar Cipher: Recover Key, Encrypt, and Decrypt

ASKED 2026-02-08 | 254 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 243/288

ABOUT THIS QUESTION

Topic: Digits and characters: peel one at a time and carry
Optimal answer from: written and tested here for this exact question
LeetCode match: LC 1160 / Caesar cipher (related: character shifts)
Frequency: Single report | Last asked: 2026-02-08 | Roles: SWE Intern

QUESTION

A message was scrambled with a Caesar cipher: every lowercase letter was shifted forward by the same secret amount k (0 <= k < 26) with wraparound (z +1 -> a); all non-lowercase characters were left unchanged. You are given one example pair — a known plaintext and its corresponding ciphertext — from which you must recover the secret shift, and then implement the cipher in both directions. Specifically implement: find_shift(plaintext, ciphertext) -> int : return the shift k (0..25) that maps plaintext to ciphertext . The two strings have equal length and contain at least one lowercase letter that pins down k . encrypt(text, k) -> str : shift each lowercase letter forward by k with wraparound; leave other characters unchanged. decrypt(text, k) -> str : the inverse of encrypt — shift each lowercase letter backward by k with wraparound.

Function signature

```
def find_shift(plaintext: str, ciphertext: str) -> int:
    ...
def encrypt(text: str, k: int) -> str:
    ...
def decrypt(text: str, k: int) -> str:
    ...
```

Follow-up (as reported): Given only a ciphertext (no key and no plaintext), how would you break the cipher? Discuss trying all 26 shifts (brute force) or English letter-frequency analysis.

EXAMPLES

Example 1	
Input	`find_shift("abc", "def")`
Output	`3`
Why	`'a'->'d', 'b'->'e', 'c'->'f' is a uniform forward shift of 3.`

Example 2	
Input	`encrypt("hello world", 3)`
Output	`"khoor zruog"`
Why	`Each letter moves forward 3 ('h'->'k', 'o'->'r', 'w'->'z'); the space is unchanged.`

Example 3	
Input	`decrypt("khoor zruog", 3)`
Output	`"hello world"`
Why	`decrypt inverts encrypt, so the original message is recovered.`

QUESTION

Example 4	
Input	`encrypt("xyz", 3)`
Output	`"abc"`
Why	`Wraparound: 'x'->'a', 'y'->'b', 'z'->'c'.`

CONSTRAINTS

- 1 <= len(text) <= 10^5 . text may contain lowercase letters, spaces, digits, and punctuation; only lowercase letters a – z are shifted.
- For find_shift , len(plaintext) == len(ciphertext) and they align character by character.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_find_shift(plaintext, ciphertext):
    for k in range(26):
        if encrypt(plaintext, k) == ciphertext:
            return k
    return -1
```

TIME	SPACE
O(26 n)	O(n)

WHY IT IS SLOW

It encrypts the whole text for each guess although one letter pair fixes the key.

OPTIMAL SOLUTION · Key from one letter pair, shared shift helper

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 You only have a ciphertext. How do you break it?

Try all 26 shifts and score each candidate by how English it looks: add a weight for every common letter (e, t, a, o, i, n, s, h, r, d, l, u) and keep the shift with the best score. Longer texts make the frequency signal reliable.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Caesar Cipher: Recover Key, Encrypt, and
# Decrypt
#
# A message was scrambled with a Caesar cipher: every
# lowercase letter was shifted forward by the same secret
# amount k (0 <= k < 26) with wraparound ( z +1 -> a );
# all non-lowercase characters were left unchanged. You
# are given one example pair — a known plaintext and its
# corresponding ciphertext — from which you must recover
# the secret shift, and then implement the cipher in both
```

```
# directions. Specifically implement:
# find_shift(plaintext, ciphertext) -> int : return the
# shift k (0..25) that maps plaintext to ciphertext . The
# two strings have equal length and contain at least one
# lowercase letter that pins down k . encrypt(text, k) ->
# str : shift each lowercase letter forward by k with
# wraparound; leave other characters unchanged.
# decrypt(text, k) -> str : the inverse of encrypt — shift
# each lowercase letter backward by k with wraparound.
#
# FOLLOW-UPS (answered below, in this order):
# 1. You only have a ciphertext. How do you break it?
#
# INTERVIEW NOTES
# A Caesar shift moves each lowercase letter forward by k
# modulo 26. The key is recovered from any one lowercase
# letter pair: (ord(cipher) - ord(plain)) mod 26. Encrypt
# adds k, decrypt subtracts k, both with the same helper
# and the wrap-around handled by the modulo.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main functions ----
# A Caesar cipher moves every lowercase letter k places along the
# alphabet (wrapping z to a). Other characters stay as they are.

# Idea: one letter pair is enough to recover the key: the key is how far
```

```
# the ciphertext letter is from the plaintext letter, modulo 26.
# Complexity: Time O(1) usually (the first letter), O(n) at worst, Space O(1)
def find_shift(plaintext, ciphertext):
    for i in range(len(plaintext)):
        if "a" <= plaintext[i] <= "z":
            return (ord(ciphertext[i]) - ord(plaintext[i])) % 26
    return 0


# The same helper shifts forward (encrypt) or backward (decrypt, shift by -k).
# Complexity: Time O(n), Space O(n)
def shift_text(text, k):
    result = ""
    for ch in text:
        if "a" <= ch <= "z":
            position = ord(ch) - ord("a")                    # 0 for "a" ... 25 for "z"
            ch = chr((position + k) % 26 + ord("a"))
        result += ch
    return result


# Complexity: Time O(n), Space O(n)
def encrypt(text, k):
    return shift_text(text, k)


# Complexity: Time O(n), Space O(n)
def decrypt(text, k):
    return shift_text(text, -k)
```

```
# ---- main question: tests ----
def test_main_question():
    assert find_shift("abc", "def") == 3
    assert encrypt("hello world", 3) == "khoor zruog"
    assert decrypt("khoor zruog", 3) == "hello world"
    assert encrypt("xyz", 3) == "abc"
    assert encrypt("a1 b!", 2) == "c1 d!"


# ========================================================
# FOLLOW-UP 1: You only have a ciphertext. How do you break it?
# ========================================================
# ---- follow-up 1: function ----
# Answer: try all 26 keys and score each decrypted text by how English it
```

```
# looks: give every common letter (e, t, a, o, i, n, s, h, r, d, l, u) a
# weight (e the biggest) and add up the weights of the letters. The key
# with the best score wins. Longer texts make the letter-frequency signal
# reliable.
# Complexity: Time O(26 * n), Space O(n)
def break_caesar(ciphertext):
    common = "etaoinshrdlu"
    weight = {}
    for i in range(len(common)):
        weight[common[i]] = 12 - i
    best_key = 0
    best_score = -1
    for k in range(26):
        text = decrypt(ciphertext, k)
        score = 0
        for ch in text:
            if ch in weight:
                score += weight[ch]
        if score > best_score:
            best_score = score
            best_key = k
    return best_key


# ---- follow-up 1: tests ----
def test_follow_up_1():
    sample = "the quick brown fox jumps over the lazy dog "
    sample += "while the sun sets over the quiet hills "
    sample += "and the rivers run to the sea"
    for k in range(26):
        assert decrypt(encrypt(sample, k), k) == sample
        assert break_caesar(encrypt(sample, k)) == k


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 17 OF 22

Group by a canonical key

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Group by a canonical key | 1 questions | tap a title to jump to it

READ FIRST: the code template for these 1 questions (tap here) >

E 33. Given an array of strings, return the words that can be typed using letters of the alphabet on only o... **Exponent no date**

CODE TEMPLATE | GROUP 17 OF 22 | 1 QUESTIONS, ONE PATTERN

Group by a canonical key

The one idea these questions share

Find a key that is the same for everything that should be grouped (sorted letters, letter counts, shifts between letters, the keyboard row) and put the items into a dict under that key. Equal key means same group.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import defaultdict


def group_by_key(words, key):
    groups = defaultdict(list)
    for w in words:
        groups[key(w)].append(w)              # CHANGE: the key function decides everything
    return list(groups.values())
```

2. Helper functions that usually get written next to it

```
from collections import Counter


def letter_counts(word):                    # a hashable signature that ignores order
    return tuple(sorted(Counter(word).items()))


def shift_signature(word):
    return tuple((ord(word[i]) - ord(word[i - 1])) % 26 for i in range(1, len(word)))
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
33	Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.	key = the keyboard row of each letter; a word qualifies when its letters map to one row

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 33 of 38 | topic 17 of 22: Group by a canonical key | 1 of 1

Template fit: key = the keyboard row of each letter; a word qualifies when its letters map to one row | no date

EASY | Exponent | 33 of 38

Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 16.5 30 DAYS 0 PRIORITY 181/288

ABOUT THIS QUESTION

Topic: Group by a canonical key

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 500 Keyboard Row (inferred from short statement)

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/7293/return-words-one-keyboard-row

QUESTION

Given an array of words, return those that can be typed using letters from a single row of an American keyboard (the rows are qwertyuiop, asdfghjkl and zxcvbnm). Ignore upper and lower case and keep the input order.

Function signature

def findWords(words: list) -> list

EXAMPLES

Example 1	
Input	`['Hello', 'Alaska', 'Dad', 'Peace']`
Output	`['Alaska', 'Dad']`
Why	`'Alaska' and 'Dad' use only the middle row; 'Hello' and 'Peace' mix rows.`

Example 2	
Input	`['omk']`
Output	`[]`
Why	`'omk' mixes the top and bottom rows.`

Example 3	
Input	`['adsdf', 'sfd']`
Output	`['adsdf', 'sfd']`
Why	`Both words stay on one row.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
ROWS = [set("qwertyuiop"), set("asdfghjkl"),
        set("zxcvbnm")]
def brute(words):
    found = []
    for word in words:
        letters = set(word.lower())
        for row in ROWS:
            if letters <= row:
                found.append(word)
                break
    return found
```

TIME	SPACE
O(W * L * 3)	O(1)

WHY IT IS SLOW

Up to three scans per word; building one letter-to-row table makes each word a single scan, O(total characters).

OPTIMAL SOLUTION · Map each letter to its row and compare sets

TIME	SPACE
O(total characters)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if the keyboard layout is a parameter (Dvorak, AZERTY, a phone keypad)?

Nothing changes except the row table, which is built from the layout argument. Build the letter-to-row map once and reuse it across queries.

FOLLOW-UP 2 Other layouts (Dvorak, AZERTY) or a phone keypad.

Pass the rows as a parameter and build the letter-to-row dict from it. Nothing else changes.

FOLLOW-UP 3 A different keyboard layout.

Pass the rows as a parameter and build the letter-to-row map from it (a phone keypad or Dvorak works the same way). Letters that are on no row make the word invalid, which the code checks.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given an array of strings, return the
# words that can be typed using letters of the alphabet on
# only one row of American keyboard.
#
# Given an array of words, return those that can be typed
# using letters from a single row of an American keyboard
# (the rows are qwertyuiop, asdfghjkl and zxcvbnm). Ignore
# upper and lower case and keep the input order.
#
# Signature:
```

```
#   def findWords(words: list) -> list
#
# Examples:
# ['Hello', 'Alaska', 'Dad', 'Peace']  ->  ['Alaska',
#     'Dad']
# ['omk']  ->  []
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if the keyboard layout is a parameter (Dvorak,
# AZERTY, a phone keypad)?
# 2. Other layouts (Dvorak, AZERTY) or a phone keypad.
# 3. A different keyboard layout.
#
# INTERVIEW NOTES
# Each letter belongs to exactly one row, so the question
# for a word is whether all its letters map to the same
# row. Build the letter-to-row lookup once and check each
# word with a set of rows.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
QWERTY = ["qwertyuiop", "asdfghjkl", "zxcvbnm"]


def row_map(rows):
    # letter -> the number of its row; built once and reused. Change `rows`
    # for another keyboard layout (follow-up 1).
    row_of = {}
    for i in range(len(rows)):
        for ch in rows[i]:
            row_of[ch] = i
    return row_of


# ---- main question: main function ----
# Which words can be typed with the letters of ONE keyboard row?
# Idea: build a dictionary from letter to row number (one entry per letter).
# A word qualifies if all of its (lowercased, so "Alaska" works) letters are
# on the same row. A letter that is on no row makes the word invalid.
# Complexity: Time O(total length of all words), Space O(1) (26 letters)
def findWords(words, rows=QWERTY):
    row_of = row_map(rows)
    result = []
    for word in words:
        letters = word.lower()
        valid = True
        used_rows = set()
        for ch in letters:
            if ch not in row_of:
                valid = False                          # a letter on no row
                break
            used_rows.add(row_of[ch])
        if valid and len(used_rows) <= 1:
            result.append(word)
    return result
```

```
# ---- main question: tests ----
def test_main_question():
    assert findWords(["Hello", "Alaska", "Dad", "Peace"]) == ["Alaska", "Dad"]
    assert findWords([]) == []
    assert findWords(["", "a1"]) == [""]


# ========================================================
# FOLLOW-UP 1: What if the keyboard layout is a parameter (Dvorak, AZERTY, a phone keypad)?
# ========================================================


# ========================================================
# FOLLOW-UP 2: Other layouts (Dvorak, AZERTY) or a phone keypad
```

```
# ========================================================


# ========================================================
# FOLLOW-UP 3: A different keyboard layout
# ========================================================
# Answer: pass the rows as a parameter (rows=...) and build the
# letter-to-row map from it: a phone keypad or Dvorak works the same way.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    keypad = ["abc", "def", "ghi"]
    assert findWords(["bad", "fed", "cab"], keypad) == ["fed", "cab"]
    dvorak = ["pyfgcrl", "aoeuidhtns", "qjkxbmwvz"]
    assert findWords(["tent"], dvorak) == ["tent"]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 18 OF 22

K-way merge: a heap of the next item from each source

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

K-way merge: a heap of the next item from each source | 1 questions | tap a title to jump to it

READ FIRST: the code template for these 1 questions (tap here) >

E 34. Find k pairs with smallest sums **PracHub 2025-11-01**

CODE TEMPLATE | GROUP 18 OF 22 | 1 QUESTIONS, ONE PATTERN

K-way merge: a heap of the next item from each source

The one idea these questions share

Put the first item of every list (with its list number and position) on a heap. Pop the smallest, record it, and push the next item from the same list. After k pops you have the k best without sorting everything.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
import heapq


def k_way(lists, k):
    heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]   # CHANGE 1: what you seed with
    heapq.heapify(heap)
    out = []
    while heap and len(out) < k:
        val, i, j = heapq.heappop(heap)
        out.append(val)
        if j + 1 < len(lists[i]):                                     # CHANGE 2: the successor
            heapq.heappush(heap, (lists[i][j + 1], i, j + 1))
    return out
```

2. Helper functions that usually get written next to it

```
def merge_two(a, b):                      # the simple two-list case of the same idea
    i = j = 0
    out = []
    while i < len(a) or j < len(b):
        if j == len(b) or (i < len(a) and a[i] <= b[j]):
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    return out
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
34	Find k pairs with smallest sums	seed with (a[i] + b[0], i, 0); the successor of pair (i, j) is (i, j + 1)

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 34 of 38 | topic 18 of 22: K-way merge: a heap of the next item from each source | 1 of 1

Template fit: seed with (a[i] + b[0], i, 0); the successor of pair (i, j) is (i, j + 1) | asked 353 days ago

EASY | IN APP LC 373 | PracHub | Technical Screen | 34 of 38

Find k pairs with smallest sums

POSTED 2025-11-01 | 353 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 32.6 | 30 DAYS 0 | PRIORITY 139/288

ABOUT THIS QUESTION

Topic: K-way merge: a heap of the next item from each source

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #373 Find K Pairs with Smallest Sums

LeetCode match: LC 373 Find K Pairs with Smallest Sums

Posted on PracHub: 2025-11-01 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/find-k-pairs-with-smallest-sums

QUESTION

You are given two **sorted** integer arrays **`arr1`** and **`arr2`** (each sorted in non-decreasing order), and an integer **`k`**. Consider all possible pairs formed by taking one element from **`arr1`** and one element from **`arr2`**. For indices **`i`** and **`j`**, the pair is **`(arr1[i], arr2[j])`** with sum **`arr1[i] + arr2[j]`**. Your task is to return the **k pairs `(arr1[i], arr2[j])`** with the smallest sums. If there are fewer than **`k`** total possible pairs, return all possible pairs. You may return the pairs in any order. ---

Function Signature

(one possible form)

```
kSmallestPairs(arr1: List[int], arr2: List[int], k: int) -> List[Tuple[int, int]]
```

Example

```
arr1 = [1, 7, 11] arr2 = [2, 4, 6] k = 3
```

All possible pairs and their sums are:

- (1, 2) -> 3 - (1, 4) -> 5 - (1, 6) -> 7 - (7, 2) -> 9 - (7, 4) -> 11 - (7, 6) -> 13 - (11, 2) -> 13 - (11, 4) -> 15 - (11, 6) -> 17
  The 3 pairs with the smallest sums are:

```
[(1, 2), (1, 4), (1, 6)]
```

(The order among these returned pairs does not matter.) Explain an efficient algorithm to solve this problem and analyze its time and space complexity. You may assume the arrays are large, so a solution that explicitly generates and sorts all **`len(arr1) * len(arr2)`** pairs is too slow.

EXAMPLES

Example 1	
Input	`[1, 7, 11], [2, 4, 6], 3`
Output	`[(1, 2), (1, 4), (1, 6)]`
Why	`Pairs (one from each array) in order of their sum.`

QUESTION

Example 2	
Input	`[1, 1, 2], [1, 2, 3], 2`
Output	`[(1, 1), (1, 1)]`
Why	`Equal numbers give equal pairs.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a, b, k):
    pairs = [(x, y) for x in a for y in b]
    pairs.sort(key=lambda p: p[0] + p[1])
    return pairs[:k]
```

TIME	SPACE
O(m * n log(m * n))	O(m * n)

WHY IT IS SLOW

m * n pairs sorted: with 10^5 elements in each array that is 10^10 pairs. Because both arrays are sorted, a min-heap that starts from each x's first pair and expands only the next pair in that row is O(k log k).

OPTIMAL SOLUTION · Min-heap over the sorted rows

TIME	SPACE
O(k log min(k, m))	O(min(k, m))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The arrays are not sorted.

Sort both first (O(m log m + n log n)); the heap method needs sorted rows. For one huge array you may instead use a heap of size k with a selection of candidates.

FOLLOW-UP 2 Only the k-th smallest sum, with huge arrays.

Binary search the sum value and count how many pairs have sum at most it with two pointers (O(m + n) per check): O((m + n) log(range)) with O(1) space, no heap.

FOLLOW-UP 3 Three arrays, or k pairs with the largest sums.

For largest sums run the same algorithm on the reversed arrays (or negate the values). For three arrays use a heap of triples (i, j, l), pushing the three neighbours with a visited set. **Cost:** O(k log k) with the heap of triples; reversing the arrays gives the largest sums at the same cost.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```python
# MAIN QUESTION: Find k pairs with smallest sums
#
# You are given two **sorted** integer arrays arr1 and
# arr2 (each sorted in non-decreasing order), and an
# integer k. Consider all possible pairs formed by taking
# one element from arr1 and one element from arr2. For
# indices i and j, the pair is (arr1[i], arr2[j]) with sum
# arr1[i] + arr2[j]. Your task is to return the **k
# pairs** (arr1[i], arr2[j]) with the smallest sums. If
# there are fewer than k total possible pairs, return all
# possible pairs. You may return the pairs in any order.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The arrays are not sorted.
# 2. Only the k-th smallest sum, with huge arrays.
# 3. Three arrays, or k pairs with the largest sums.
#
# INTERVIEW NOTES
# Because both arrays are sorted, the pair (i, j + 1) is
# never smaller than (i, j), so each row i of the 'matrix
# of sums' is sorted. The k smallest pairs are the k
# smallest items of a collection of sorted rows, which is
# a heap problem: start with the first element of each row
# (or the first few) and each time you pop a pair, push
# its successor in the row.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# a and b are sorted. Find the k pairs (x from a, y from b) with the
# smallest sums x + y.
# Idea: think of a table with a[i] + b[j]; every row i is sorted. A
# min-heap holds the best UNUSED pair of every row. Start with the first
# pair of each row (only the first k rows can matter). Take the smallest
# out, record the pair, and offer the next pair of that row, (i, j + 1).
# After k pops you have the k smallest sums.
# Complexity: Time O(k log k), Space O(k)
def k_smallest_pairs(a, b, k):
    if not a or not b or k <= 0:
        return []
    heap = []
    for i in range(min(k, len(a))):
        heap.append((a[i] + b[0], i, 0))
    heapq.heapify(heap)
    pairs = []
    while heap and len(pairs) < k:
        total, i, j = heapq.heappop(heap)
        pairs.append((a[i], b[j]))
        if j + 1 < len(b):
            heapq.heappush(heap, (a[i] + b[j + 1], i, j + 1))
    return pairs


# ---- main question: tests ----
```

```
def test_main_question():
    a = [1, 7, 11]
    b = [2, 4, 6]
    assert k_smallest_pairs(a, b, 3) == [(1, 2), (1, 4), (1, 6)]
    assert k_smallest_pairs(a, b, 4)[3] == (7, 2)
    assert k_smallest_pairs([1, 1, 2], [1, 2, 3], 2) == [(1, 1), (1, 1)]
    assert k_smallest_pairs([], [1], 3) == []
    assert len(k_smallest_pairs([1], [1, 2], 10)) == 2     # k bigger than the number of pairs: all of them


# ============================================================
# FOLLOW-UP 1: The arrays are not sorted
# ============================================================
# ---- follow-up 1: function ----
# Answer: sort both first (O(m log m + n log n)): the heap method needs
# sorted rows.
# Complexity: Time O(m log m + n log n + k log k), Space O(m + n + k)
def k_smallest_pairs_unsorted(a, b, k):
    return k_smallest_pairs(sorted(a), sorted(b), k)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert k_smallest_pairs_unsorted([7, 1], [4, 2], 2) == [(1, 2), (1, 4)]
```

```
# ============================================================
# FOLLOW-UP 2: Only the k-th smallest sum, with huge arrays
# ============================================================
# ---- follow-up 2: helper functions ----
def count_at_most(a, b, limit):
    # how many pairs have a sum <= limit? Two pointers: as x grows, fewer
    # values of b fit, so the pointer into b only moves down.
    count = 0
    j = len(b) - 1
    for x in a:
        while j >= 0 and x + b[j] > limit:
            j -= 1
        count += j + 1
    return count


# ---- follow-up 2: function ----
# Answer: binary search the SUM itself. For a guessed sum, count how many
# pairs are at most that big (O(m + n) with two pointers); the k-th smallest
# is the smallest sum whose count is at least k. O((m + n) log range)
# time and O(1) space: no heap.
# Complexity: Time O((m + n) log(range of sums)), Space O(1)
def kth_smallest_sum(a, b, k):
    low = a[0] + b[0]
    high = a[len(a) - 1] + b[len(b) - 1]
    while low < high:
        middle = (low + high) // 2
        if count_at_most(a, b, middle) >= k:
            high = middle
        else:
            low = middle + 1
    return low


# ---- follow-up 2: tests ----
def test_follow_up_2():
    a = [1, 7, 11]
    b = [2, 4, 6]
    sums = []
```

```
    for x in a:
        for y in b:
            sums.append(x + y)
    sums.sort()
    for k in range(1, 10):
        assert kth_smallest_sum(a, b, k) == sums[k - 1]    # the 4th smallest sum is 9 (7 + 2)


# ============================================================
# FOLLOW-UP 3: Three arrays, or k pairs with the largest sums
# ============================================================
# ---- follow-up 3: function ----
# Answer: for the LARGEST sums run the same code on the negated values
# (the biggest numbers become the smallest). For three arrays use a heap
# of triples (i, j, l) and push the three neighbours of the popped one,
# with a set of visited triples.
# Complexity: Time O(m log m + n log n + k log k), Space O(m + n + k)
def k_largest_pairs(a, b, k):
    negated_a = []
    for x in sorted(a, reverse=True):
        negated_a.append(-x)
    negated_b = []
    for y in sorted(b, reverse=True):
        negated_b.append(-y)
    pairs = []
    for x, y in k_smallest_pairs(negated_a, negated_b, k):
        pairs.append((-x, -y))
    return pairs


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert k_largest_pairs([1, 7, 11], [2, 4, 6], 2) == [(11, 6), (11, 4)]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 19 OF 22

Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up | 1 questions | tap a title to jump to it

READ FIRST: the code template for these 1 questions (tap here) >

E 35. Next Birthday (BST In-Order Successor with Wraparound) **ColorStack 2025-10-17**

CODE TEMPLATE | GROUP 19 OF 22 | 1 QUESTIONS, ONE PATTERN

Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

The one idea these questions share

Same recursive walk as Tier 1, but now the information also travels down: lower and upper bounds for a binary search tree, or the target nodes you are looking for. The BST rule (left < node < right) lets you go left or right instead of visiting everything.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def check(node, lo=float("-inf"), hi=float("inf")):
    if node is None:
        return True                          # CHANGE 1: base case
    if not lo < node.val < hi:               # CHANGE 2: the rule this node must obey
        return False
    return check(node.left, lo, node.val) and check(node.right, node.val, hi)


def descend(root, x):                        # BST search: one path, O(height)
    best, node = None, root
    while node:
        if node.val > x:                     # CHANGE 3: go left or right by the rule
            best, node = node.val, node.left
        else:
            node = node.right
    return best
```

2. Helper functions that usually get written next to it

```
def inorder(root):                           # left, node, right: sorted order for a BST
    stack, node = [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        yield node.val
        node = node.right


def leftmost(root):                          # the smallest key (the wrap-around answer)
    while root.left:
        root = root.left
    return root.val
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
35	Next Birthday (BST In-Order Successor with Wraparound)	descend(): go left after a hit (a closer candidate may exist), right otherwise; if no candidate exists wrap around to the leftmost node

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 35 of 38 | topic 19 of 22: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up | 1 of 1

Template fit: descend(): go left after a hit (a closer candidate may exist), right otherwise; if no candidate exists wrap around to the leftmost node | asked 368 days ago | frequency Single report

EASY | IN APP LC 285 | ColorStack | technical interview | 35 of 38

Next Birthday (BST In-Order Successor with Wraparound)

ASKED 2025-10-17 | 368 DAYS AGO | OLDER

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 240/288**

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

Optimal answer from: your app's interview approach (LeetCode #285)

LeetCode match: LC 285 Inorder Successor in BST

Frequency: Single report | Last asked: 2025-10-17 | Roles: SWE Intern

QUESTION

This is the "alarm / next-birthday" problem: from a set of birthdays stored in a binary search tree, find the next one that will occur. Each birthday is stored as an integer day-of-year in [1, 365] , and the tree is a valid BST keyed on that value. Given the root of the BST and an integer today , return the next upcoming birthday: the smallest key that is strictly greater than today . If no key is strictly greater than today , the calendar wraps around, so return the smallest key in the whole tree. If the tree is empty, return None .

Function signature

```
from typing import Optional
class TreeNode:
    def __init__(self, val: int, left: "Optional[TreeNode]" = None,
                 right: "Optional[TreeNode]" = None) -> None:
        self.val = val
        self.left = left
        self.right = right
def next_birthday(root: Optional[TreeNode], today: int) -> Optional[int]:
    """Return the smallest key strictly greater than today, wrapping around
    to the smallest key if none is greater. Return None if the tree is empty."""
    ...
```

Follow-up (as reported): If the input were an unsorted binary tree (not a BST), extract all values, sort them, and binary-search for the successor. Hint: bisect_right .

EXAMPLES

Example 1	
Input	`root = above, today = 25`
Output	`30`
Why	`Smallest key strictly greater than 25 is 30.`

Example 2	
Input	`root = above, today = 40`
Output	`10`
Why	`No key > 40, so wrap around to the smallest key, 10.`

QUESTION

Example 3	
Input	`root = above, today = 5`
Output	`10`
Why	`Smallest key greater than 5 is 10.`

Example 4	
Input	`root = None, today = 100`
Output	`None`

CONSTRAINTS

- 0 <= number of nodes <= 10^5 ; all node values are distinct and in [1, 365] .
- 1 <= today <= 365 .
- Aim for O(h) time where h is the tree height (do not sort all nodes if you can avoid it).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_next_birthday(root, today):
    keys = []
    def inorder(node):
        if node:
            inorder(node.left)
            keys.append(node.val)
            inorder(node.right)
    inorder(root)
    if not keys:
        return None
    return next(
        (k for k in keys if k > today), keys[0]
    )
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

It visits every node although the BST order points to the answer.

OPTIMAL SOLUTION · Descend the BST once

TIME	SPACE
O(h)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is not a BST. What now?

Collect every value, sort once, then binary-search with bisect_right for the first value above today and wrap to index 0. O(n log n) once, O(log n) per query.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Next Birthday (BST In-Order Successor
# with Wraparound)
#
# This is the "alarm / next-birthday" problem: from a set
# of birthdays stored in a binary search tree, find the
# next one that will occur. Each birthday is stored as an
# integer day-of-year in [1, 365] , and the tree is a
# valid BST keyed on that value. Given the root of the BST
# and an integer today , return the next upcoming
# birthday: the smallest key that is strictly greater than
# today . If no key is strictly greater than today , the
# calendar wraps around, so return the smallest key in the
# whole tree. If the tree is empty, return None .
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is not a BST. What now?
#
# INTERVIEW NOTES
# Walk down the BST once. At a node whose value is above
# today it is a candidate, so remember it and go left to
# look for a smaller candidate; otherwise go right. The
# last candidate is the successor. If there is none, the
# answer wraps to the leftmost node.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ---- main question: main function ----
# Idea: the birthdays are in a binary SEARCH tree. Walk down once: a
# value above today is a candidate (the answer might be even closer in
```

```
# its left subtree), a value at or below today sends us right. If no
# value is above today, wrap around to the smallest value (the leftmost node).
# Complexity: Time O(h) for the height h of the tree, Space O(1)
def next_birthday(root, today):
    if root is None:
        return None
    best = None
    node = root
    while node:
        if node.val > today:
            best = node.val
            node = node.left
        else:
```

```
            node = node.right
    if best is not None:
        return best
    node = root
    while node.left:
        node = node.left
    return node.val


# ---- main question: tests ----
def example_tree():
    return TreeNode(20, TreeNode(10), TreeNode(30, None, TreeNode(40)))


def test_main_question():
    root = example_tree()
    assert next_birthday(root, 25) == 30
    assert next_birthday(root, 40) == 10                    # nothing later: wrap around
    assert next_birthday(root, 5) == 10
    assert next_birthday(None, 100) is None


# ========================================================
# FOLLOW-UP 1: The tree is not a BST. What now?
# ========================================================
# ---- follow-up 1: function ----
# Answer: the ordering can no longer be used, so look at every value:
# the best one is the smallest value above today, or the smallest value
# overall if there is none. O(n) per question. For many questions on the
# same tree, sort the values once and binary-search (bisect) for the
# first value above today: O(log n) per question.
# Complexity: Time O(n), Space O(n)
```

```
def next_birthday_unsorted(root, today):
    if root is None:
        return None
    smallest = None
    next_one = None
    stack = [root]
    while stack:
        node = stack.pop()
        if smallest is None or node.val < smallest:
            smallest = node.val
        if node.val > today and (next_one is None or node.val < next_one):
            next_one = node.val
        for child in (node.left, node.right):
            if child:
                stack.append(child)
    if next_one is not None:
        return next_one
    return smallest


# ---- follow-up 1: tests ----
def test_follow_up_1():
    messy = TreeNode(30, TreeNode(40), TreeNode(10, TreeNode(20)))     # not in search-tree order
    assert next_birthday_unsorted(messy, 25) == 30
    assert next_birthday_unsorted(messy, 40) == 10
    assert next_birthday_unsorted(example_tree(), 25) == next_birthday(example_tree(), 25)
    assert next_birthday_unsorted(None, 1) is None


# ========================================================
# RUN EVERYTHING
# ========================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 20 OF 22

Parent pointers: store who is above each node and climb

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Parent pointers: store who is above each node and climb | 1 questions | tap a title to jump to it

READ FIRST: the code template for these 1 questions (tap here) >

E 36. Tree Node. **Exponent no date**

CODE TEMPLATE | GROUP 20 OF 22 | 1 QUESTIONS, ONE PATTERN

Parent pointers: store who is above each node and climb

The one idea these questions share

Keep parent[x] (or manager[x]). Climbing is a while loop that follows parent until it runs out. Questions about common ancestors, cycles and who reports to whom are all small variations of that climb.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def climb(x, parent):
    while x is not None:
        yield x
        x = parent.get(x)              # CHANGE 1: stop rule (no parent, or a repeat = cycle)


def lowest_common(a, b, parent):
    above = set(climb(a, parent))
    for x in climb(b, parent):         # CHANGE 2: what you look for while climbing
        if x in above:
            return x
    return None
```

2. Helper functions that usually get written next to it

```
def depth(x, parent):
    return sum(1 for _ in climb(x, parent)) - 1


def has_cycle(parent):
    for start in parent:
        seen, x = set(), start
        while x is not None:
            if x in seen:
                return True
            seen.add(x)
            x = parent.get(x)
    return False
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
36	Tree Node.	the SQL version of the same idea: p_id is the parent pointer, NULL means root, being someone's parent means inner, else leaf

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 36 of 38 | topic 20 of 22: Parent pointers: store who is above each node and climb | 1 of 1

Template fit: the SQL version of the same idea: p_id is the parent pointer, NULL means root, being someone's parent means inner, else leaf | no date

EASY | Exponent | 36 of 38

Tree Node.

ASKED (ANY CO.) ~5 MONTHS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 278/288**

ABOUT THIS QUESTION

Topic: Parent pointers: store who is above each node and climb

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match (tree node classification)

Last asked (any company): 5 months ago, about 2026-05 (Exponent shows only a rough age) | +38 interviews

Original: https://www.aced.io/questions/3957/tree-node

QUESTION

A binary tree is stored in a table tree_node_table(id, p_id), where p_id is the id of the parent node (NULL for the root). Write a query that labels every node: 'Root' if it has no parent, 'Inner' if it has a parent and at least one child, and 'Leaf' if it has a parent and no children. The output has the columns id and node_types, ordered by id.

Function signature

```
-- columns: id, node_types
```

EXAMPLES

Example 1

Input	`CREATE TABLE tree_node_table (id INTEGER, p_id INTEGER);` `INSERT INTO tree_node_table VALUES (1, NULL), (2, 1), (3, 1), (4, 2), (5, 2);`

Output	`1, Root`
	`2, Inner`
	`3, Leaf`
	`4, Leaf`
	`5, Leaf`

Why	`Node 1 has no parent, node 2 has a parent and children 4 and 5, and nodes 3, 4 and 5 have no children.`

Example 2

Input	`CREATE TABLE tree_node_table (id INTEGER, p_id INTEGER);` `INSERT INTO tree_node_table VALUES (1, NULL);`
Output	`1, Root`
Why	`A tree with one node: it has no parent, so it is the Root (not a Leaf).`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
SELECT id, 'Root' AS node_types
FROM tree_node_table WHERE p_id IS NULL
UNION
SELECT id, 'Inner'
FROM tree_node_table t
WHERE p_id IS NOT NULL
  AND EXISTS (SELECT 1 FROM tree_node_table c
              WHERE c.p_id = t.id)
UNION
SELECT id, 'Leaf'
FROM tree_node_table t
WHERE p_id IS NOT NULL
  AND NOT EXISTS (SELECT 1 FROM tree_node_table c
                  WHERE c.p_id = t.id);
```

TIME	SPACE
O(n^2) without an index on p_id	**O(n)**

WHY IT IS SLOW

The correlated subquery runs once per row and the table is scanned three times, O(n^2) without an index. One CASE expression with a single set of parent ids scans once.

OPTIMAL SOLUTION · One CASE expression

TIME	SPACE
O(n) with the set of parent ids built once	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why not use NOT IN for the leaves?

If the subquery returns a NULL (the root's p_id), id NOT IN (...) is never true, so you get no rows. Filter NULLs out of the subquery or use NOT EXISTS.

FOLLOW-UP 2 The table has no row for the root's parent and p_id can point to missing ids.

Validate with a LEFT JOIN to find orphans. A node whose p_id is not NULL but missing from the table is a data error, not a root; decide whether to report it.

FOLLOW-UP 3 Add the depth of each node.

A recursive CTE starting from the root rows and joining children on p_id, carrying depth + 1, gives every node's depth in one query.

FOLLOW-UP 4 Add the depth of each node.

A recursive CTE starts from the root rows (depth 0) and joins the table to itself on p_id, adding 1 at every step; each node appears once because every node has one parent. Runs in one query, O(n) with an index on p_id.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Tree Node.
#
# A binary tree is stored in a table tree_node_table(id,
# p_id), where p_id is the id of the parent node (NULL for
# the root). Write a query that labels every node: 'Root'
# if it has no parent, 'Inner' if it has a parent and at
# least one child, and 'Leaf' if it has a parent and no
# children. The output has the columns id and node_types,
# ordered by id.
#
# Signature:
#   -- columns: id, node_types
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why not use NOT IN for the leaves?
# 2. The table has no row for the root's parent and p_id
# can point to missing ids.
# 3. Add the depth of each node.
# 4. Add the depth of each node.
#
# INTERVIEW NOTES
# Each row is a node with its parent id. A node is the
# root if it has no parent, a leaf if nobody lists it as a
# parent, and inner otherwise. Order the tests so the root
# is decided first (a single-node tree is a root, not a
# leaf). Beware NOT IN with NULLs: it returns nothing if
# the subquery contains a NULL.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import sqlite3


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# The table has one row per node; the root has p_id NULL.
SCHEMA = "CREATE TABLE tree_node_table (id INT, p_id INT)"
INSERT = "INSERT INTO tree_node_table VALUES (?, ?)"
```

```


# Complexity: Time O(n^2) for the correlated EXISTS check without an index (O(n log n) with an index on p_i
d), Space O(n)
def run(rows, sql):
    # run one query on a fresh in-memory database holding `rows`
    db = sqlite3.connect(":memory:")
    db.execute(SCHEMA)
    db.executemany(INSERT, rows)
    result = db.execute(sql).fetchall()
    db.close()
    return result
```

```
# ---- main question: main function ----
# Idea: one CASE expression, tested in order.
#   Root:  no parent.
#   Inner: has a parent AND somebody lists it as their parent.
#   Leaf:  has a parent and nobody lists it as their parent.
# The Root test comes first, so a one-node tree is a Root, not a Leaf.
TYPES_SQL = """
SELECT t.id,
  CASE
    WHEN t.p_id IS NULL THEN 'Root'
    WHEN EXISTS (SELECT 1 FROM tree_node_table c
                 WHERE c.p_id = t.id) THEN 'Inner'
    ELSE 'Leaf'
  END AS node_types
FROM tree_node_table t
ORDER BY t.id
"""


# ---- main question: tests ----
#       1
#      / \
#     2   3
#    / \
#   4   5
ROWS = [(1, None), (2, 1), (3, 1), (4, 2), (5, 2)]


# Complexity: Time O(n) with the set of parent ids built once, Space O(n)
def test_main_question():
    expected = [(1, "Root"), (2, "Inner"), (3, "Leaf"), (4, "Leaf"), (5, "Leaf")]
    assert run(ROWS, TYPES_SQL) == expected
```

```
    assert run([(7, None)], TYPES_SQL) == [(7, "Root")]      # a single node is the Root


# ========================================================
# FOLLOW-UP 1: Why not use NOT IN for the leaves?
# ========================================================
# ---- follow-up 1: function ----
# Answer: the list inside NOT IN contains the root's NULL p_id, and
# "id NOT IN (..., NULL)" is never true, so you get NO rows. Filter the
# NULL out of the list, or use NOT EXISTS (below).
NOT_IN_SQL = """
SELECT id FROM tree_node_table
WHERE id NOT IN (SELECT p_id FROM tree_node_table)
"""

LEAVES_SQL = """
SELECT t.id FROM tree_node_table t
WHERE NOT EXISTS (SELECT 1 FROM tree_node_table c
                  WHERE c.p_id = t.id)
ORDER BY t.id
"""


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert run(ROWS, NOT_IN_SQL) == []                       # the NULL ruins NOT IN
    assert run(ROWS, LEAVES_SQL) == [(3,), (4,), (5,)]


# ========================================================
# FOLLOW-UP 2: The table has no row for the root's parent and p_id can point to missing ids
```

```
# ============================================================
# ---- follow-up 2: function ----
# Answer: look for orphans with a LEFT JOIN. A node whose p_id is not
# NULL but is missing from the table is a data error, not a root;
# decide whether to report it.
ORPHANS_SQL = """
SELECT t.id FROM tree_node_table t
LEFT JOIN tree_node_table p ON p.id = t.p_id
WHERE t.p_id IS NOT NULL AND p.id IS NULL
ORDER BY t.id
"""


# ---- follow-up 2: tests ----
```

```
def test_follow_up_2():
    assert run(ROWS + [(9, 42)], ORPHANS_SQL) == [(9,)]      # node 9 points at a parent that does not exis
t
    assert run(ROWS, ORPHANS_SQL) == []


# ============================================================
# FOLLOW-UP 3: Add the depth of each node
# ============================================================


# ============================================================
# FOLLOW-UP 4: Add the depth of each node
# ============================================================
# ---- follow-up 3: function ----
# Answer: a recursive query. Start from the roots (depth 0), then
# repeatedly join the table to the nodes found so far on p_id, adding 1.
# Every node appears once because it has one parent. One query, O(n)
# with an index on p_id.
DEPTH_SQL = """
WITH RECURSIVE walk(id, depth) AS (
  SELECT id, 0 FROM tree_node_table WHERE p_id IS NULL
  UNION ALL
  SELECT t.id, w.depth + 1
  FROM tree_node_table t JOIN walk w ON t.p_id = w.id
)
SELECT id, depth FROM walk ORDER BY id
"""


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert run(ROWS, DEPTH_SQL) == [(1, 0), (2, 1), (3, 1), (4, 2), (5, 2)]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given a binary tree, identify which nodes are leaf nodes, root nodes or inner nodes. Leaf nodes: A node with no children Root nodes: A node with no parent Inner nodes: A node with both children and parent nodes You are given the following table tree_node_table: idp_idintegerinteger where p_id is the id of the parent node. Your output should contain the following columns: id, node_types (’Root’, ‘Inner’, ‘Leaf’).No answers contributed by the community yet.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 21 OF 22

Build or transform a grid with index arithmetic

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Build or transform a grid with index arithmetic | 1 questions | tap a title to jump to it

READ FIRST: the code template for these 1 questions (tap here) >

E 37. Filled ASCII Diamond **ColorStack 2025-09-26**

CODE TEMPLATE | GROUP 21 OF 22 | 1 QUESTIONS, ONE PATTERN

Build or transform a grid with index arithmetic

The one idea these questions share

Decide where each output cell comes from with a formula in (r, c), then build the whole grid with one comprehension. Rotation, diamonds, block summaries and prefix tables are all 'cell(r, c) = something computed from r and c'.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def build_grid(rows, cols, cell):
    return [[cell(r, c) for c in range(cols)] for r in range(rows)]    # CHANGE: the formula for one cell
```

2. Helper functions that usually get written next to it

```
def prefix_2d(matrix):
    rows, cols = len(matrix), len(matrix[0])
    p = [[0] * (cols + 1) for _ in range(rows + 1)]
    for r in range(rows):
        for c in range(cols):
            p[r + 1][c + 1] = matrix[r][c] + p[r][c + 1] + p[r + 1][c] - p[r][c]
    return p


def snake_cells(rows, cols):
    return [(r, c if r % 2 == 0 else cols - 1 - c) for r in range(rows) for c in range(cols)]
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
37	Filled ASCII Diamond	row i has pad = |i - R| underscores on each side and 2 * (R - pad) + 1 x's in the middle

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 37 of 38 | topic 21 of 22: Build or transform a grid with index arithmetic | 1 of 1

Template fit: row i has pad = |i - R| underscores on each side and 2 * (R - pad) + 1 x's in the middle | asked 389 days ago | frequency Single report

EASY | ColorStack | technical interview | 37 of 38

Filled ASCII Diamond

ASKED 2025-09-26 | 389 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 235/288

ABOUT THIS QUESTION

Topic: Build or transform a grid with index arithmetic

Optimal answer from: written and tested here for this exact question

LeetCode match: Simulation (print a diamond)

Frequency: Single report | Last asked: 2025-09-26 | Roles: SWE Intern

QUESTION

Given a non-negative integer radius R , render a filled diamond on a square grid of side 2*R + 1 . The grid is indexed by (i, j) with 0 <= i, j <= 2*R , and its center is (R, R) . A cell belongs to the diamond (draw 'x') if its Manhattan distance from the center is at most R , i.e. abs(i - R) + abs(j - R) <= R . Every other cell is background (draw '_'). Return the picture as a list of 2*R + 1 strings, one per row (top to bottom), each of length 2*R + 1 .

Function signature

```
def render_diamond(R: int) -> list[str]:
    ...
```

Follow-up (as reported): The naive fill is O(R^2). Can you produce each row directly from its width instead of testing every cell? (Row i is 'x' repeated 2*(R - abs(i-R)) + 1 times, padded with '_' .)

EXAMPLES

Example 1	
Input	`R = 1`
Output	`["_x_", "xxx", "_x_"]`
Why	`Center (1,1). Row 0 has distance 1 from center vertically, so only column 1 (distance 0) is inside.`

Example 2	
Input	`R = 2`
Output	`["__x__", "_xxx_", "xxxxx", "_xxx_", "__x__"]`
Why	`Each row i allows columns within (R - abs(i-R)) of the center.`

Example 3	
Input	`R = 0`
Output	`["x"]`
Why	`A 1x1 grid whose only cell is the center.`

CONSTRAINTS

- 0 <= R <= 500 .
- For R = 0 the result is the single cell ["x"] .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_diamond(R):
    return [
        "".join(
            (
                "x"
                if abs(i - R) + abs(j - R) <= R
                else:
                    "_"
            )
            for j in range(2 * R + 1)
        )
        for i in range(2 * R + 1)
    ]
```

TIME	SPACE
O(R^2)	O(R^2)

WHY IT IS SLOW

Each of the (2R + 1)^2 cells runs the distance test.

OPTIMAL SOLUTION · Build each row from its width

TIME	SPACE
O(R^2) characters	O(R^2)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Draw only the outline of the diamond.

Keep the same loop but put 'x' only where the distance is exactly R; inside cells become background. In the row form: the two end cells of each row are 'x' (one cell at the tips).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Filled ASCII Diamond
#
# Given a non-negative integer radius R , render a filled
# diamond on a square grid of side 2*R + 1 . The grid is
# indexed by (i, j) with 0 <= i, j <= 2*R , and its center
# is (R, R) . A cell belongs to the diamond (draw 'x' ) if
# its Manhattan distance from the center is at most R ,
# i.e. abs(i - R) + abs(j - R) <= R . Every other cell is
# background (draw '_' ). Return the picture as a list of
# 2*R + 1 strings, one per row (top to bottom), each of
# length 2*R + 1 . Function signature: def
# render_diamond(R: int) -> list[str]: ... Follow-up (as
# reported): The naive fill is O(R^2). Can you produce
# each row directly from its width instead of testing
# every cell? (Row i is 'x' repeated 2*(R - abs(i-R)) + 1
# times, padded with '_' .)
```

```
#
# FOLLOW-UPS (answered below, in this order):
# 1. Draw only the outline of the diamond.
#
# INTERVIEW NOTES
# A cell is inside the diamond when its Manhattan distance
# from the centre is at most R. Testing every cell is
# O(R^2) work for an O(R^2) picture, which is already
# optimal, but each row can be written directly: row i has
# width 2 * (R - |i - R|) + 1 of 'x' centred in a row of
# underscores.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: the diamond has 2 * R + 1 rows. Row i is `pad` = |i - R| steps away
# from the middle row, so it has `pad` underscores on each side and
# 2 * (R - pad) + 1 letters "x" in the middle.
# Complexity: Time O(R^2), Space O(R^2) for the output
def render_diamond(R):
    rows = []
    for i in range(2 * R + 1):
        pad = i - R
        if pad < 0:
            pad = -pad                                  # the distance from the middle row
        rows.append("_" * pad + "x" * (2 * (R - pad) + 1) + "_" * pad)
    return rows


# ---- main question: tests ----
def test_main_question():
    assert render_diamond(1) == ["_x_", "xxx", "_x_"]
    assert render_diamond(2) == ["__x__", "_xxx_", "xxxxx", "_xxx_", "__x__"]
    assert render_diamond(0) == ["x"]


# ========================================================
# FOLLOW-UP 1: Draw only the outline of the diamond
# ========================================================
# ---- follow-up 1: function ----
# Answer: keep the same loop but put "x" only at the two ends of the
# letter stretch of each row (positions pad and 2R - pad); everything
# inside becomes background. At the tips both ends are the same cell.
# Complexity: Time O(R^2), Space O(R^2) for the output
def render_hollow_diamond(R):
    rows = []
    for i in range(2 * R + 1):
        pad = i - R
        if pad < 0:
            pad = -pad
        row = ["_"] * (2 * R + 1)
        row[pad] = "x"
        row[2 * R - pad] = "x"
        rows.append("".join(row))
    return rows
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert render_hollow_diamond(2) == ["__x__", "_x_x_", "x___x", "_x_x_", "__x__"]
    assert render_hollow_diamond(0) == ["x"]
    assert render_hollow_diamond(1) == ["_x_", "x_x", "_x_"]


# =====================================================
# RUN EVERYTHING
```

```
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 22 OF 22

Split into pieces, fix each piece, join (and pack words into lines)

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
EASY

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Split into pieces, fix each piece, join (and pack words into lines) | 1 questions | tap a title to jump to it

READ FIRST: the code template for these 1 questions (tap here) >

E 38. Implement string formatting and request tracking tasks **PracHub 2026-02-11**

CODE TEMPLATE | GROUP 22 OF 22 | 1 QUESTIONS, ONE PATTERN

Split into pieces, fix each piece, join (and pack words into lines)

The one idea these questions share

Parsing questions split the text on a separator, transform every piece with one small rule, and join again. Layout questions go word by word and decide whether the next word still fits on the current line.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def reformat(text, sep="_"):
    head, *rest = text.split(sep)
    return head + "".join(p[:1].upper() + p[1:] for p in rest)   # CHANGE: the rule applied to each piece


def pack_lines(words, width):
    lines, cur = [], []
    for w in words:
        if cur and len(" ".join(cur + [w])) > width:             # does the next word still fit?
            lines.append(cur)
            cur = []
        cur.append(w)
    return lines + ([cur] if cur else [])
```

2. Helper functions that usually get written next to it

```
def spread_spaces(line_words, width):           # distribute spaces between words, extra ones on the left
    gaps = len(line_words) - 1
    if gaps == 0:
        return line_words[0].ljust(width)
    spaces, extra = divmod(width - sum(map(len, line_words)), gaps)
    out = ""
    for k, w in enumerate(line_words[:-1]):
        out += w + " " * (spaces + (1 if k < extra else 0))
    return out + line_words[-1]
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
38	Implement string formatting and request tracking tasks	split, fix each piece, join; the request-tracking part is a small dict

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 38 of 38 | topic 22 of 22: Split into pieces, fix each piece, join (and pack words into lines) | 1 of 1

Template fit: split, fix each piece, join; the request-tracking part is a small dict | asked 251 days ago

EASY | PracHub | Technical Screen | 38 of 38

Implement string formatting and request tracking tasks

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 180/288

ABOUT THIS QUESTION

Topic: Split into pieces, fix each piece, join (and pack words into lines)

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

LeetCode match: LC 246 Strobogrammatic Number (+ formatting and tracking subtasks) (inferred from overview)

Posted on PracHub: 2026-02-11 (updated 2026-05-21)

Original: https://prachub.com/interview-questions/implement-string-formatting-and-request-tracking-tasks

QUESTION

Four small tasks. (A) snake_to_camel(s): convert snake_case to camelCase, ignoring leading, trailing and repeated underscores. (B) format_thousands(n): write an integer with a comma between every group of three digits (keep the minus sign). (C) ocr_valid(t, rot): an OCR scanner may read a label upside down; given the rotation table rot (character -> character seen after turning 180 degrees), return true if t, or t turned 180 degrees (reversed and mapped through rot), is an integer from 1 to 200 without leading zeros. (D) Server: begin(rid, ts) and end(rid, ts) arrive in any order for each request id; drain(watermark) returns the completed requests as 'id begin=b end=e' lines in order of end time, only those with end at most the watermark (all of them when no watermark is given).

Function signature

```
def snake_to_camel(s: str) -> str
def format_thousands(n: int) -> str
def ocr_valid(t: str, rot: dict) -> bool
class Server:
    def begin(self, rid, ts) -> None
    def end(self, rid, ts) -> None
    def drain(self, watermark=None) -> list
```

EXAMPLES

Example 1	
Input	`'make__it_camel_'`
Output	`'makeItCamel'`
Why	`Repeated and trailing underscores are ignored.`

Example 2	
Input	`1234567`
Output	`'1,234,567'`
Why	`A comma before every third digit from the right.`

QUESTION

Example 3	
Input	`-1000`
Output	`'-1,000'`
Why	`The sign stays at the front.`

Example 4	

Input	`rot = {'0': '0', '1': '1', '2': '2', '5': '5', '6': '9', '8': '8', '9': '6'}`
	`ocr_valid('186', rot)`
	`ocr_valid('981', rot)`
	`ocr_valid('500', rot)`

Output	`ocr_valid('186', rot) -> true`
	`ocr_valid('981', rot) -> true`
	`ocr_valid('500', rot) -> false`

Why	`981 is above 200, but turned upside down it reads 186, which is valid. 500 is too large and its rotation 005 has leading zeros.`

Example 5	

Input	`sv = Server()`
	`sv.begin('r2', 5)`
	`sv.end('r1', 4)`
	`sv.begin('r1', 1)`
	`sv.end('r2', 9)`
	`sv.drain(8)`
	`sv.drain()`

Output	`sv.drain(8) -> ['r1 begin=1 end=4']` `sv.drain() -> ['r2 begin=5 end=9']`
Why	`r1 finished at 4 and r2 at 9. Draining with watermark 8 prints only r1; draining everything then prints r2.`

BRUTE FORCE · Task A by walking characters

BRUTE FORCE · PSEUDO-CODE

```
def to_camel(s):
    result, up = "", False
    for ch in s:
        if ch == "_":
            up = True
        else:
            result += ch.upper() if up else ch
            up = False
    return result
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear and fine; the split-based version is shorter. The only care is leading and repeated underscores.

BRUTE FORCE · Task B with the language's format specifier

BRUTE FORCE · PSEUDO-CODE

```
def format_number(n):
    return format(n, ",")  # built-in separator
```

TIME	SPACE
O(d)	O(d)

WHY IT IS SLOW

Correct and O(d), but it hides the algorithm; interviews usually ask for the grouping by hand: peel three digits at a time from the right.

BRUTE FORCE · Task C by checking all three readings with a straightforward helper

BRUTE FORCE · PSEUDO-CODE

```
def is_valid_rotation(text):
    if in_range(text):  # a number 1..200
        return True
    if not all(ch in ROTATE for ch in text):
        return False
    turned = [ROTATE[c] for c in reversed(text)]
    return in_range("".join(turned))
```

TIME	SPACE
O(L)	O(L)

WHY IT IS SLOW

It is already linear; there is little to speed up. The risk is correctness: unmapped characters make the rotation invalid, and leading zeros are not valid numbers.

BRUTE FORCE · Task D by storing all events and sorting when asked

BRUTE FORCE · PSEUDO-CODE

```
begin, end = {}, {}
def on_begin(rid, t):
    begin[rid] = t
def on_end(rid, t):
    end[rid] = t
def print_finished():
    done = [r for r in end if r in begin]
    done.sort(key=lambda r: (end[r], r))
    for r in done:
        print(r, begin[r], end[r])
        del begin[r], end[r]
```

TIME	SPACE
O(n log n) per print	O(n)

WHY IT IS SLOW

Each print sorts everything pending, O(n log n); a min-heap of completed requests prints in order with O(log n) per event, releasing only entries below a safe watermark.

OPTIMAL SOLUTION · Task A: snake_case to camelCase

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task B: thousands separators

TIME	SPACE
O(d)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task C: valid as-is or after a 180 degree rotation

TIME	SPACE
O(L)	O(L)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task D: request tracker that prints completed requests in end-time order

TIME	SPACE
O(log n) per event	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why not print as soon as a request completes in the follow-up?

Because another request that completes later can still have an earlier end time. An ordered printer needs a contract for when it is safe to emit: either a watermark (everything with end <= W is final because ends arrive within some known lateness), or a flush at the end of input. State that contract, then the heap pops only below the watermark.

FOLLOW-UP 2 How do you handle bad input (duplicate begin, end before begin, never-completed requests)?

Duplicate begins overwrite or are rejected by policy; an end timestamp earlier than its begin is flagged as invalid rather than printed. Requests that stay in the pending map are the leak to watch: evict by age with a timer or a second heap keyed on first-seen time, and report them as incomplete.

FOLLOW-UP 3 Rotation map provided at runtime with extra pairs.

Nothing changes: the map is a parameter. Validate that it is consistent (rot of rot is the original) if you control it, and treat characters that are not in it as invalid.

FOLLOW-UP 4 The tracker must bound its memory.

Evict requests whose begin or end has been waiting longer than a retention window (a second heap keyed by first-seen time) and report them as incomplete. **Cost:** O(log n) per event for the eviction heap; memory O(requests inside the retention window).

FOLLOW-UP 5 Another locale: a different separator or grouping (Indian style 12,34,567).

Parameterise the separator and the group size: the lowest group may differ from the others (3 digits, then groups of 2 in the Indian system). The loop peels a group from the right each time.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement string formatting and request
# tracking tasks
#
# Four small tasks. (A) snake_to_camel(s): convert
# snake_case to camelCase, ignoring leading, trailing and
# repeated underscores. (B) format_thousands(n): write an
# integer with a comma between every group of three digits
# (keep the minus sign). (C) ocr_valid(t, rot): an OCR
# scanner may read a label upside down; given the rotation
# table rot (character -> character seen after turning 180
# degrees), return true if t, or t turned 180 degrees
# (reversed and mapped through rot), is an integer from 1
# to 200 without leading zeros. (D) Server: begin(rid, ts)
# and end(rid, ts) arrive in any order for each request
# id; drain(watermark) returns the completed requests as
# 'id begin=b end=e' lines in order of end time, only
# those with end at most the watermark (all of them when
# no watermark is given).
#
# Signature:
#   def snake_to_camel(s: str) -> str
#   def format_thousands(n: int) -> str
#   def ocr_valid(t: str, rot: dict) -> bool
#   class Server:
#       def begin(self, rid, ts) -> None
#       def end(self, rid, ts) -> None
#       def drain(self, watermark=None) -> list
#
# Examples:
# 'make__it_camel_'  ->  'makeItCamel'
# 1234567  ->  '1,234,567'
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why not print as soon as a request completes in the
# follow-up?
# 2. How do you handle bad input (duplicate begin, end
# before begin, never-completed requests)?
# 3. Rotation map provided at runtime with extra pairs.
# 4. The tracker must bound its memory.
# 5. Another locale: a different separator or grouping
# (Indian style 12,34,567).
#
# INTERVIEW NOTES
# Four independent tasks, so solve and test each
# separately and say the contract for each before coding:
```

```
# what an empty or negative input returns, what 'valid'
# means for the rotated number, and for the tracker when
# it is safe to print in order. For the tracker,
# completion is the second event of a pair, and order
# printing is a heap that is drained only below a
# watermark.
```

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main functions ----
# TASK A: snake_case to camelCase, ignoring leading, trailing and
# repeated underscores.
# Idea: split on "_" and skip the empty parts (that handles every odd
# underscore in one move). The first word stays as it is; every later
# word gets a capital first letter.
# Complexity: Time O(n), Space O(n)
def snake_to_camel(s):
    words = []
    for part in s.split("_"):
        if part != "":
            words.append(part)
    if len(words) == 0:
        return ""
    result = words[0]
    for word in words[1:]:
        result += word.capitalize()
    return result


# TASK B: write an integer with a comma between every group of three
# digits (keep the minus sign).
# Idea: peel three digits at a time from the RIGHT end of the digit text.
# Complexity: Time O(d) for d digits, Space O(d)
def format_thousands(n):
```

```
    sign = ""
    if n < 0:
        sign = "-"
    digits = str(abs(n))
    groups = []
    while len(digits) > 3:
        groups.append(digits[-3:])
        digits = digits[:-3]
    groups.append(digits)
    groups.reverse()
    return sign + ",".join(groups)


# TASK C: an OCR scanner may read a label upside down. ocr_valid is True
# if the text, or the text turned 180 degrees (reversed, then every
# character mapped through the table `rot`), is an integer from 1 to
# 200 without leading zeros.
# Complexity: Time O(L), Space O(1)
def is_valid_number(text):
    # only digits, no leading zero, value 1 to 200
    if text == "" or text[0] == "0":
        return False
    for ch in text:
        if ch < "0" or ch > "9":
            return False
    return 1 <= int(text) <= 200
```

```python
# Complexity: Time O(L), Space O(L)
def turn_around(text, rot):
    # the text as seen after turning it 180 degrees, or None if a character cannot be turned
    result = ""
    for i in range(len(text) - 1, -1, -1):
        if text[i] not in rot:
            return None
        result += rot[text[i]]
    return result


# Complexity: Time O(L), Space O(L)
def ocr_valid(t, rot):
    if is_valid_number(t):
        return True
    turned = turn_around(t, rot)
    return turned is not None and is_valid_number(turned)
```

```python
# TASK D: requests arrive as begin(rid, ts) and end(rid, ts), in any
# order for each id. drain(watermark) returns the COMPLETED requests (both
# events seen) as "id begin=b end=e" lines, in order of end time, only
# those with end <= watermark (all of them when no watermark is given).
# Idea: remember the half that arrived first; when the other half arrives
# the request is complete and goes into a min-heap ordered by end time;
# drain() pops the heap while the end time is within the watermark.
# Complexity: Time begin and end are O(log n) when a request completes, drain is O(k log n) for k lines, Sp
ace O(n)
class Server:
    def __init__(self):
        self.begins = {}                    # id -> begin time, waiting for its end
        self.ends = {}                      # id -> end time, waiting for its begin
        self.done = []                      # heap of (end time, id, begin time)

    def begin(self, rid, ts):
        if rid in self.begins:
            raise ValueError("duplicate begin")        # follow-up 2
        self.begins[rid] = ts
        self.check_complete(rid)

    def end(self, rid, ts):
        if rid in self.ends:
            raise ValueError("duplicate end")
        self.ends[rid] = ts
        self.check_complete(rid)

    def check_complete(self, rid):
        if rid in self.begins and rid in self.ends:
            begin_time = self.begins.pop(rid)
            end_time = self.ends.pop(rid)
            if end_time < begin_time:
                raise ValueError("ends before it begins")      # follow-up 2
            heapq.heappush(self.done, (end_time, rid, begin_time))

    def drain(self, watermark=None):
        lines = []
        while self.done and (watermark is None or self.done[0][0] <= watermark):
            end_time, rid, begin_time = heapq.heappop(self.done)
            lines.append(str(rid) + " begin=" + str(begin_time) + " end=" + str(end_time))
        return lines
```

```
# ---- main question: tests ----
```

```
ROT = {"0": "0", "1": "1", "6": "9", "8": "8", "9": "6"}


def test_main_question():
    assert snake_to_camel("make__it_camel_") == "makeItCamel"
    assert snake_to_camel("__a__b_") == "aB"
    assert snake_to_camel("") == ""
    assert format_thousands(1234567) == "1,234,567"
    assert format_thousands(-1000) == "-1,000"
    assert format_thousands(999) == "999"
    assert format_thousands(0) == "0"
    assert ocr_valid("196", ROT)                  # as written 196 is fine
    assert ocr_valid("961", ROT)                  # turned around it reads 196
    assert ocr_valid("09", ROT)                   # a leading zero, but turned around it reads 60
    assert not ocr_valid("201", ROT)              # too big, and the 2 cannot be turned
    assert not ocr_valid("0", ROT)
    assert not ocr_valid("", ROT)
    s = Server()
    s.begin("a", 1)
    s.begin("b", 2)
    s.end("b", 3)
    s.end("a", 5)
    assert s.drain(3) == ["b begin=2 end=3"]      # only what ended by the watermark
    assert s.drain() == ["a begin=1 end=5"]       # the rest
    assert s.drain() == []
    s.end("c", 9)                                 # the end can arrive before the begin
    s.begin("c", 4)
    assert s.drain() == ["c begin=4 end=9"]


# ======================================================
# FOLLOW-UP 1: Why not print as soon as a request completes in the follow-up?
# ======================================================
# Answer: requests complete in any order. A request that ends at time 9
# might complete (be reported) before one that ended at time 5, so printing
# at once would break the order of end times. It is only safe to print
# everything up to a watermark: a time before which no more completions can
# arrive. That is why drain() takes a watermark.


# ======================================================
# FOLLOW-UP 2: How do you handle bad input (duplicate begin, end before begin, never-completed requests)?
# ======================================================
# ---- follow-up 2: tests ----
# Answer: a duplicate begin or end, or an end time before the begin time,
```

```
# raises ValueError (see Server). Requests that never complete stay in
# `begins` or `ends`; report them separately or drop them after a timeout.
def test_follow_up_2():
    s = Server()
    s.begin("a", 1)
    for call in ("duplicate", "backwards"):
        try:
            if call == "duplicate":
                s.begin("a", 2)
            else:
                s.end("a", 0)
            raise AssertionError("should fail")
        except ValueError:
            pass


# ======================================================
```

```python

# FOLLOW-UP 3: Rotation map provided at runtime with extra pairs
# ========================================================
# Answer: ocr_valid already takes the table as an argument, so any
# extra pair (for example "2" and "5" turning into each other) just goes
# into the dictionary.
# ---- follow-up 3: tests ----
def test_follow_up_3():
    extended = dict(ROT)
    extended["2"] = "5"
    extended["5"] = "2"
    assert ocr_valid("102", extended)            # as written 102 is fine
    assert ocr_valid("501", extended) is True    # turned: reversed "105" -> "102"... valid


# ========================================================
# FOLLOW-UP 4: The tracker must bound its memory
# ========================================================
# Answer: half-finished requests (only a begin or only an end) are the
# memory risk: drop the ones older than a timeout, and keep the heap small
# by draining often. Duplicate checks need the ids, so keep them only
# for the pending requests.


# ========================================================
# FOLLOW-UP 5: Another locale: a different separator or grouping (Indian style 12,34,567)
# ========================================================
# ---- follow-up 5: function ----
# Answer: make the separator and the group sizes parameters. The lowest
```

```python
# group may differ from the others (3 digits, then groups of 2 in the
# Indian system). The loop peels one group from the right each time.
# Complexity: Time O(d), Space O(d)
def format_grouped(n, sep=",", first=3, rest=3):
    sign = ""
    if n < 0:
        sign = "-"
    digits = str(abs(n))
    groups = []
    size = first
    while len(digits) > size:
        groups.append(digits[len(digits) - size:])
        digits = digits[:len(digits) - size]
        size = rest
    groups.append(digits)
    groups.reverse()
    return sign + sep.join(groups)


# ---- follow-up 5: tests ----
def test_follow_up_5():
    assert format_grouped(1234567) == "1,234,567"
    assert format_grouped(1234567, first=3, rest=2) == "12,34,567"
    assert format_grouped(1234, sep=".") == "1.234"
    assert format_grouped(-1234567, first=3, rest=2) == "-12,34,567"


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    test_follow_up_5()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FULL QUESTION TEXT FROM THE SITE

You are given several small coding tasks. Implement each as described.

Task A — Convert snake_case to camelCase

Write a function that converts an identifier from snake_case to camelCase.

Input: a string s containing lowercase letters a-z and underscores _ .
Output: the camelCase version.
Rules/edge cases:
Consecutive underscores should be treated as a single word separator (ignore empty parts).
If s starts/ends with underscores, ignore those underscores.
The first word remains lowercase; each subsequent word is capitalized.

Example:

"hello_world" -> "helloWorld"
"__my__var__name_" -> "myVarName"
Task B — Format an integer with thousands separators

Write a function that formats a (possibly negative) integer with commas as thousands separators.

Input: an integer n (may be large; assume it fits in 64-bit signed integer).
Output: a string representation using commas every 3 digits from the right.

Examples:

123 -> "123"
1234 -> "1,234"
-12345678 -> "-12,345,678"
Task C — Validate an OCR string under 180° rotation

An OCR system outputs a string t that may contain digits and a few letters. You need to decide whether it can represent an integer between 1 and 200 inclusive under either normal reading or after a 180-degree rotation.

Define a rotation mapping rot(c) for characters; rotation is valid only if every character in the string has a mapping.
Rotating a whole string means:

reverse the string, and
replace each character c with rot(c) .

Return true if either:

t as-is is a valid base-10 integer in [1, 200] , OR
rotate180(t) is a valid base-10 integer in [1, 200] .

Assume the rotation map is provided as a dictionary (examples from the interview note included):

'6' <-> '9'
plus any additional pairs given at runtime (e.g., 'h' -> '4' , 'L' -> '7'). Characters not in the map are invalid for rotation.

Examples (assuming map includes 6<->9, 0->0, 1->1, 8->8):

"69" rotates to "69" and is valid (69 in range) -> true
"201" is out of range, rotate may or may not help -> depends on mapping -> compute
Task D — Track request begin/end times and print in a required order

You are implementing a Server class that receives events for requests. Each request has a unique requestId.

FULL QUESTION TEXT FROM THE SITE

You must implement two methods:

begin(requestId, timestamp)
end(requestId, timestamp)
Part 1

Whenever both timestamps for a request are known, print/log a line:

requestId begin=<beginTs> end=<endTs>

Assumptions:

Events may arrive out of order (an end can arrive before begin).
There may be multiple in-flight requests.
Follow-up

Instead of printing immediately when a pair is complete, the server must always print completed requests in increasing order of end time (earliest end printed first). If multiple have the same end time, break ties by requestId.

Design the data structures and logic to support this ordering efficiently.